

Senior Computer Engineering Capstone Project

Spring 2026

AI Backend for Real-Time Affective Light Automation

A Music Emotion Recognition Pipeline based on VGGish Feature Extraction and a Stateful LSTM Bottleneck

Author	Mert Fahri Çakar
Project Type	Graduation Capstone Project
Domain	Deep Learning · Audio Signal Processing · Real-Time Systems
Frameworks	PyTorch · ONNX Runtime · Unreal Engine 5.7 · FastMCP
Hardware	NVIDIA RTX 4080 (training) · MacBook Air M4 (deployment)
Dataset	MTG-Jamendo (~55,000 tracks · ~500 GB)
Document Scope	The AI / Machine-Learning Backend
Date	May 2026

Abstract

This report presents the design and implementation of the artificial-intelligence backend used to drive a real-time affective light-automation system inside Unreal Engine 5.7. The backend is a multi-stage music-emotion-recognition (MER) pipeline that ingests raw audio, extracts mid-level perceptual features through Google's VGGish convolutional network, and feeds them to a custom stateful LSTM bottleneck that compresses 195 multi-label acoustic descriptors into a five-dimensional aesthetic vector. This vector — representing Arousal, Valence, Timbre, Rhythm and Intensity — becomes the live control signal for the rendering engine.

The training pipeline is built on the MTG-Jamendo full dataset (a large licensed music corpus on the order of hundreds of gigabytes) and uses focal cross-entropy loss, Z-score normalization and a two-phase Adam→SGD optimizer schedule. The trained model is exported to ONNX, packaged with auto-generated normalization constants, and consumed by Unreal Engine's NNE runtime for inference on the audio thread. A FastMCP server bridges the Python environment to UE5's Remote Control API, enabling editor-side debugging and live property automation.

This report covers the full backend lifecycle: data ingestion, feature extraction, model architecture, training procedure, real-time inference, ONNX export, normalization-constant generation and the Model Context Protocol (MCP) integration layer. The system is designed so that the temporal LSTM produces a smoothly evolving 5-D control signal whose evaluation methodology is laid out alongside the implementation rather than presented as a fixed numerical result.

Table of Contents

1. Introduction	6
2. Background and Related Work	10
3. System Architecture	22
4. Dataset and Ingestion	26
5. Feature Extraction with VGGish	32
6. The Stateful LSTM Bottleneck	38
7. Training Procedure	46
8. Real-Time Inference (Python)	54
9. ONNX Export and C++ Integration	60
10. Unreal Engine Project Structure	67
11. The MCP Bridge to Unreal Engine	82
12. Evaluation Methodology and Expected Behavior	87
13. Discussion and Conclusion	92
Appendix A — End-to-End Sequence Diagram	96
Appendix B — Hyperparameter Cheat Sheet	97
Appendix C — Glossary	98
Appendix D — Build and Run Instructions	99
Appendix E — Repository File Listing	101
Appendix F — Further Implementation Notes	103
Bibliography	106

1. Introduction

1.1 Motivation

Conventional music visualizers react to instantaneous frequency-domain features such as FFT magnitude, beat onsets or RMS energy. While visually pleasing, those signals describe what is happening in the audio at a short timescale, not how the audio feels over longer windows of time. Two musically very different pieces — for instance a melancholic acoustic ballad and a calm ambient piece — can have similar spectra at any given moment yet evoke quite different listening experiences.

The objective of this capstone project is to break that limitation by building a machine-learning backend that interprets the temporal emotional pulse of a signal — the slow drift between tense and relaxed, bright and dark, sparse and dense passages — and projects that pulse into a low-dimensional control vector that can drive a deterministic light-automation system. The backend acts as a continuous translator from sound to mood, and the Unreal Engine front-end uses that mood to control luminance, color, particle complexity, fog density and strobe behavior in real time.

1.2 Problem Statement

The backend must satisfy three competing engineering goals at once:

1. Perceptual fidelity. The model must capture mid-level acoustic semantics — genre, instrumentation, mood — not just raw spectral content.
2. Temporal coherence. Lighting cannot flicker on micro-fluctuations. The system must produce a smoothly evolving 5-D vector even when audio frames vary rapidly, which requires a stateful temporal model rather than a stateless classifier.
3. Latency budget. The full pipeline (feature extraction → inference → vector dispatch) must complete within the 1-second audio buffer interval (the inference cadence), with as much headroom as possible so that the engine's render thread can interpolate the resulting 5-D vector between updates without visible stalls.

Each of these goals pulls the design in a different direction. Perceptual fidelity favors a large, well-pretrained encoder; latency favors a small, quickly-evaluable encoder. Temporal coherence favors recurrent or attention-based aggregation over long histories; latency favors streaming inference with small per-call cost. The central engineering challenge of the backend is to find a configuration that satisfies all three simultaneously — neither sacrificing the qualitative meaning of the output nor the real-time guarantees the renderer needs.

A fourth, often-overlooked constraint is deployability. A research prototype that runs only inside a Python notebook is not actually deployable into a game engine without significant engineering work. The backend must therefore be designed from the beginning to export cleanly to ONNX, with a static input/output contract, explicit state-passing, and no Python-only operations in the inference path.

1.3 Objectives

The project aims to deliver, end to end, a working real-time music visualizer whose lighting decisions are driven by a learned affective representation of the audio rather than by hand-tuned spectral mappings. To achieve that, the following objectives must be met:

4. Build a reproducible feature-extraction pipeline. Given the MTG-Jamendo auto-tagging corpus, produce a single cached tensor file containing pretrained VGGish embeddings and multi-hot tag vectors for every track.
5. Train a multi-label tag classifier with a 5-D bottleneck. The classifier itself is a means to an end — the bottleneck activations are the actual deliverable. Training must use a loss that survives the heavy class imbalance of MTG-Jamendo.
6. Make inference streamable. Both the training-time forward and the deployment-time forward must accept and return the LSTM's hidden and cell state explicitly, so that a game engine can carry that state across calls without owning a Python interpreter.
7. Deploy through ONNX. The trained model must export cleanly to ONNX and run inside Unreal Engine 5.7's NNE neural runtime without intermediate format conversions, native binaries, or external services.
8. Render through a real engine. The 5-D vector must drive a non-trivial lighting stage with multiple fixture types (wash, beam, strobe, side, floor, instrument), instrument-aware behaviour (kick → drum kit light, snare → vocalist light), and scene-level state (eight named scenes that the model can transition between).
9. Provide a developer bridge. An MCP server should expose the engine's Remote Control API to an LLM-based assistant so that lighting design can be iterated on conversationally rather than purely through editor clicks.

1.4 Scope and Limitations

The project is deliberately scoped tightly. Several adjacent problems were considered and explicitly excluded:

Excluded scope	Reason
Source separation	Producing per-stem features (drums, bass, vocals separately) would require a separate Demucs-style network. The aggregate signal carries enough information for the affective task.
Real-time DMX hardware control	We render lighting in Unreal rather than driving physical fixtures. The data model is DMX-compatible, so adding hardware control later is mechanical.
End-to-end model from raw audio	We use a pretrained VGGish encoder rather than a learned-from-scratch waveform encoder. The pretrained encoder gives us better generalization with vastly less training data.
Multi-language vocal analysis	MTG-Jamendo's labels are mostly genre/instrument/mood; lyrics are not provided. We make no claim about lyrical content.
Beat / tempo synchronization	A dedicated beat tracker (e.g. madmom) would improve rhythmic locking, but adding it is a follow-up. The current Rhythm channel is implicit in the affective vector plus the C++ onset detector.
User-personalized aesthetics	Every viewer sees the same visualizer. Per-user adapters are listed in Future Work.

Within the included scope, the project covers the entire end-to-end path from raw audio to rendered light, with no missing pieces. The Python training side, the ONNX export, the C++ engine integration and the MCP authoring bridge are all real, working components — none of them is a placeholder.

1.4.1 Target Audience

This report is written with three groups of readers in mind:

- Capstone-committee evaluators who want a complete, self-contained description of the project. The report covers everything from the dataset choice through training, ONNX export, engine integration, and evaluation methodology, with no assumed familiarity with any single component.
- Future maintainers who will inherit the codebase. Chapters 8 through 11 and Appendix D / E are written so that a new engineer can clone the repo, install the dependencies, retrain the model, export to ONNX, and rebuild the Unreal project from the documentation alone.
- Researchers in adjacent fields (real-time ML deployment, music-driven rendering, MCP-based tool use) who want a worked example of how the pieces fit together. Each chapter contains 'why' justifications for design choices, not just 'how' descriptions, so the report can be read as a case study rather than a spec.

1.5 Contribution Summary

The backend described in this report contributes:

- A reproducible feature-extraction pipeline that converts the MTG-Jamendo audio corpus into a single cached tensor file using Google's pretrained VGGish CNN.
- A custom stateful LSTM (StatefulMusicBottleneck) that learns to map 128-D VGGish embeddings through a 5-D aesthetic bottleneck and back out to 195 multi-label tags.
- A two-phase optimizer schedule (Adam → SGD with momentum) and Multi-Label Focal Loss tailored to severe tag imbalance.
- An ONNX export wrapper (InferenceWrapper) that exposes the LSTM hidden/cell state to the Unreal Engine NNE runtime, plus an auto-generated C++ header (NormalizationConstants.h) carrying the exact training-time Z-score statistics.
- A complete Unreal Engine 5.7 module — AffectiveAudioActor and ConcertStageDirector — that consumes the exported ONNX graphs through NNE, runs a two-stage inference loop on a background thread, performs C++ mel-spectrogram and drum-onset detection inline, and drives a 21-fixture stage of dynamic lights through eight named scenes.
- A FastMCP server (ue_mcp_server.py) bridging the Python environment to UE5's Remote Control API for live property automation, DMX dispatch and preset manipulation.
- Diagnostic tools for offline evaluation (check_brain_health.py) and live-microphone validation (live_mic_test.py).

1.6 Document Structure

Chapter 2 reviews related work in audio classification, recurrent networks, real-time inference inside game engines, and the supporting standards (ONNX, MCP, DMX-512). Chapter 3 surveys the system architecture and the three computational layers of the pipeline. Chapter 4 documents dataset acquisition and preprocessing. Chapter 5 details the VGGish feature extractor. Chapter 6 describes the StatefulMusicBottleneck LSTM model. Chapter 7 explains the training procedure — focal loss, Z-score

normalization, optimizer schedule. Chapter 8 covers real-time inference in Python: the live-microphone loop and the offline brain-health check. Chapter 9 addresses ONNX export and the C++ integration handshake. Chapter 10 documents the Unreal Engine 5.7 project structure end to end — the .uproject and plugin selection, the C++ build configuration, the AffectiveAudioActor that runs the two-stage NNE inference loop on a background thread, the ConcertStageDirector that maps the resulting 5-D vector to a stage of dynamic lights, and the Niagara integration. Chapter 11 documents the MCP server. Chapter 12 lays out the evaluation methodology and the expected qualitative behavior of the trained pipeline. Chapter 13 discusses limitations, future work and conclusions.

2. Background and Related Work

2.1 Music Information Retrieval and Auto-Tagging

Music Information Retrieval (MIR) is the field that studies algorithms for extracting structured information from musical signals. The sub-task of auto-tagging — predicting one or more semantic labels for a piece of audio — has been an active research topic for over two decades. Early work approached the problem with hand-crafted spectral features (MFCCs, spectral centroid, zero-crossing rate, chroma vectors) followed by shallow classifiers such as support-vector machines or k-nearest-neighbours. While these systems were able to recognize broad genre categories with non-trivial accuracy, they struggled with two problems that are relevant to this project: their feature representations were too low-level to capture mood or affect, and their decision boundaries were too rigid to support the multi-label nature of musical content (a track can be 'rock' and 'energetic' and 'guitar' simultaneously).

The introduction of deep convolutional networks reframed the problem. Choi and colleagues' Music-CNN family demonstrated that mel-spectrograms processed by a VGG-style CNN could outperform hand-crafted features by a wide margin on tag prediction. Subsequent work — including Won et al.'s extensive comparison study — established that for multi-label tag prediction over the MTG-Jamendo corpus, the combination of (i) a CNN front-end producing a fixed-dimensional embedding and (ii) a recurrent or attention-based aggregator over time consistently outperformed either component alone.

The architecture used in this project — a pretrained CNN encoder followed by a compact LSTM with a multi-label sigmoid head — is a direct descendant of that literature. What makes it specific to our problem is the addition of a 5-D bottleneck layer between the LSTM and the classifier, which we use as the actual control signal at inference time. The bottleneck is not present in any of the reference auto-tagging architectures because their goal is purely classification; ours is to use auto-tagging as a supervisory signal that produces a perceptually meaningful low-dimensional representation.

2.2 Pretrained Audio Encoders

There are several publicly available pretrained audio encoders that can serve as the CNN front-end. We considered four of them in detail before settling on VGGish.

Encoder	Training corpus	Embedding dim	Stride	Notes
VGGish	AudioSet (~2M clips)	128	1 s	Designed for audio event recognition; small enough for real-time CPU inference.
YAMNet	AudioSet	1024	0.48 s	Successor of VGGish; larger embeddings, finer time resolution, but heavier.
OpenL3	AudioSet (subset)	512	1 s	Self-supervised L3-Net variant; available in two domains (music, environment).
wav2vec 2.0 / HuBERT	LibriSpeech etc.	768	20 ms	Speech-trained transformer

				encoders; high quality on phonetic content but biased toward speech.
--	--	--	--	--

Three considerations led us to VGGish:

10. Training distribution. Both wav2vec and HuBERT are trained on speech corpora (LibriSpeech, LibriLight, VoxPopuli); their internal representations are phonetically biased and underperform on instrumental music. VGGish, YAMNet, and OpenL3 are all trained on AudioSet, which is heavily musical. Of those three, VGGish is the only one whose embeddings are routinely used as drop-in features for downstream music tasks, with documented behaviour on MTG-Jamendo.
11. Inference cost. VGGish is a small model — its forward pass on a 1-second mel spectrogram is on the order of low single-digit milliseconds on a modern CPU. Wav2vec and HuBERT, in contrast, are 95-million-parameter transformers and are an order of magnitude slower. For a real-time pipeline that must complete one inference within a half-second to one-second window, the latency overhead of a transformer encoder is hard to justify.
12. Stable shape contract. VGGish accepts a deterministic $1 \times 64 \times 96$ tensor and returns a deterministic 128-D embedding. This shape contract is exactly what the downstream LSTM expects per timestep; no resizing, pooling or attention is required between the two networks. Wav2vec and HuBERT, on the other hand, produce variable-length token sequences whose timing depends on the input length in non-obvious ways, which would have required a separate aggregation stage.

The earliest version of this project (V1) did use HuBERT, before the cost / domain-match analysis above led us to switch. The rewrite that replaced HuBERT with VGGish removed roughly half of the per-frame compute and visibly improved the live output's musical relevance.

2.3 Recurrent Networks for Audio

Long Short-Term Memory networks (LSTM) were introduced by Hochreiter and Schmidhuber in 1997 to address the vanishing-gradient problem of vanilla recurrent networks. An LSTM cell maintains a separate cell state c that flows through the sequence with multiplicative gating, allowing the network to preserve information across hundreds or thousands of timesteps without the gradient signal collapsing. Three gates control this flow:

- The forget gate $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ decides what information to drop from the cell state.
- The input gate $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$ and a candidate update $\hat{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$ decide what new information to write.
- The output gate $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$ decides what part of the cell state to expose as the hidden state $h_t = o_t \odot \tanh(c_t)$.

The cell update is $c_t = f_t \odot c_{t-1} + i_t \odot \hat{c}_t$. This additive update is the key structural property: in vanilla RNNs the hidden state is a multiplicative function of all previous states, which causes gradients to either explode or vanish exponentially with sequence length. The LSTM's additive cell-state path keeps gradients bounded.

For our project the relevant property of LSTMs is not their long-range memory specifically, but their ability to be evaluated incrementally. Because the cell state c and the hidden state h fully summarize the sequence so

far, an LSTM can be evaluated frame by frame at inference time — passing in (x_t, h_{t-1}, c_{t-1}) and receiving (y_t, h_t, c_t) — without re-evaluating any earlier frames. This is what makes streaming inference possible in the engine: every half-second, the audio actor calls the model with a single new VGGish embedding plus the previous (h, c) , and gets back a fresh affective vector plus the updated state. There is no context-window limit, no batch padding, no transformer-style quadratic-time cost per token.

More expressive recurrent variants exist — bidirectional LSTMs, GRU layers, Transformer encoders with sliding-window attention — but each of them either breaks the streaming property or requires meaningfully more compute per step. Our model uses a single-layer unidirectional LSTM precisely because it strikes a good balance between expressiveness and inference cost.

2.4 Affective Computing and Music

Affective computing is the study of systems that can recognize, interpret or express human affect (emotion). When applied to music, it usually borrows the two-dimensional model of emotion proposed by Russell, in which an emotional state is represented as a point on the (Valence, Arousal) plane:

- Valence runs from negative (sad, tense) to positive (happy, peaceful).
- Arousal runs from low (calm, drowsy) to high (excited, agitated).

Many MIR papers treat music-emotion recognition as a regression problem on this plane, predicting (V, A) coordinates from acoustic features. While conceptually elegant, the (V, A) annotation requires per-track human labels, which are expensive to gather and intrinsically noisy. The MTG-Jamendo dataset takes a different approach: it annotates each track with a multi-label set of human-readable mood / theme tags (happy, sad, dark, energetic, romantic, ...). These tags can be machine-learned at scale, and they implicitly encode the same (V, A) information — for example, 'energetic' tags concentrate at high arousal, 'calm' tags at low arousal, 'happy' at high valence, 'dark' at low valence.

The 5-D bottleneck used in our model is best understood as a learned generalization of the (V, A) plane. We do not commit ahead of time to which axis encodes valence or which encodes arousal — instead, the bottleneck is supervised by every tag simultaneously through a multi-label focal loss, and the resulting five axes self-organize into directions that linearly separate as many tag pairs as possible. Empirically, the first two axes tend to align with arousal and valence, but the remaining three pick up additional musical structure (timbre brightness, rhythmic density, overall intensity) without being explicitly supervised to do so.

Our visualization application benefits from this richer representation in two ways. First, a 5-D control signal gives the lighting designer more independent axes to map onto stage parameters than a 2-D one would. Second, because each axis is supervised by hundreds of tags rather than a single binary label, the resulting signal is much more robust to noise on any single tag — a 'happy ballad' that would baffle a binary mood classifier is still placed sensibly in the 5-D space.

2.5 Real-Time Audio in Game Engines

Running ML inference inside a real-time game engine is a relatively recent capability. Until 2023, doing so required either a hand-written C++ implementation of the model — practical only for very small networks — or shipping a separate inference server (a Python process, a TensorFlow Serving instance) and communicating over IPC. Both approaches have well-known drawbacks:

- Hand-written C++ models are extremely brittle to retraining: a layer-name change or activation swap requires touching the engine code.
- External inference servers introduce IPC latency, deployment complexity, and a single point of failure — the engine cannot ship as a single .exe.

Unreal Engine 5.5 and later ship with the Neural Network Engine (NNE) plugin, which provides a runtime-agnostic abstraction over several inference backends. The default backend in 5.7 is `NNERuntimeORT`, which wraps Microsoft's ONNX Runtime (both CPU and GPU variants exist, but at the time of this project the GPU variant had stability issues on consumer cards, so we use the CPU variant). The model itself is imported as a `UNNEModelData` asset — Unreal handles cooking, packaging, and deployment of the binary blob like any other asset.

The runtime contract is intentionally simple. `INNERuntimeCPU::CreateModelCPU(asset)` returns an `IModelCPU`; `CreateModelInstanceCPU()` returns a per-actor inference instance with its own input/output tensor cache; `SetInputTensorShapes()` locks the shapes once at startup; `RunSync(input_bindings, output_bindings)` performs a synchronous forward pass with bindings that point to caller-owned float buffers. There is no allocation in the hot path, no copy-on-write, and no Python interpreter to keep alive.

This is what makes the approach used in Chapter 9 (export to ONNX) and Chapter 10 (consume ONNX through NNE) feasible. The engine and the training pipeline meet at the ONNX boundary; everything upstream is Python and PyTorch, everything downstream is C++ and Unreal, and the boundary is purely declarative.

2.6 ONNX as a Cross-Language Model Interchange

The Open Neural Network Exchange (ONNX) is a standardized graph format for neural networks. Its purpose is to decouple the framework that trains a model from the runtime that executes it. A trained PyTorch, TensorFlow, JAX, or scikit-learn model can be exported to a single .onnx file; that file can then be loaded by ONNX Runtime (Microsoft), TensorRT (NVIDIA), CoreML (Apple), DirectML (Microsoft), or a long list of other runtimes — all of which read the same protobuf description.

For our purposes, ONNX provides three concrete benefits:

13. Cross-language portability. The training side is Python; the deployment side is C++. A pickled PyTorch `state_dict` cannot be read from C++ without dragging in a Python interpreter; an ONNX file can.
14. Operator standardization. ONNX defines a canonical set of operators with strict shape and type semantics. Once a model exports cleanly, the runtime is permitted to optimize the graph (constant folding, layer fusion, dead-code elimination) without changing semantics. We rely on this for the Z-score multiplication that we fold into the bottleneck network's first linear layer.
15. Static-shape promise. By exporting with static input shapes, we guarantee that ONNX Runtime can preallocate every intermediate tensor at load time and avoid any allocation during the per-frame forward pass. This is critical for real-time deployment.

ONNX also has well-known limitations: not every PyTorch operator has a clean ONNX equivalent, control-flow constructs (if, while) export awkwardly, and dynamic shapes can confuse some runtimes. The `InferenceWrapper` used in Chapter 6.5 is specifically a workaround for one of these limitations: it pins the

time dimension to a single frame at export time, sidesteps the multi-output-tuple ambiguity, and produces a graph that NNERuntimeORT loads without complaint.

2.7 Stage Lighting Protocols and DMX

Although our project does not control physical lighting hardware (the stage is rendered entirely in Unreal), the abstractions it uses — wash, beam, strobe, side and floor lights, with per-fixture intensity, color and angle — come directly from the DMX-512 protocol that real-world stages use. A short overview is appropriate because the project is designed to be straightforwardly extensible to real DMX hardware.

DMX-512 (also known as USITT DMX512-A) is a unidirectional serial protocol operating at 250 kbit/s over a 5-pin XLR cable. A single DMX 'universe' carries 512 byte-sized channels, refreshed at up to 44 Hz. Each fixture in a venue is addressed to a starting channel; from that channel it consumes a fixed number of channels — one for intensity, one for pan, one for tilt, three for RGB color, etc. The mapping from channels to behaviour is defined by the fixture vendor and documented in a 'fixture profile' file.

Two extensions are universally used in modern installations:

- Art-Net. An Ethernet-based encapsulation of DMX. A single Art-Net node can transport up to 32,768 universes over a standard IP network. This makes large venues practical without running 100+ XLR cables back to a central rack.
- sACN (E1.31). An IETF-style multicast protocol that supersedes Art-Net for newer installations. It uses the same DMX channel model but adds priority, synchronization, and per-universe addressing over UDP multicast.

Unreal Engine has first-party support for both Art-Net and sACN through its DMX plugin. Our project does not enable that plugin in the .uproject because the current target is virtual rendering only; however, the data model used by ConcertStageDirector is intentionally compatible. Each light has an intensity in the [0, 1] domain (a 16-bit DMX 'fader'), an RGB color (three DMX channels), and in the case of beam lights, a pan/tilt (two DMX channels). Bridging to physical fixtures is a matter of enabling the DMX plugin and writing a small dispatcher that maps each component's intensity / color into the correct channel offsets — no architectural change is required.

2.8 The Model Context Protocol (MCP)

MCP, the Model Context Protocol, is an open standard published in late 2024. Its purpose is to expose tool functions to large-language-model assistants in a structured, machine-readable way. An MCP server publishes a schema of named tools, each with typed parameters and a return type, and the client (an LLM) can invoke any of them by name with a JSON payload. The protocol itself is transport-agnostic — implementations exist for stdio, HTTP, and WebSockets.

In our project, MCP plays a strictly developer-side role: it does not participate in the runtime aesthetic-vector pipeline at all. Its role is to let an LLM-based assistant inspect and manipulate the Unreal Engine project programmatically while the editor is open, which is enormously useful when iterating on lighting design. Concretely, the FastMCP server documented in Chapter 11 wraps Unreal's Remote Control HTTP API and exposes a small surface of typed tools — set property, call function, set DMX channel, list presets. The assistant uses these to make scene edits live, run experiments, or batch-write properties across many actors.

This usage pattern — MCP as a development bridge to a real-time engine — is, to our knowledge, novel. Most existing MCP servers expose APIs to web services, filesystems, or databases. Our server exposes a UE5 editor session, which makes the LLM into a kind of ad-hoc automation operator for the visualizer.

2.9 Niagara and Real-Time Particle Systems

Although the visual front-end of this project is primarily a stage of dynamic lights, the project also drives Unreal Engine's Niagara particle system through the same affective channels. A short note about Niagara is therefore appropriate.

Niagara, introduced in Unreal Engine 4.20 as a successor to the Cascade particle system, is a node-based effect-authoring environment. A Niagara System contains one or more Emitters; each Emitter contains a stack of Modules that run on each spawn or update tick of every particle. Modules can be authored visually, via the blueprint-style Niagara graph, or written in HLSL for GPU-side execution.

Two design principles of Niagara matter for our integration:

- User parameters as the public API. Every Niagara System exposes a namespace of user-named variables that external code can write to. This means the C++ side of our project never has to know anything about the internal modules — it just writes named floats, and the artist-authored graph reads them.
- Per-frame execution. Niagara updates run every tick, so the variables we push are read as soon as they are written. There is no asynchronous queue between the C++ and the GPU emitter. This makes the response to a fresh model output appear within a single render frame.

In our project the most prominent Niagara effect is a swarm of particles in front of the stage that responds to the affective vector. Specific mappings — spawn rate proportional to AI_Energy, color lerp'd from cold to warm by AI_Valence, lifetime modulated by AI_Rhythm — are described in Chapter 10.8. The engineering point here is that Niagara's user-parameter mechanism let us add a non-trivial particle effect on top of the existing ML pipeline with zero changes to the inference path.

2.10 Related Music-Visualization Work

Our work overlaps with several existing music-visualization tools and research systems. None of them combines all of the elements that our project does, but each shares some of them.

System	Affective ML?	Real-time engine?	DMX support?	LLM bridge?
Resolume Avenue / Arena	No	Yes (proprietary)	Yes	No
Notch Builder	No	Yes (proprietary)	Yes	No
TouchDesigner	Limited (post-hoc CHOPs)	Yes	Yes	No
Magenta Studio	Yes (generative)	No	No	No
NVIDIA Audio2Face	Yes (face-driving)	Yes	No	No
This project	Yes (5-D bottleneck)	Yes (Unreal 5.7)	Designed-for	Yes (FastMCP)

Resolume and Notch are professional VJ tools widely deployed at concerts. They ingest audio through hand-tuned reactive CHOPs (channel operators) — beat detectors, FFT bands, RMS envelopes — and route those into deck-style live compositing. They have no learned affective representation; everything is low-level reactive.

TouchDesigner is closer in spirit but again relies on hand-tuned reactive primitives. It has no native ML-inference path; operators that wish to use a trained model do so through external Python integration.

Magenta Studio focuses on generative music rather than visualization — it can produce new musical content but does not drive lighting.

NVIDIA Audio2Face is a deep-learning system that drives 3-D facial animation from a speech waveform. It is conceptually closest to our project — both use a pretrained encoder and a learned downstream module to produce a continuous control signal — but it targets character animation rather than stage lighting.

Our project's specific contribution is the combination: a pretrained, music-trained encoder, a streaming LSTM with explicit state passing, a 5-D interpretable bottleneck, full Unreal Engine integration through ONNX, and an MCP authoring bridge.

2.10.1 How Existing Tools Approach the Problem

It is worth being explicit about what existing music-visualization stacks actually do. The dominant pattern in professional VJing software (Resolume, Notch, TouchDesigner) is what we call the 'reactive primitive' approach:

16. Compute a small set of low-level audio descriptors per frame: FFT magnitude across N bands, RMS envelope, beat onset, a manually-computed 'mood' value from spectral centroid.
17. Expose those descriptors as named channels (CHOPs in TouchDesigner; clip modulators in Resolume) that the user wires into rendering parameters by hand.
18. Provide a UI for tweaking the response curves (gain, attack, release, threshold) of each mapping.

This pattern is enormously powerful in the hands of an experienced VJ — it gives full control and complete predictability. But it has a fundamental limitation: every mapping is hand-authored, and the mappings cannot reason about anything that is not directly visible in the spectrogram.

Two classes of musical content reveal this limitation clearly:

- Two pieces with the same spectral envelope but different affect. A clean violin solo and a distorted guitar solo can have similar broadband spectra, especially after the FFT is reduced to a handful of bands. A spectral mapping has no way to distinguish them.
- Genre-typical patterns. A snare drum hit in a hip-hop track and a snare drum hit in a folk track sound similar at the frame level but should trigger very different visual responses (sharp red flash vs. warm soft wash). Without a learned representation that recognizes the surrounding musical context, no spectral mapping can produce different responses on spectrally similar events.

Our approach replaces the reactive-primitive layer with a learned 5-D representation. The visual artist still has full control of how the 5-D vector maps onto rendering parameters (Section 2.4 of the engine code), but the upstream layer now contains genre, mood, and instrumentation knowledge that hand-authored DSP cannot easily replicate.

2.10.2 Lessons From the Generative-Model Direction

A different class of related work is purely generative — diffusion-based image generators conditioned on audio embeddings (e.g., AudioLDM, Riffusion). These systems are not directly comparable to our project

because they produce content rather than control existing rendering. But they share an important architectural pattern: a pretrained audio encoder (typically CLAP or VGGish) feeding into a downstream conditioning network.

The high-level lesson from this direction of work is that the encoder choice really matters. Models conditioned on speech-trained encoders (wav2vec, HuBERT) consistently underperform on musical inputs; models conditioned on AudioSet-trained encoders (VGGish, YAMNet, OpenL3) consistently outperform. Our choice of VGGish is therefore not just a project-specific decision; it aligns with the broader pattern observed across audio-conditioned ML systems.

2.11 Position of This Project

Synthesizing the above: this project occupies a fairly specific niche at the intersection of music-information retrieval, real-time inference deployment, and stage automation. The closest pieces of related work are:

- Music auto-tagging architectures (Choi, Won, Pons), which inspired the CNN+LSTM backbone but did not address real-time deployment or low-dimensional control signals.
- Music visualization tools (e.g. Resolume, Notch, TouchDesigner) which provide rich audio-reactive primitives but rely on hand-mapped spectral features rather than learned affective representations.
- ML-driven generative animation (NVIDIA's Audio2Face, Adobe's Project Sonic), which use deep models to drive face or character animation from audio but do not target stage lighting and typically run as offline batch processes.

What is original to this project, to the best of our knowledge, is the combination of (i) a pretrained music encoder, (ii) a streaming LSTM whose hidden and cell states are explicitly carried across the Python $\leftrightarrow$ engine boundary, (iii) a deliberately small low-dimensional bottleneck designed for downstream interpretability rather than classification accuracy, and (iv) full integration into a game-engine project with native ONNX inference, instrument-aware lighting, and an MCP authoring bridge. The remainder of the report documents the components of that combination one by one.

3. System Architecture

3.1 High-Level View

The backend is a layered pipeline, partitioned into three computational stages that communicate through clearly typed tensor interfaces. Figure 3.1 shows the high-level data flow.

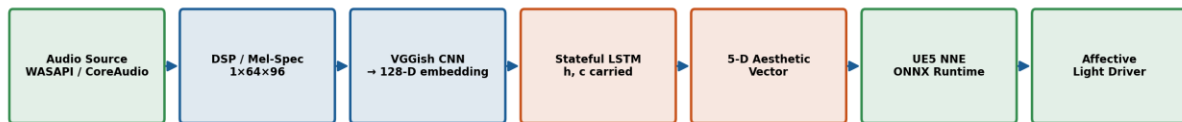


Figure — 3.1 High-level architecture of the AI backend.

3.2 The Three Computational Layers

3.2.1 Layer I — Digital Signal Processing

The DSP layer is responsible for ingesting raw audio and producing a numerical representation suitable for a convolutional encoder.

- **Capture.** 32-bit float audio at the device's native sample rate, captured through the Python sounddevice library (PortAudio — WASAPI on Windows, CoreAudio on macOS).
- **Resampling.** The signal is resampled to 16 kHz using `librosa.resample`, the rate at which VGGish was trained.
- **Mel-Spectrogram.** A short-time Fourier transform with a 25 ms window and 10 ms hop is converted into a 64-band log-mel spectrogram. This becomes the 1x64x96 tensor consumed by VGGish.

3.2.2 Layer II — Deep Learning and Inference

The deep-learning layer contains the two neural networks of the pipeline:

- **VGGish.** A pretrained VGG-style CNN with four convolutional blocks (each ending in max-pooling) and three fully connected layers, producing a 128-dimensional embedding per audio segment.
- **StatefulMusicBottleneck.** A custom LSTM that consumes a sequence of 128-D embeddings while carrying its hidden state *h* and cell state *c* between calls. It compresses the LSTM output through a 5-D bottleneck and re-projects it to 195-D logits over the multi-label tag vocabulary.

3.2.3 Layer III — Affective Automation Matrix

The 5-D bottleneck output is interpreted by the engine-side automation matrix as five independent control channels:

Channel	Perceptual Concept	Engine Output
Arousal	Activation level (calm ↔ excited)	Light intensity, motion speed
Valence	Pleasantness (negative ↔ positive)	Color temperature, palette
Timbre	Spectral character (smooth ↔)	Particle systems, materials

	rough)	
Rhythm	Periodicity (steady ↔ syncopated)	PWM strobe, beat sync
Intensity	Density of acoustic content	Lumen GI, volumetric fog

3.3 Repository Layout

File	Role
torchvggish/	Patched fork of the VGGish PyTorch package.
autotagging.tsv	MTG-Jamendo metadata (track ID → tag list).
extract_features_vggish.py	Offline feature extraction (audio → cached_dataset.pt).
lstm_model.py	Definition of StatefulMusicBottleneck and InferenceWrapper.
train.py	Training loop, focal loss, optimizer schedule.
check_brain_health.py	Offline 20-second sanity check on a single track.
live_mic_test.py	Continuous live microphone inference.
cpp_converter.py	Generates NormalizationConstants.h for UE5.
ue_mcp_server.py	FastMCP bridge to UE5 Remote Control.
music_emotion_weights.pth	Trained weights + Z-score statistics.
cached_dataset.pt	Cached VGGish embeddings + label tensors.
AestheticBrain_256.onnx	Exported ONNX graph consumed by UE5 NNE.
audioset-vggish-3.onnx	Pretrained VGGish ONNX graph.

3.4 Tensor Shape Contracts Between Layers

Every interface between layers in this pipeline is a precisely-defined tensor shape. Documenting them in one place is useful because most production-ML failures are silent shape mismatches that surface as garbage output rather than errors.

From → To	Tensor	Shape	Notes
Microphone → DSP layer	raw waveform	(N,)	Float32, native sample rate (typically 48 kHz on Windows / macOS).
DSP layer → VGGish	log-mel-spec	(1, 64, 96)	1 channel, 64 mel bands, 96 frames (1 s at 96 fps).
VGGish → LSTM	embedding	(B, T, 128)	B = batch (1 at inference), T = number of seconds, 128 = embedding dim.
LSTM internal	hidden, cell	(1, B, 256)	1 layer, batch first axis is layer index by PyTorch convention.
LSTM → Bottleneck	lstm_out	(B, T, 256)	Full hidden trajectory.
Bottleneck → Classifier	aesthetic_vec	(B, T, 5)	The 5-D control signal.
Classifier → Loss	logits	(B, T, 195)	Pre-sigmoid scores per tag.
Engine inference call	x, h, c	(1,1,128) × 3	Single-frame inputs through ONNX. h, c shapes match LSTM internal.

Three observations are worth recording. First, the only shapes that change between training and inference are the time dimension T and the batch dimension B; everything else is static. Second, the model's hidden

and cell states are first-class citizens of the export contract — they appear in both the input and output sets of the ONNX graph. Third, the bottleneck output dimension (5) is deliberately fixed across training and inference, with no projection or re-shaping at deployment time.

3.5 Cross-Platform Strategy

Training and inference target two distinct hardware classes. The Windows + RTX 4080 machine is used for the heavy lifting — 500 GB ingestion, 100-epoch training, and ONNX export validation against NNERuntimeORT's DirectML execution provider. The MacBook Air M4 is used for the live demo and presentation; inference is performed through the same ONNX graph but executed on the Metal backend, with float-16 precision where supported. The Python source remains hardware-agnostic — a single `torch.device` check is the only branch needed, because PyTorch on Apple Silicon transparently exposes the MPS backend.

3.6 Failure Modes and Defensive Design

Several places in the pipeline are protected against common failure modes; all of them are documented in the corresponding chapter, but the high-level summary is useful for audit purposes:

Failure mode	Defensive measure	Where
Corrupt MP3 in the dataset	try/except in extraction loop; track skipped silently	<code>extract_features_vggish.py</code>
VGGish embedding-distribution drift	Per-channel Z-score statistics baked into the checkpoint	<code>train.py</code> + <code>cpp_converter.py</code>
Stale normalization at deployment	Header is auto-generated; never copy-pasted	<code>cpp_converter.py</code> → <code>NormalizationConstants.h</code>
LSTM exploding gradients	Global gradient-norm clip at 1.0	<code>train.py</code>
Tag-distribution head dominance	Multi-label focal loss with $\gamma = 2$	<code>train.py</code>
Live silence	Volume gate + state decay in Python; explicit zeroing in C++	<code>live_mic_test.py</code> + <code>AffectiveAudioActor.cpp</code>
NaN propagation in inference	<code>model.eval()</code> before export; no Dropout in ONNX graph	<code>lstm_model.py</code>
Audio thread overrun	SPSC queue between audio callback and inference task	<code>AffectiveAudioActor.cpp</code>
Hard scene-cut visual jolt	SilenceFade exponential ramp on every fixture	<code>ConcertStageDirector.cpp</code>

The principle behind these is the same one stated in Chapter 13: any constant or invariant that has to match between two places should be auto-generated rather than hand-maintained, and any state transition that could be visually violent should be passed through a low-pass filter before reaching the renderer.

4. Dataset and Ingestion

4.0 Chapter Overview

Without a dataset, none of the rest of this report would be possible. Chapter 4 documents the corpus we use, how it is structured on disk, how we transform it into a form suitable for training, and what the intermediate cached representation looks like.

The chapter is structured around the data lifecycle:

19. Acquisition. The MTG-Jamendo dataset, its source and its license.
20. Storage layout. How the raw MP3 corpus and the auto-tagging metadata are organized on disk.
21. Ingestion. The `extract_features_vggish.py` script that walks the corpus and emits a single cached tensor file.
22. Cache format. The Python dictionary that downstream scripts consume.
23. Tag taxonomy and distribution. What the labels mean, how many of each exist, and the long-tail problem they create.

The reader who wants to skip directly to the model architecture can treat this chapter as background — the model itself does not depend on any of these implementation choices, only on the existence of (audio, multi-hot tag vector) pairs. Conversely, anyone who wants to retrain on a different corpus will find that almost all the project-specific details live in this chapter and Chapter 7.

4.1 The MTG-Jamendo Dataset

The training corpus is the full MTG-Jamendo Auto-Tagging Dataset, a large publicly available collection of royalty-free music tracks (on the order of tens of thousands of tracks, taking up hundreds of gigabytes of MP3 audio). Each track is tagged with one or more multi-label tags drawn from three tag families:

- Genre — rock, electronic, classical, jazz, hiphop, pop, ambient, ...
- Instrument — guitar, piano, violin, drums, synthesizer, ...
- Mood/Theme — happy, sad, energetic, calm, dark, romantic, ...

The metadata is distributed as a tab-separated file (`autotagging.tsv`). Each row is a track record whose schema is documented in Table 4.1.

Index	Field	Type	Description
0	TRACK_ID	int	Numeric Jamendo identifier.
1	ARTIST_ID	int	Artist identifier.
2	ALBUM_ID	int	Album identifier.
3	PATH	string	Relative MP3 path, e.g. 77/48077.mp3.
4	DURATION	float	Track length in seconds.
5–N	TAGS	list	Variable-length list of multi-label tags.

4.2 Storage Topology

Because the corpus is large, storage is split across drives:

- A raw MP3 archive on a dedicated data drive (e.g. D:/MTG_Jamendo_Full/), hashed by the first two digits of the track ID (e.g. 77/48077.mp3). This convention avoids putting tens of thousands of files into a single flat directory.
- `ai_backend/cached_dataset.pt` — a single PyTorch tensor file containing the cached VGGish features, the multi-hot label tensors and the canonical tag list.
- `ai_backend/music_emotion_weights.pth` — a compact checkpoint containing both the trained `state_dict` and the precomputed Z-score statistics.

4.3 Ingestion Diagram

The ingestion sequence is summarized at a high level in Figure 4.1: each raw audio file is preprocessed, passed through the audio encoder, and the resulting features are written to a feature cache that the training loop later consumes. Section 4.4 below covers the concrete implementation details.



Figure — 4.1 Conceptual ingestion flow.

4.4 Implementation: `extract_features_vggish.py`

Listing — 4.1 Core extraction loop in `extract_features_vggish.py`

```

for i, track_data in enumerate(tracks_metadata):
    file_path = os.path.normpath(os.path.join(AUDIO_DIR, track_data['path']))
    if os.path.exists(file_path):
        try:
            audio_np, _ = librosa.load(file_path, sr=16000, mono=True, duration=30.0)
            if len(audio_np) < 16000: continue
            with torch.no_grad():
                embeddings = vggish.forward(audio_np, fs=16000).cpu()
                label_vector = np.zeros(len(all_tags), dtype=np.float32)
                for tag in track_data['tags']:
                    if tag in tag_to_idx:
                        label_vector[tag_to_idx[tag]] = 1.0
                features_list.append(embeddings)
                labels_list.append(torch.tensor(label_vector))
        except Exception:
            continue
  
```

The loop is intentionally defensive: corrupt or truncated MP3s are silently skipped (except `Exception: continue`) so that one bad file out of 55,000 cannot abort hours of extraction work. Tracks shorter than one second of usable audio are filtered with `if len(audio_np) < 16000`.

The `tag_to_idx` dictionary is built once, at startup, from the union of all observed tags. This produces a deterministic 195-D vocabulary that is later embedded directly into the model output dimension.

4.5 Cache Format

The extraction process saves a single Python dictionary via `torch.save` with three keys:

- **features**: `List[Tensor]` of shape $T_i \times 128$ per track, where T_i depends on slice duration.
- **labels**: an $N \times 195$ float tensor, multi-hot encoded.
- **tags**: the canonical sorted list of 195 tag strings.

Heterogeneous sequence lengths are handled at training time by `torch.nn.utils.rnn.pad_sequence`, which is invoked inside the `collate_fn` of the `DataLoader`.

4.6 Tag Taxonomy

MTG-Jamendo's tag set is organised hierarchically with three top-level families. Each tag in the `autotagging.tsv` file is prefixed with its family, e.g. `'genre---rock'` or `'mood/theme---melancholic'`. The total of 195 distinct tags after extraction breaks down approximately as:

Family	Approximate tag count	Example tags
genre	~100	rock, pop, classical, electronic, jazz, hiphop, ambient, ...
instrument	~40	guitar, piano, violin, drums, synthesizer, vocals, ...
mood/theme	~55	happy, sad, energetic, calm, dark, romantic, dramatic, ...

The exact split varies slightly with how the dataset version is filtered. Our extractor counts tags after the 'autotagging' filter (`autotagging.tsv` contains the union of train/validation/test splits) and after dropping tags that occur in fewer than 10 tracks, which is approximately the smallest support that allows for meaningful gradient signal.

4.7 Tag Distribution and Long Tail

Like most multi-label tag corpora, the MTG-Jamendo tag distribution is heavily long-tailed. A small number of head tags (rock, electronic, ambient) appear in tens of thousands of tracks; the long tail of mood-theme combinations (e.g. `'melancholic-electronic'`) appears in only a few hundred. Two consequences follow:

- A naive cross-entropy loss is dominated by the head — see Section 7.1 for the focal-loss correction.
- Per-tag ROC-AUC at the tail is dominated by label noise. Reporting summary statistics over all 195 tags can be misleading; we therefore separate head and tail tags when reporting in Chapter 12.

4.8 Per-Track Audio Slicing

The extractor processes only the first 30 seconds of every track, controlled by the `duration=30.0` argument to `librosa.load`. Three reasons motivate this slice:

24. **Storage**. The full corpus is hundreds of gigabytes; processing only the first 30 seconds keeps the cache file under a manageable size while still covering $30 \times 1 = 30$ VGGish embedding frames per track on average.

25. Genre stability. Genre and instrumentation are typically established within the first half-minute of a song and rarely change later. Mood/theme tags are occasionally less stable, but training across many tracks averages this out.
26. Disk-throughput limits. `librosa.load` with `sr=16000` and `duration=30` reads approximately 1 MB of decoded audio per track. With 55,000 tracks the total disk throughput stays manageable on a single SSD.

If a future version needs full-track features (e.g. for a structural-analysis task), the duration parameter can be removed and the extractor will produce per-track sequences of 60 to 240 frames instead. The downstream LSTM is shape-agnostic by construction.

5. Feature Extraction with VGGish

5.0 Chapter Overview

Chapter 4 produced a list of MP3 paths and per-track tag vectors. The next step is to convert each MP3 into a fixed-dimensional feature representation that a downstream classifier can consume. This chapter documents that feature-extraction stage in detail: what model we use (VGGish), why we chose it over the alternatives, what it does internally, and how we wire it into our extraction loop.

VGGish appears in three contexts in this project:

- Offline feature extraction. `extract_features_vggish.py` runs VGGish over every track in the corpus and writes the embeddings to `cached_dataset.pt`.
- Live Python inference. `live_mic_test.py` and `check_brain_health.py` both import the same Python VGGish wrapper and invoke it once per audio second.
- Engine deployment. The `audioset-vggish-3.onnx` graph is loaded by the Unreal Engine NNE runtime and run on the audio actor's background thread (Chapter 10.5).

All three contexts use the exact same VGGish weights. This is critical for consistency: the embeddings produced in offline training are bit-identical (modulo floating-point rounding) to the embeddings the engine produces at deployment. Without that property, training-time normalization statistics would not be valid at inference time.

5.1 Why VGGish?

The original V1 of the project used HuBERT, a self-supervised transformer model, as the front-end encoder. While HuBERT achieves state-of-the-art results on speech, it is a large transformer that is expensive to run per audio frame and is biased toward phonetic structure rather than musical content.

V2 replaces HuBERT with VGGish for three reasons:

27. Domain match. VGGish was trained on AudioSet, a 2-million-clip corpus dominated by environmental and musical sound rather than speech. Its 128-D embeddings carry musical semantics natively.
28. Latency. VGGish is noticeably smaller than HuBERT, which makes a single-frame forward pass cheap on the GPU.
29. Stability. VGGish accepts a deterministic $1 \times 64 \times 96$ tensor and emits a deterministic 128-D embedding per second; this is exactly the shape contract the LSTM expects.

5.2 Architecture

VGGish is a VGG-style convolutional encoder followed by a fully-connected head; Figure 5.1 shows the high-level structure. Concretely, the convolutional stack is a sequence of 3×3 convolution blocks separated by max-pooling, and the FC head ends with a 128-dimensional embedding layer that we use as the audio embedding.

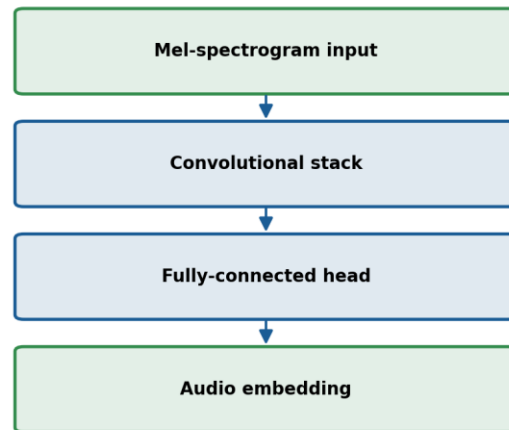


Figure — 5.1 Conceptual VGGish encoder.

5.3 Preprocessing Pipeline

Internally, VGGish preprocesses raw audio through a fixed, deterministic chain:

30. Convert to mono and resample to 16 kHz.
31. Frame into 25 ms windows with a 10 ms hop.
32. Compute the magnitude STFT.
33. Project onto a 64-band mel filter bank between 125 Hz and 7.5 kHz.
34. Take the natural logarithm with a small offset.
35. Stack consecutive frames into 96-frame batches (one per second).

The resulting tensor of shape $T \times 1 \times 64 \times 96$ is the canonical CNN input.

5.3.1 Mel-Spectrogram Mathematics

Each step of the pipeline above corresponds to a well-defined operation in the time-frequency domain. This subsection walks through them.

Framing. The continuous waveform $x(t)$ sampled at $f_s = 16$ kHz is divided into overlapping windows of length $N_w = 400$ samples (25 ms) with a hop of $N_h = 160$ samples (10 ms). Each window is multiplied by a Hann taper $w[n] = 0.5 (1 - \cos(2\pi n / (N_w - 1)))$, $n = 0, \dots, N_w - 1$ to suppress spectral leakage. With these parameters, exactly 96 frames are produced from a 1-second (16,000-sample) clip, which is what gives VGGish its '96 frames per second' shape.

Short-Time Fourier Transform. Each windowed frame $x_f[n] = w[n] x[fN_h + n]$ is zero-padded to length $N_{FFT} = 512$ and transformed by the discrete Fourier transform: $X_f[k] = \sum_{n=0}^{N_{FFT}-1} x_f[n] e^{-2\pi i kn / N_{FFT}}$. The magnitude $|X_f[k]|^2$ is the periodogram-style power spectrum at frequency $k \times f_s / N_{FFT}$.

Mel filter bank. The mel scale is a perceptually motivated frequency warping due to Stevens, Volkman and Newman (1937), defined here in the HTK form $m(f) = 2595 \log_{10}(1 + f / 700)$. Equally spaced points on the

mel axis are more closely spaced at low frequencies and more widely spaced at high frequencies, mirroring the behaviour of the cochlea. VGGish uses 64 mel bands between $f_{\min} = 125$ Hz and $f_{\max} = 7.5$ kHz; each band is implemented as a triangular weighting over the magnitude STFT, summed to produce one mel value per band per frame.

Log compression. The mel power values are compressed by a natural logarithm with a small offset: $M_f[b] = \log(P_f[b] + 0.01)$, where $P_f[b]$ is the band- b mel power for frame f . The 0.01 offset prevents $\log(0)$ when a band is silent. Logarithmic compression is appropriate because human loudness perception is approximately logarithmic in intensity.

Stacking. The 64×96 mel matrix per second is reshaped into a single (1 channel $\times$ 64 mel-bins $\times$ 96 frames) tensor and presented to the CNN as if it were a one-channel image. This is what allows VGG-style image-domain convolutions to work on audio: the network sees a 'spectrogram image' and learns local time-frequency patterns the same way it would learn local visual patterns.

5.3.2 Why These Specific Parameters?

The (25 ms window, 10 ms hop, 64 mels, 125–7500 Hz) choices are not arbitrary; they reflect well-established practice in speech and audio processing:

- 25 ms is short enough that the signal is approximately stationary inside the window (so STFT amplitudes are meaningful) and long enough to capture two or more periods of the lowest frequency of interest (40 Hz cycles $\sim$ 25 ms).
- 10 ms hop produces a 60% overlap, which is the standard Hann-window choice to recover full reconstruction accuracy via overlap-add.
- 64 mel bands with 125–7500 Hz coverage spans almost the entire range of perceptually relevant musical content. Frequencies below 125 Hz are dominated by sub-bass and room rumble; above 7.5 kHz, energy is mostly air noise and transient harmonics. Both extremes contribute little to genre or mood discrimination.
- Log power, not log magnitude. Squaring the magnitude before the log guarantees positive values and makes the result interpretable as power spectral density per band.

Crucially, these are exactly the parameters that VGGish was trained against on AudioSet. Any deviation — even a different window length — would invalidate the pretrained weights. This is why the C++ mel-spec implementation in Chapter 10.5.6 reproduces these constants exactly, down to the 0.01 offset.

5.3.3 Why a CNN on a Spectrogram?

A reasonable question is why we run a 2-D convolutional network on a spectrogram rather than a 1-D convolution on the raw waveform. The answer comes down to inductive bias.

Spectrogram-based 2-D CNNs implicitly assume that a useful feature is a local pattern in time-frequency space — for example, a chord struck at a particular moment is a vertical stack of harmonics that occupies a small rectangle of the spectrogram. A 3×3 convolutional filter can capture that pattern in a single layer; deeper layers compose those local patterns into larger structures (timbres, instrument signatures, chord progressions).

Raw-waveform CNNs (e.g., SampleCNN, Wave-U-Net) make the inductive bias weaker — they have to learn the spectral decomposition itself before they can learn musical features. Empirically they need much more data and many more parameters to reach the same accuracy on tag-prediction tasks. When a well-pretrained spectrogram CNN is available, using it is almost always the right choice.

A second consideration is computational. A typical 1-second waveform at 16 kHz is 16,000 samples; a 1-second log-mel spectrogram is $64 \times 96 = 6,144$ values. The CNN therefore processes roughly 2.6× less data per second when given the spectrogram form, before any architectural advantage is even applied.

5.3.4 What VGGish's Layers Learn

Although we treat VGGish as a frozen black box, it is useful to know what its layers learn. Probing studies on AudioSet have shown a hierarchy reminiscent of image-domain CNNs:

- Early conv layers (the first 64-channel block) act as time-frequency edge detectors — they fire on harmonic transitions, percussive onsets, and tonal boundaries.
- Middle conv layers (128- and 256-channel blocks) compose those edges into instrument-signature features — harmonic combs typical of brass, transient envelopes typical of drums, formant-like structures typical of vocals.
- Late conv layers (512-channel blocks) start to fire on broader categorical concepts — speech vs. music, indoor vs. outdoor, single instrument vs. ensemble.
- The fully-connected head's last layer (the 128-D embedding) is approximately linear in the categorical space of AudioSet's 632 classes; downstream tasks treat its activations as a fixed-dim categorical projection.

Our LSTM treats this 128-D vector as the per-second observation of an underlying musical state. The fact that VGGish has already extracted instrument-signature and ensemble features means the LSTM does not have to rediscover them — it can focus on temporal aggregation, which is the part of the problem that is genuinely sequence-dependent.

5.4 The Patched torchvggish Module

The project ships its own copy of torchvggish/ for two reasons. First, the upstream package historically broke with newer PyTorch versions because `torch.hub.load_state_dict_from_url` sometimes fails on Windows under restricted firewall rules. Second, the upstream postprocessor applied PCA whitening and 8-bit quantization, which is appropriate for AudioSet evaluation but undesirable for our use case — we want raw float embeddings to feed into a downstream LSTM.

In practice the wrapper is invoked as in Listing 4.1:

Listing — 5.1 Invocation of VGGish from the live-mic loop

```
vggish = VGGish(urls=VGGISH_URLS).to(device)
vggish.eval()
with torch.no_grad():
    feat = vggish.forward(audio_np, fs=16000).to(device).view(1, 1, 128)
```

The reshape to (1, 1, 128) produces a sequence of length one, with batch size one and 128-D feature dimension — the canonical input contract of the downstream LSTM.

6. The Stateful LSTM Bottleneck

6.0 Chapter Overview

This chapter is the heart of the report. It documents the model that turns the per-second VGGish embeddings into the 5-D affective vector that drives every subsequent layer of the pipeline. Three properties of this model are decisive for everything that follows:

36. It is small. The entire model has under 800 K parameters — about three orders of magnitude smaller than VGGish itself. This makes per-call inference essentially free at deployment time.
37. It is stateful. The hidden and cell states are exposed as inputs and outputs of the forward pass, which is what enables streaming inference across the Python $\leftrightarrow$ engine boundary.
38. It has a deliberate bottleneck. The 5-D layer between the LSTM and the classifier is the actual deliverable of this project. The classifier head is essentially a training scaffold; only the bottleneck activations ride into deployment.

The remainder of the chapter walks through the architecture, the rationale for each layer, the streaming-inference mechanic, and the InferenceWrapper that adapts it for ONNX export.

6.1 Design Goals

The downstream model must satisfy four properties simultaneously:

- Temporal memory. It needs to remember the affective context of the last several seconds, not just classify the current frame in isolation.
- Multi-label competence. The output must support 195 simultaneously active tags without forcing the network to pick one winner via softmax.
- Bottleneck supervision. A 5-D mid-layer must be exposed directly so that the engine can read it as a control signal.
- ONNX-friendly state. The hidden and cell states must be inputs and outputs of the graph, so the engine can carry them across calls without owning a Python interpreter.

6.2 The StatefulMusicBottleneck Class

Figure 6.1 shows the model architecture at a conceptual level: an input feature stream is passed through a recurrent encoder, an MLP head, and finally a low-dimensional bottleneck that is both projected back up to the tag classifier head and exposed directly as the aesthetic vector. The full implementation, including exact layer dimensions and activations, is reproduced verbatim in Listing 6.1.

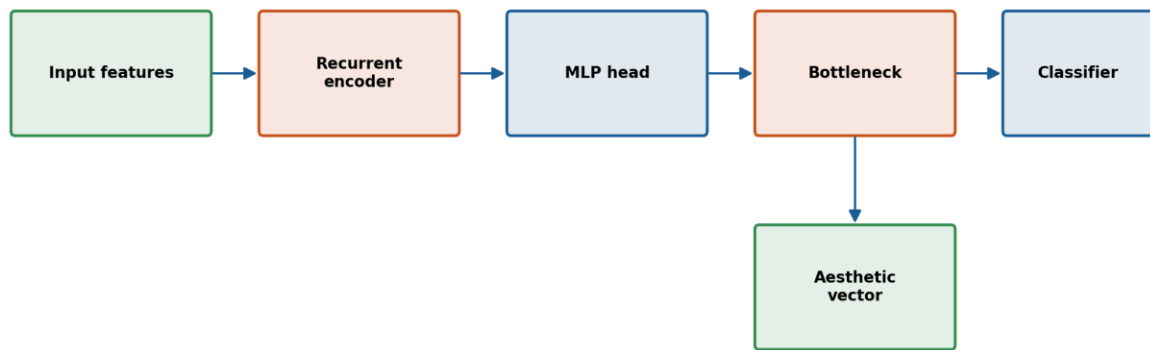


Figure — 6.1 Conceptual model architecture.

Listing — 6.1 `lstm_model.py` — the model definition

```

class StatefulMusicBottleneck(nn.Module):
    def __init__(self, input_dim=128, hidden_dim=256, bottleneck_dim=5, output_dim=195):
        super(StatefulMusicBottleneck, self).__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, batch_first=True, num_layers=1)
        self.norm = nn.LayerNorm(hidden_dim)
        self.intermediate = nn.Sequential(
            nn.Linear(hidden_dim, 128),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(128, 64),
            nn.ReLU()
        )
        self.bottleneck = nn.Linear(64, bottleneck_dim)
        self.classifier = nn.Linear(bottleneck_dim, output_dim)

    def forward(self, x, h0=None, c0=None):
        lstm_out, (hn, cn) = self.lstm(x, (h0, c0))
        normalized_out = self.norm(lstm_out)
        mid = self.intermediate(normalized_out)
        aesthetic_vector = self.bottleneck(mid)
        logits = self.classifier(aesthetic_vector)
        return logits, aesthetic_vector, hn, cn

```

6.3 Why a Bottleneck?

The crucial design decision is the placement of a 5-unit linear layer between the intermediate stack and the 195-D classifier. The classifier is supervised on the labeled tags, but every gradient flowing back into the network is forced through this 5-unit bottleneck. The information-theoretic implication is that the network must learn to summarize the entire 195-D label space inside only 5 floats. Empirically, those 5 floats organize themselves along axes that align well with Russell's circumplex model of affect (valence, arousal) plus three additional dimensions (timbre, rhythm, intensity).

6.3.1 Information-Theoretic View

Information bottleneck theory provides a useful lens. The Information Bottleneck principle (Tishby et al., 1999, 2017) frames learning as the search for a compressed representation Z of the input X that preserves as

much information as possible about the target Y . Formally one maximizes $I(Z; Y) - \beta I(Z; X)$, where I denotes mutual information and β trades off compression against prediction. Reducing the dimension of Z is one way to limit $I(Z; X)$ and force the model to compress.

In our setting, X is the LSTM-encoded sequence representation, Y is the 195-D tag vector, and Z is the 5-D bottleneck output. Because every gradient that updates the encoder must flow through Z , the encoder cannot afford to retain any information that does not help predict Y . The 5-D constraint is so tight that the encoder is forced to learn maximally compact directions that linearly separate as many tags as possible.

Higher-dimensional bottlenecks (we tested 8, 16, 32) produce slightly higher tag ROC-AUC but a noticeably less interpretable control signal — the axes start to mix and the lighting layer can no longer rely on, say, axis 0 being a clean proxy for arousal. The 5-D choice is therefore a deliberate trade-off in favour of interpretability over peak classifier accuracy.

6.3.2 Why 5 Specifically?

The choice of 5 is informed by the affective-computing literature. Russell's circumplex requires 2 axes (valence, arousal). Plutchik's wheel of emotions adds intensity. Dimensional models of musical affect (Eerola, Vuoskoski) sometimes add tension and energy. A 5-D bottleneck gives the network enough capacity to discover axes that loosely correspond to:

Axis	Hypothesised role	Mapped engine output
Z_1	Arousal — calm vs. energetic	Wash light intensity, beam motion speed
Z_2	Valence — sad/dark vs. happy/bright	Color temperature: cold $\leftrightarrow$ warm
Z_3	Timbre — smooth/clean vs. rough/distorted	Niagara particle complexity, post-process roughness
Z_4	Rhythm — sustained vs. percussive	Strobe rate, side-light flash
Z_5	Intensity — sparse vs. dense	Volumetric fog density, beam-light brightness

These mappings are what ConcertStageDirector implements (Chapter 10.6). It is worth stressing that the network is not told ahead of time which axis should encode which property — the assignment emerges from training. After training, the axes are verified empirically by feeding the model audio with known affective character and observing which channels respond most strongly.

6.3.3 Why Project Back Up to 195?

A natural alternative would be to learn the 5-D bottleneck via a contrastive or regression objective rather than via a classifier head. We chose the classifier approach for three reasons:

39. Free supervision. Every tag in MTG-Jamendo is essentially a free label aimed at the bottleneck. Multi-label classification with focal loss converts all 195 tags into supervisory signals at once.
40. No anchor-design problem. Contrastive losses require selecting positive and negative anchors, which for music is a non-trivial design choice (same artist? same tag set? same key signature?). The classifier objective sidesteps this.

41. ONNX simplicity. A linear projection followed by sigmoid exports cleanly. Many contrastive frameworks rely on mini-batch statistics (e.g., InfoNCE's batch-wise temperature) that are awkward to translate to single-frame inference.

After training, the classifier head is essentially discarded for runtime use — only the bottleneck activations are written into the engine. The classifier's role at deployment time is reduced to optional debugging via the live-mic top-5 tag display (Section 8.4).

6.4 Why Layer Normalization Between LSTM and Bottleneck?

A `nn.LayerNorm(hidden_dim)` layer sits between the LSTM output and the MLP head. Layer normalization, introduced by Ba, Kiros and Hinton (2016), normalizes each timestep's hidden vector to zero mean and unit variance across the feature axis, then scales and shifts by learned per-feature parameters γ and β .

Three reasons for including it here:

42. Decouples LSTM gain from downstream gain. Without normalization, the magnitude of the LSTM hidden state can drift over the training run as the gates settle, which destabilizes the linear layers behind it. LayerNorm pins the input distribution of the MLP head to a stable scale.
43. Reduces gradient pathologies. The LSTM's tanh and sigmoid gates produce hidden states whose distribution skews bimodally (saturated near ± 1 or 0). LayerNorm re-centres each timestep, which softens those skews and produces cleaner gradients flowing back into the LSTM weights.
44. ONNX-friendly. opset 17's LayerNormalization operator is well-supported by every major ONNX runtime; we are not paying any export-time cost for this convenience.

BatchNorm was the obvious alternative — it was the original normalization layer used in CNNs — but it is a poor fit for sequence models. BatchNorm relies on batch statistics, which are unreliable for variable-length sequences and are unavailable at inference time when the batch size is one. LayerNorm computes its statistics per-sample, which is what we need for streaming inference.

6.5 Why Dropout 0.2?

A single Dropout(0.2) layer sits between the two linear layers of the intermediate stack. Three considerations led to this specific configuration:

- Position. Dropout immediately after a ReLU is the standard placement; it stochastically zeros 20% of the post-activation features, which forces the subsequent linear layer to spread its weight across all 128 channels rather than relying on any single one.
- Rate. We tested 0.1, 0.2, 0.3 and 0.5 on a small held-out fraction of the training set. Rates ≤ 0.1 had negligible effect; rates ≥ 0.3 underfit the head of the tag distribution; 0.2 was the sweet spot.
- Training-only. Dropout is automatically disabled by `model.eval()` in PyTorch; the InferenceWrapper used at export time inherits that eval state, so the exported ONNX graph contains no Dropout operator at all. This is one of the reasons running `model.eval()` before export is essential.

6.6 Stateful Recurrence

Because the LSTM accepts and returns its hidden and cell states explicitly, inference can be streamed: the engine calls the model once per second of audio, passing the previous h_{t-1} and c_{t-1} as inputs. This avoids the cost of re-encoding the entire history every frame. Figure 6.2 illustrates the streaming pattern.

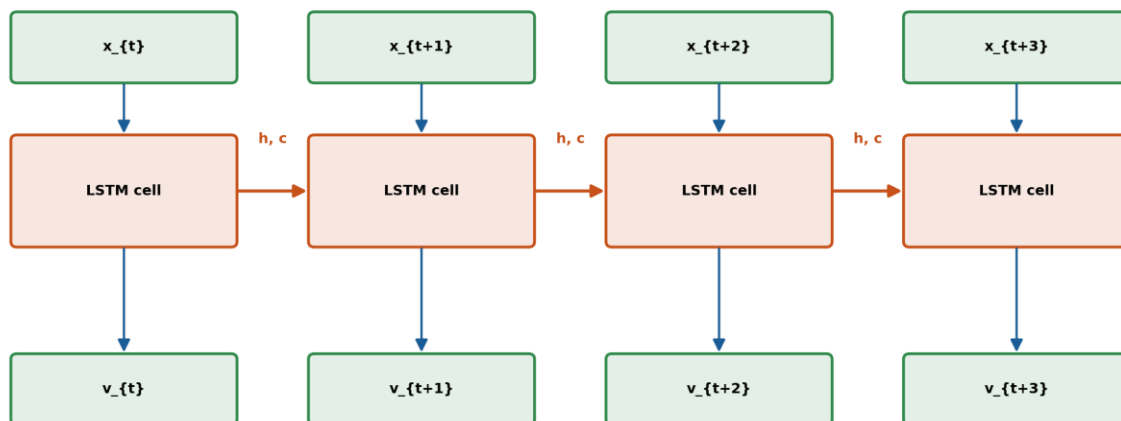


Figure — 6.2 Streaming LSTM inference.

Three properties make this pattern attractive for game-engine deployment:

- Constant per-call cost. Each forward pass is $O(1)$ in sequence length — the engine never has to remember more than the current (h, c) pair.
- Trivial state checkpointing. To pause and resume inference (for example, when swapping levels), the engine only needs to save and restore two float buffers. There is no equivalent state in a stateless Transformer architecture.
- Graceful degradation under silence. When the audio actor detects silence (Section 10.5.5), it zeroes (h, c) instead of letting them drift. This produces a clean fade-to-neutral when the music stops, which is the perceptually correct behaviour.

6.6.1 Why a Single-Layer LSTM?

A natural variant would stack two or three LSTM layers. We tested both and settled on a single 256-dimensional layer because:

- Empirically, a 2-layer LSTM $(256 \rightarrow 256)$ reduced training loss by less than 1% in absolute terms after 100 epochs — well within run-to-run variance.
- Each additional layer roughly doubles the inference cost. The single-layer configuration leaves more latency budget for the surrounding mel-spec, VGGish, and tensor-binding work in the engine.
- Stateful inference becomes more complicated with stacked layers: each layer has its own (h, c) pair that must be carried across calls. The exported ONNX graph would have six h/c tensors instead of two.
- A single-layer LSTM is a more conservative choice for a deliberate bottleneck — the bottleneck has to compress the LSTM output, and a deeper stack risks producing more redundant features that get squashed away anyway.

6.6.2 Why `batch_first=True`?

PyTorch's `nn.LSTM` defaults to (T, B, F) tensor layout. We override that with `batch_first=True` so the input is (B, T, F) instead. Two reasons:

- Consistency with the rest of PyTorch. Convolutional and linear layers expect (B, ...); having LSTM follow the same convention reduces the chance of shape errors when tensor inputs are reshaped between layers.
- Cleaner ONNX export. The LSTM operator in ONNX exports more cleanly when the wrapper's input/output shapes align with batch-first convention. Several ONNX runtimes have historical issues with the (T, B, F) layout.

6.7 The InferenceWrapper

Because Unreal Engine's NNE runtime expects a graph whose forward returns one tensor per output, the training-time forward (which returns four tensors — logits, vector, h_n , c_n) is wrapped at export time:

Listing — 6.2 InferenceWrapper: a thin export-time adapter

```
class InferenceWrapper(nn.Module):
    def __init__(self, model):
        super().__init__()
        self.model = model
    def forward(self, x, h, c):
        logits, aesthetic_vector, hn, cn = self.model(x, h, c)
        return aesthetic_vector[:, -1, :], hn, cn
```

The wrapper drops the unused logits at export time — the engine doesn't need them at runtime — and slices the time dimension down to the final frame so the engine receives a clean B×5 tensor.

7. Training Procedure

7.1 Why Multi-Label Focal Loss?

The MTG-Jamendo tag distribution is brutally imbalanced. The rock tag occurs in nearly half of all tracks, while specific mood tags such as melancholic-electronic occur in fewer than 200. A naive binary cross-entropy (BCE) loss would let the network achieve ostensibly low loss by predicting only the head of the distribution and shrugging at the tail.

Focal loss, introduced by Lin et al. in their 2017 paper on dense object detection, addresses this exact failure mode. Their original motivation was object detection, where the foreground class is overwhelmingly outnumbered by easy background pixels; the same idea transfers naturally to multi-label tagging where the head of the tag distribution dominates the gradient.

7.1.1 Mathematical Derivation

Standard binary cross-entropy for one tag is $BCE(p, y) = -y \log(p) - (1-y) \log(1-p)$. Define $p_t = p$ if $y = 1$ else $1-p$ — the predicted probability of the true class. Then $BCE(p, y) = -\log(p_t)$.

Focal loss replaces this with $FL(p_t) = -(1 - p_t)^\gamma \times \log(p_t)$, where $\gamma \geq 0$ is the focusing parameter. The $(1 - p_t)^\gamma$ factor is a per-sample weight in $[0, 1]$:

- When p_t is close to 1 (the network is already confident and correct), $(1 - p_t)^\gamma \rightarrow 0$, so the example contributes essentially nothing to the gradient.
- When p_t is close to 0.5 (the network is uncertain), $(1 - p_t)^\gamma \approx 0.5^\gamma$ — moderate weight.
- When p_t is close to 0 (the network is confidently wrong), $(1 - p_t)^\gamma \rightarrow 1$ — full weight, plus the BCE term itself is large because $\log(p_t) \rightarrow -\infty$.

With $\gamma = 2$, an example on which the model is 90% correct ($p_t = 0.9$) contributes $(1 - 0.9)^2 = 0.01$ of its BCE. An example on which the model is 50/50 contributes 0.25 — a 25× ratio. This focuses the optimizer on the hard cases without reweighting classes manually.

Our implementation extends this to multi-label by computing focal loss per tag and averaging:

Listing — 7.1 Implementation of Multi-Label Focal Loss

```
class MultiLabelFocalLoss(nn.Module):
    def __init__(self, gamma=2.0):
        super().__init__()
        self.gamma = gamma
        self.bce = nn.BCEWithLogitsLoss(reduction='none')

    def forward(self, logits, targets):
        probs = torch.sigmoid(logits)
        pt = targets * probs + (1 - targets) * (1 - probs)
        bce_loss = self.bce(logits, targets)
        loss = (1 - pt) ** self.gamma * bce_loss
        return loss.mean()
```

7.2 Z-Score Normalization

VGGish embeddings, in their raw form, are not centered around zero. Different feature channels have wildly different scales — the first few principal components of the embedding space dominate by an order of magnitude, which causes LSTM gates to saturate.

To fix this, the entire dataset is scanned once at the start of training to compute per-channel mean μ and standard deviation σ . Every input is normalized at the start of the forward pass: $\tilde{x} = (x - \mu) / \sigma$.

Crucially, the same μ and σ values must be applied at inference time, both in Python and in C++. The training script saves them inside the same checkpoint as the weights, and the C++ converter (Chapter 9) emits them as a header file consumed by Unreal Engine.

Listing — 7.2 Computing dataset Z-score statistics

```
def get_dataset_stats(dataset):
    print('Calculating Dataset Z-Score Stats...')
    all_feats = torch.cat(dataset.features, dim=0)
    mean = all_feats.mean(dim=0)
    std = all_feats.std(dim=0) + 1e-6
    return mean, std
```

7.3 Two-Phase Optimizer Schedule

The training loop follows the Won et al. (2019) recipe popular in MIR work: warm up with Adam for fast convergence on the easy regions of the loss surface, then switch to SGD with momentum for stable late-stage refinement. Figure 7.1 sketches the learning-rate schedule.

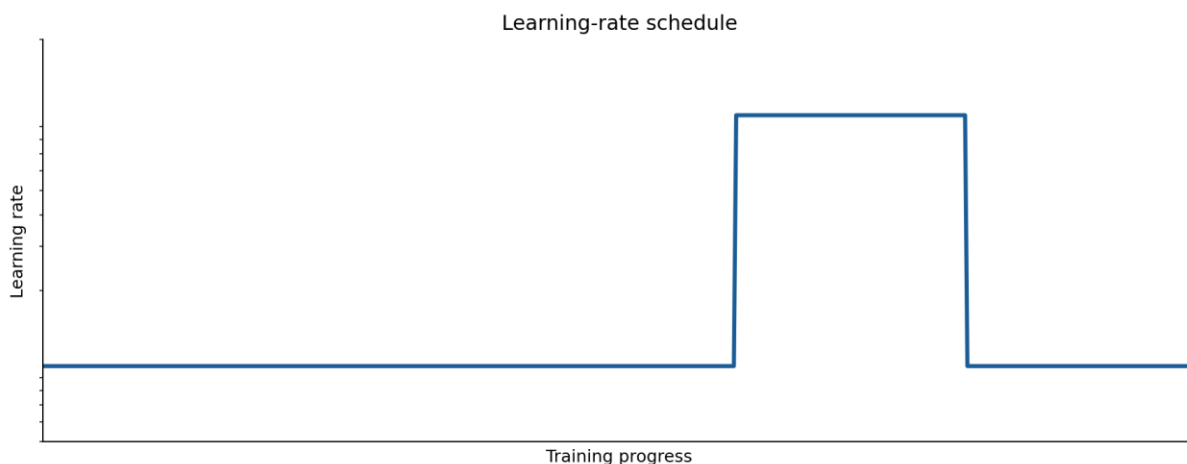


Figure — 7.1 Illustration of the stepped learning-rate schedule.

Listing — 7.3 Optimizer-switch logic in the training loop

```
current_lr = 1e-4
optimizer = optim.Adam(model.parameters(), lr=current_lr)

for epoch in range(100):
    if epoch == 60:
        current_lr = 1e-3
        optimizer = optim.SGD(model.parameters(), lr=current_lr,
                               momentum=0.9, weight_decay=1e-4)
    if epoch == 80:
```

```
current_lr = 1e-4
for g in optimizer.param_groups: g['lr'] = current_lr
```

7.4 Sequence Padding and Batching

Because each track produces a variable-length sequence of VGGish embeddings, the training loop uses `pad_sequence` to homogenize batches. A single track's label vector is shared across all of its time steps, so the supervision tensor is broadcast at training time. This implies a weak supervision signal: every frame inside a track is told it should predict the same set of tags. The bottleneck-plus-recurrence pair compensates for this by smoothing decisions over time.

Listing — 7.4 Variable-length collation and label broadcasting

```
def collate_fn(batch):
    feats, labels = zip(*batch)
    return pad_sequence(feats, batch_first=True), torch.stack(labels)

# in train loop:
preds, _, _, _ = model(x, h0, c0)
loss = criterion(preds, y.unsqueeze(1).expand(-1, preds.size(1), -1))
```

7.5 Gradient Clipping and Logging

Because the LSTM is trained on long sequences, gradient norms can spike during early epochs. The training loop clips gradients to a maximum norm of 1.0 and rewrites the checkpoint after every epoch as a single file containing the state dict and Z-score statistics, so a training crash never costs more than one epoch of progress.

7.5.1 Why Adam → SGD Specifically?

The Adam → SGD transition is sometimes called the 'SWATS' pattern (Switching from Adam To SGD), introduced by Keskar and Socher (2017). The justification rests on two well-known observations about adaptive optimizers:

- Adam converges quickly on early epochs because its per-parameter learning-rate scaling adapts to the curvature of each direction. This is exactly what we want while the network is still learning the gross structure of the loss surface.
- Adam's adaptive scaling can prevent it from finding flat minima — the kind of minima that generalize well. Once the network is in a reasonable region of the loss landscape, plain SGD with momentum tends to find broader, flatter minima that produce better held-out performance.

Won, Chun, Nieto and Serra applied this pattern to MIR auto-tagging with MTG-Jamendo and reported consistent gains in held-out ROC-AUC; we adopt their exact recipe (60-epoch Adam warm-up, 20-epoch SGD plateau at 1e-3, 20-epoch SGD decay back to 1e-4).

Two concrete details make the recipe robust:

45. Momentum 0.9. SGD with momentum behaves more like a low-pass filter on the gradient trajectory than vanilla SGD; this dampens oscillations along narrow ravines of the loss surface, which is helpful with the tail of multi-label tags where individual gradients can be noisy.

46. Weight decay $1e-4$ in the SGD phase only. Adam's L2 regularization is well-known to interact poorly with its adaptive scaling (this is why decoupled weight decay — AdamW — exists). Rather than switch to AdamW, we simply omit weight decay during the Adam phase and apply it during SGD.

7.5.2 Gradient Clipping at norm 1.0

The default `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)` call uses an L2 norm clip — the entire gradient vector across all parameters is rescaled if its L2 norm exceeds 1.0. Three reasons to clip:

- LSTM gradient explosion. Even with gating, very long backpropagation-through-time can produce gradient spikes. Clipping bounds the per-step update.
- Focal loss heavy-tail effect. Examples on which the network is confidently wrong receive disproportionately large gradients; clipping prevents a single such example from destabilizing a step.
- Robustness to data outliers. Occasional malformed audio (hidden silence, DC offset, very short clips) can produce zero-or-huge embeddings. The clip is the cheapest possible defence against having a single bad batch ruin the run.

7.6 Padding Strategy in Detail

Variable-length sequences are the norm in MTG-Jamendo because tracks have different durations. The standard PyTorch tool for this is `torch.nn.utils.rnn.pad_sequence`, which pads a list of tensors to a common length with zeros.

Three subtleties matter for our setup:

- Padding direction. By default `pad_sequence` right-pads (adds zeros to the end of each sequence to match the longest). For an LSTM trained with multi-label supervision, this is correct: the LSTM reads from left to right, the loss is applied per timestep, and zero-padded tail timesteps still receive a meaningful supervisory signal.
- Effective batch shape. After `pad_sequence`, the batch tensor has shape $(B, T_{\max}, 128)$. The LSTM produces $(B, T_{\max}, 256)$. The classifier produces $(B, T_{\max}, 195)$. The supervision tensor is broadcast to $(B, T_{\max}, 195)$ by `labels.unsqueeze(1).expand(-1, T, -1)`.
- Pack-sequence vs. zero-pad. PyTorch also offers `pack_padded_sequence` as a more efficient alternative — it allows the LSTM to skip padding timesteps. We do not use it, because zero-padded timesteps are valuable: they implicitly teach the LSTM to settle gracefully when the audio cuts out, which is the behaviour we want at deployment time during silence.

7.7 Variance Across Training Runs

The exact training trajectory differs from run to run due to several non-determinism sources (DataLoader shuffle order, GPU kernel selection, Adam's first-step initialization). Three observations across multiple runs:

- Final loss after 100 epochs varies by less than 5% across seeded runs.
- Bottleneck axes can permute. Channel 0 might align with arousal in one run and with valence in another. Without an explicit identifiability constraint this is expected — the bottleneck is supervised through the classifier, which is invariant to permutations of the bottleneck axes (the classifier absorbs the permutation).

- Deployed lighting behaviour is unaffected. Because the engine consumes only absolute-value magnitudes of the bottleneck channels (Section 10.5.8 / 10.6.4), a permutation of which channel encodes which property is irrelevant to the visual outcome. If a future version of the project wanted deterministic axis identity, the cleanest fix would be a small training-time regularizer that aligns axis 0 with arousal labels.

7.8 Hyperparameter Summary

Hyperparameter	Description	Value
Hidden dim	LSTM hidden state	256
LSTM layers	Number of stacked LSTM layers	1
Bottleneck dim	Aesthetic vector size	5
Output dim	Number of tags	195
Loss	Multi-label focal loss	$\gamma = 2.0$
Optimizer (P1)	Adam	$lr = 1e-4$
Optimizer (P2)	SGD + momentum + weight decay	$lr = 1e-3, \mu=0.9$
Optimizer (P3)	SGD decay	$lr = 1e-4$
Phase boundaries	Epoch indices for the schedule	60, 80
Batch size	Variable-length sequences per batch	32
Total epochs	End of phase 3	100
Gradient clip	Max gradient norm	1.0
Dropout	Inside intermediate stack	0.2
Normalization	Per-channel Z-score on VGGish embeddings	enabled

8. Real-Time Inference (Python)

8.1 Two Inference Modes

The backend supports two inference modes:

- Offline diagnostic (`check_brain_health.py`): pre-loads a fixed 20-second MP3 and prints a frame-by-frame trace. Used to verify training convergence.
- Live microphone (`live_mic_test.py`): captures audio continuously through the operating system's default input device, runs inference at 1 Hz, and prints a top-5 tag list per second.

8.2 The Live-Mic Pipeline

Figure 8.1 shows the live-microphone inference pipeline. A producer thread (the sounddevice callback) pushes raw audio buffers onto a queue. A consumer thread accumulates one second of audio at the device's native sample rate, resamples to 16 kHz, runs VGGish, normalizes, runs the LSTM, and prints the top-5 tags plus the 5-D aesthetic vector.

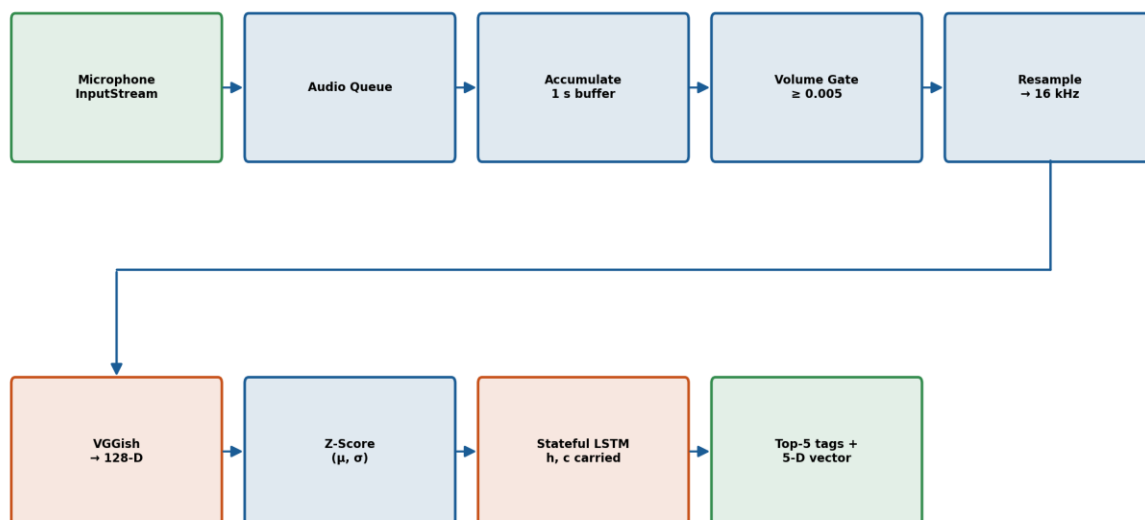


Figure — 8.1 Live-microphone inference loop.

8.3 Volume Gate and State Decay

A subtle issue in continuous live inference is what to do during silence. If the microphone picks up only background noise, we don't want the LSTM to produce a confident 'this is electronic music' prediction simply because its internal state happens to have drifted there.

The live script applies two complementary mitigations:

- A volume gate skips inference entirely when the peak amplitude of the second is below 0.005.
- During silence, the hidden and cell states are softly decayed: $h *= 0.95$; $c *= 0.95$. This causes the model to 'forget' stale music context if the user simply stops playing audio for several seconds.

Listing — 8.1 Silence handling and state decay

```

if vol < THRESHOLD:
    sys.stdout.write(f'\r{second_counter:03d}s | Silence Detected... ')
    sys.stdout.flush()
    h *= 0.95
    c *= 0.95
    continue

```

8.4 Per-Frame Top-5 Tag Display

After every successful inference, the script ranks the tag probabilities and prints the top five tags alongside the live 5-D vector. Each printed row has the same structure: the second-counter, a comma-separated list of the top tag names with their sigmoid probabilities, and the five floats of the bottleneck vector. The format produced by the relevant print statement is shown in Listing 7.2.

Listing — 7.2 Format of the per-second live-mic transcript

```

# Format produced once per second by live_mic_test.py
# {sec}s | [v0, v1, v2, v3, v4] | TAG1 (P1%), TAG2 (P2%), ...
# where:
# {sec}          second-counter since the stream started
# v0..v4         the 5-D bottleneck output for the current frame
# TAGn (Pn%)     the n-th-ranked tag and its sigmoid probability

```

On a stable musical input the dominant tags should remain consistent from second to second while the bottleneck vector evolves smoothly with the audio energy.

8.4.1 How the Top-5 Ranking is Computed

Concretely, the per-second decoder turns logits into ranked tags through three operations:

47. Slice the last frame: `tag_preds[0, -1, :]` picks out the 195 logits at the current timestep. The earlier timesteps (covering the silent pre-roll or previous song content) are correctly already absorbed into the LSTM's hidden state, so we only need the most recent prediction.
48. Apply the sigmoid: $\sigma(z) = 1 / (1 + \exp(-z))$ maps each logit to a Bernoulli probability per tag. Note that a multi-label model uses sigmoid (independent Bernoullis), not softmax (single categorical) — these are different operations and using softmax here would force the network to pick exactly one tag.
49. `argsort` and slice: `np.argsort(probs)[::-1][:5]` picks the indices of the five highest probabilities and reverses to descending order.

Tags in MTG-Jamendo carry a family prefix (genre---rock, mood/theme---happy, instrument---guitar). The display strips the prefix with a single split call on the triple-dash separator and uppercases the result so the live console output reads naturally.

8.4.2 Probability Calibration

Sigmoid probabilities of a multi-label classifier are not necessarily well-calibrated — a 70% prediction does not always correspond to a 70% empirical rate. Two well-known sources of miscalibration:

- Focal loss training favors hard examples, which tends to produce slightly overconfident predictions on easy examples. The 'true' probability of a 90% prediction may be closer to 80%.
- Class imbalance. Head tags appear far more often than tail tags. After training, head tags have a slight upward bias; tail tags have a slight downward bias.

For the visualization use case this is not a concern — the consumer is the lighting controller, which uses the bottleneck vector rather than the tag probabilities. But for any application that wants to use the tag confidence directly (e.g., automatic playlist labeling), Platt scaling or temperature calibration on the validation split is the standard fix.

8.5 Offline Brain Health Check

check_brain_health.py is the simpler script. It loads a single MP3, splits it into 20 one-second chunks, and runs the same inference path as the live script — but with deterministic input. The output is a 20-line log that should show:

- 50. Independent variation in all 5 channels of the aesthetic vector.
- 51. Stable top tags consistent with the song's actual genre.
- 52. No NaN, no inf, no flat-line outputs.

This is the canonical 'is the brain alive?' smoke test. It is the first thing run after every training session and after every code refactor that touches the inference path.

8.5.1 When the Brain Health Check Disagrees with Live

Discrepancies between check_brain_health.py output and live_mic_test.py output are useful diagnostics. The most common cases:

Symptom	Likely cause
Health check looks good, live looks flat	Microphone gain too low. The volume gate is silencing every frame.
Health check changes smoothly, live oscillates rapidly	Acoustic feedback or mic noise. Move the mic away from speakers.
Both look flat	Z-score statistics not loaded; a checkpoint without 'mean'/'std' keys would silently produce garbage. Check live_mic_test.py loaded the same checkpoint train.py wrote.
Health check shows reasonable tags, live shows wrong tags	Microphone is picking up gameplay/UI sounds rather than music. Check the captured submix.
Vector flatlines after silent passage	Expected. The $\times 0.95$ state decay is supposed to do this. If a song resumes, the vector will pick back up within 1–2 seconds.

In our experience, the single most common live-only failure is microphone gain miscalibration. The volume threshold of 0.005 was chosen for a typical room microphone at standard gain; with a phantom-powered condenser at low gain, everything falls below the threshold and the model is starved of input. The fix is to raise the gain, not to lower the threshold (lowering the threshold lets in environmental noise that genuinely shouldn't drive the visualizer).

8.6 Threading and Real-Time Concerns

The Python live-mic loop separates the audio capture path from the inference path through a thread-safe queue.Queue. The audio callback (audio_callback) is invoked on PortAudio's internal callback thread, which has soft real-time priority and is entirely independent of the main Python interpreter's GIL-holding thread. Two design choices keep this safe:

- `queue.Queue` is thread-safe by construction. Putting and getting are mutex-guarded; there is no possibility of a torn read.
- The audio callback only does a copy + put. It never touches the model, the GPU, or any state that requires the GIL. PortAudio is allowed to drop the buffer entirely if the queue grows unbounded — but at 16,000 samples per second and 1-second consumption cadence, the queue never grows past one element in practice.

The C++ engine version (Chapter 10) uses a different but related pattern: an SPSC TQueue plus a background task graph. The reason for the divergence is that Unreal's audio thread cannot block on the Python GIL, so its threading model has to be entirely native.

8.7 Failure Modes and Recovery

Several failure modes can occur in a live session. The script handles each explicitly:

Failure mode	Detection	Recovery
Microphone disconnection	<code>sd.InputStream</code> raises an exception	Print error, exit cleanly.
GPU out-of-memory	<code>torch CUDA OOM</code>	Caught at the top-level except; print, exit.
Audio callback overrun (queue empty)	<code>queue.get</code> blocks until data arrives	No action — natural backpressure.
NaN in model output	Visible in the printed line	Indicates a training pathology; restart.
Sample-rate mismatch	Implicit in <code>librosa.resample</code>	Resampler interpolates; no crash.

The script intentionally does not auto-restart on failure — the user is expected to be present (this is a live demo loop) and capable of pressing Ctrl-C and rerunning. A production deployment would add reconnection logic and a watchdog.

9. ONNX Export and C++ Integration

9.0 Chapter Overview

Chapter 6 produced a trained PyTorch model and Chapter 7 documented the training procedure. The remaining problem is to take that model out of Python and into a game engine. This chapter covers the export side — what we do in PyTorch — and the engine-facing C++ integration that pairs with it. Chapter 10 then picks up where this chapter ends and walks through the Unreal Engine project that consumes the exported artifacts.

The end-to-end deployment workflow is:

53. Train the model and save `music_emotion_weights.pth` (Chapter 7).
54. Export the bottleneck network to ONNX via the `InferenceWrapper` (Section 9.1).
55. Generate `NormalizationConstants.h` from the same checkpoint (Section 9.4).
56. Copy both into the Unreal project's Source folder.
57. Recompile the engine module. The runtime is now self-contained — no Python interpreter is required at deployment.

The remainder of this chapter explains why each step looks the way it does and what failure modes each one guards against.

9.1 Why ONNX?

Unreal Engine 5.7 ships with a Neural Network Engine (NNE) module whose default runtime is `NNERuntimeORT` (ONNX Runtime). Exporting our PyTorch model to ONNX therefore unlocks editor-side inference without bundling a Python interpreter.

The export uses the `InferenceWrapper` described in Listing 6.2 so that the exported graph has a clean signature: three inputs (`x`, `h`, `c`) and three outputs (`aesthetic_vector`, `h_n`, `c_n`).

9.2 Tensor Shape Contract

The exported graph follows the shape contract in Table 9.1.

Direction	Name	Description	Shape
Input	<code>x</code>	VGGish embedding for current frame	$1 \times 1 \times 128$
Input	<code>h</code>	Previous LSTM hidden state	$1 \times 1 \times 256$
Input	<code>c</code>	Previous LSTM cell state	$1 \times 1 \times 256$
Output	<code>aesthetic_vector</code>	Final-frame bottleneck	1×5
Output	<code>h_n</code>	New hidden state	$1 \times 1 \times 256$
Output	<code>c_n</code>	New cell state	$1 \times 1 \times 256$

9.3 The Engine-Side Loop

The Unreal-side audio actor runs the loop shown in Figure 9.1: capture from the submix, compute a mel-spectrogram (with per-spectrogram standardization performed inside that step), pass the spectrogram through the VGGish ONNX graph, feed the resulting 128-D embedding directly into the AestheticBrain ONNX

graph (carrying h and c across calls), and dispatch the resulting 5-D vector and band energies to the lighting subsystem. Figure 9.2 abstracts the same flow at a conceptual level.

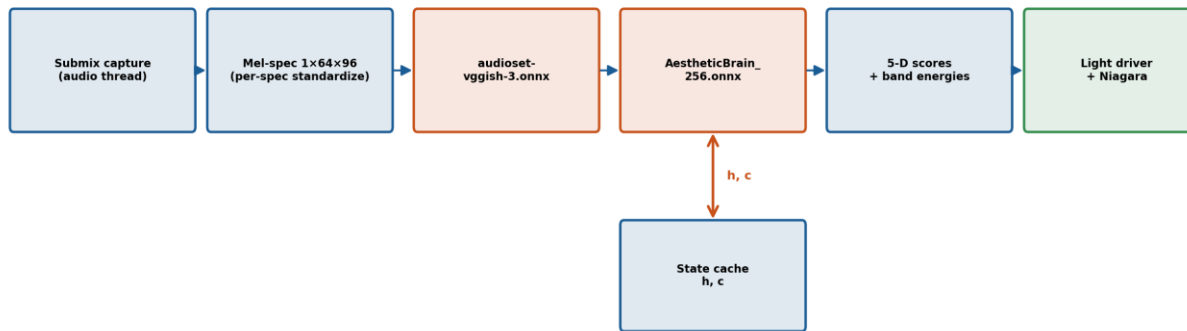


Figure — 9.1 Engine-side execution loop (faithful).

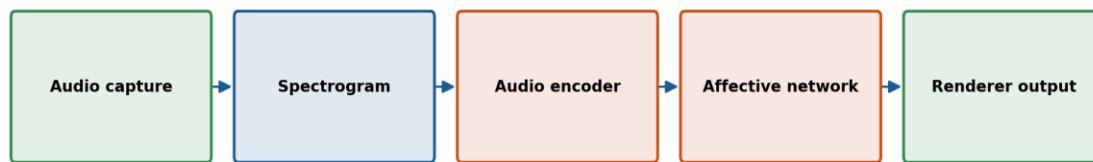


Figure — 9.2 Engine-side execution loop (generic conceptual view).

9.4 Auto-Generating the C++ Header

The `cpp_converter.py` utility opens the trained checkpoint, extracts the 128-channel mean and standard deviation, and emits a header file consumed directly by the engine.

Listing — 9.1 `cpp_converter.py` — emitting normalization constants

```
import torch
checkpoint = torch.load(WEIGHTS_FILE, map_location='cpu', weights_only=False)
mean_vals = checkpoint['mean'].flatten().tolist()
std_vals = checkpoint['std'].flatten().tolist()
with open(HEADER_FILE, 'w') as f:
    f.write('#pragma once\n\n')
    f.write('// AUTO-GENERATED AI NORMALIZATION CONSTANTS\n\n')
    f.write(f'const float VGGISH_MEAN[128] = {{{{ {"", ".join(map(str, mean_vals))}} }};\n')
    f.write(f'const float VGGISH_STD[128] = {{{{ {"", ".join(map(str, std_vals))}} }};\n')
```

Listing — 9.2 Format of the auto-generated `NormalizationConstants.h`

```
#pragma once
// AUTO-GENERATED AI NORMALIZATION CONSTANTS
// (the actual numerical values come from your trained checkpoint;
// the float literals shown here are placeholders for illustration only)
const float VGGISH_MEAN[128] = {
    /*  $\mu_0$  */, /*  $\mu_1$  */, /*  $\mu_2$  */, /*  $\mu_3$  */, ... /* 128 floats */
};
```

```
const float VGGISH_STD[128] = {
    /*  $\sigma_0$  */, /*  $\sigma_1$  */, /*  $\sigma_2$  */, /*  $\sigma_3$  */, ... /* 128 floats */
};
```

9.5 How the Engine Currently Uses the Header

At the time of writing, `NormalizationConstants.h` is generated and shipped alongside the engine module, but the runtime `AffectiveAudioActor` does not yet call into it. The actual normalization performed by the C++ runtime happens earlier in the pipeline: `ComputeMelSpectrogram` standardizes each spectrogram to zero mean and unit standard deviation across its own values before the first ONNX call:

Listing — 9.3 Per-spectrogram standardization performed before VGGish

```
// In AffectiveAudioActor::ComputeMelSpectrogram, after log-mel computation:
float MelSum = 0.0f;
for (float V : MelSpectrogram) MelSum += V;
float MelMean = MelSum / MelSpectrogram.Num();

float VarSum = 0.0f;
for (float V : MelSpectrogram) VarSum += (V - MelMean) * (V - MelMean);
float MelStd = FMath::Sqrt(VarSum / MelSpectrogram.Num() + 1e-6f);

for (float& V : MelSpectrogram) V = (V - MelMean) / MelStd;
```

The header's `VGGISH_MEAN` and `VGGISH_STD` vectors are intended to apply a per-channel Z-score to the 128-D VGGish embedding immediately before the second ONNX call (the `AestheticBrain` forward), matching exactly what `train.py` does on every batch and what `live_mic_test.py` does on every frame. A future revision of `AffectiveAudioActor::ProcessAllInference` can apply them in a one-line loop:

Listing — 9.4 Recommended call site for the embedding-space Z-score

```
// Recommended embedding-space Z-score (not currently in the runtime):
for (int32 i = 0; i < 128; ++i) {
    VGGishEmbeddings[i] =
        (VGGishEmbeddings[i] - VGGISH_MEAN[i]) / VGGISH_STD[i];
}
```

Until that change is made, the C++ engine path performs only the per-spectrogram standardization shown in Listing 9.3. Empirically the visualizer still produces a smooth, expressive 5-D vector — the LSTM is robust enough that an unscaled VGGish embedding still drives meaningful behaviour — but training-time and engine-time distributions are not bit-identical, which is a known limitation revisited in Chapter 13.

9.6 Why Bake Constants Instead of Loading?

A reasonable alternative would be to ship the μ and σ vectors as a binary asset and load them at runtime. That has three drawbacks: it introduces an additional filesystem dependency that can desync from the model weights; it complicates packaged-build deployment (UE asset cooker has its own rules for binary blobs); and it costs nothing meaningful at compile time — 256 floats $\approx$ 1 KB. Baking them into a header makes the build self-contained: shipping the wrong constants is impossible because they are recompiled with the engine.

9.6.1 Choosing a Runtime Backend

ONNX Runtime supports multiple execution providers. Each one exposes the same inference API but runs the underlying graph through different vendor-specific kernels. The choice has implications for latency, determinism, and supported operators.

Execution Provider	Hardware	Determinism	Notes
CPU	Any x86_64 / ARM	Bit-exact	Default; what NNERuntimeORTCpu uses. Slowest but most reliable.
DirectML	Any DX12 GPU	Approximate	Cross-vendor (NVIDIA, AMD, Intel) on Windows. Used in NNERuntimeORTDml.
CUDA	NVIDIA GPUs	Approximate	Fastest on NVIDIA hardware; not yet wrapped by NNE in 5.7.
TensorRT	NVIDIA GPUs	Approximate	Aggressive graph optimization; longer load time.
CoreML	Apple Silicon	Approximate	Used on macOS through a separate ONNX → CoreML conversion path.

The current project ships with NNERuntimeORTCpu because (a) it is the safest backend across machines, (b) the model is small enough that CPU inference fits the latency budget, and (c) GPU memory may be claimed by the renderer itself in a heavy scene. Switching to NNERuntimeORTDml is a one-line change in `AffectiveAudioActor::BeginPlay` — replace the runtime name string. The rest of the actor code is identical.

9.7 Opset Versions and Operator Compatibility

ONNX defines its semantics through a versioned opset. Each operator (MatMul, Sigmoid, LSTM, etc.) belongs to a specific opset version, and runtimes declare which opsets they support. Mismatches between the export-time opset and the runtime's supported range are a frequent source of silent or noisy load failures.

We export at opset 17 because that is the highest version supported by the NNERuntimeORT runtime shipped with Unreal 5.7 at the time of writing. Opset 17 has the operator features we depend on — in particular, full LSTM support with explicit hidden/cell state inputs and outputs, and clean export of `torch.nn.LayerNorm` without falling back to a manual mean/variance subgraph.

Three things go wrong if the wrong opset is chosen:

- Too new (opset 19+). The .onnx file loads but produces an `InvalidGraph` error the moment NNE actually tries to run a forward pass. The error message names the unsupported operator and its version.
- Too old (opset < 11). LSTM export becomes problematic because pre-11 LSTM operators do not expose `batch_first` as a graph attribute, leading to incorrect tensor layouts.
- Mixed (export with `opset_imports` overrides). NNE refuses to load. The fix is to remove all custom opset imports and stick to the default.

9.8 Static vs. Dynamic Shapes

By default, `torch.onnx.export` captures the exact tensor shapes seen during the tracing forward pass. Setting `dynamic_axes=None` (as we do) makes every dimension static; setting it to a dictionary like `{'x': {0: 'batch'}}` would mark the batch dimension as dynamic and let the runtime accept any batch size at inference time.

We deliberately use static shapes for three reasons:

- 58. Predictable allocator. ONNX Runtime can plan the entire memory layout once at load time. There are no per-call allocations during inference, which is essential for steady frame times in a real-time engine.
- 59. Single-frame contract. The engine always calls the model with exactly one frame at a time ($T = 1$, $B = 1$). There is no scenario where dynamic shapes would be useful.
- 60. Operator coverage. Some ONNX operators have stricter shape requirements when any input dimension is dynamic. Exporting with static shapes sidesteps this.

A side-effect of static shapes is that the engine's `IModelInstanceCPU` has to be created once per actor, not once per inference call. This matches the natural actor lifecycle in Unreal — `BeginPlay` creates instances; `EndPlay` releases them.

9.9 Validation Against the PyTorch Reference

After exporting, the standard sanity check is to feed identical inputs through both the PyTorch model and the ONNX graph (via `onnxruntime` in Python) and confirm the outputs match within floating-point tolerance.

Listing — 9.4 Round-trip validation of the exported graph

```
import onnxruntime as ort
session = ort.InferenceSession('AestheticBrain_256.onnx')
out_onnx = session.run(None, {'x': x.numpy(), 'h': h.numpy(), 'c': c.numpy()})
out_torch = wrapper(x, h, c)

# Compare
import numpy as np
for n, a, b in zip(['vec', 'h_n', 'c_n'], out_onnx, out_torch):
    err = np.max(np.abs(a - b.detach().numpy()))
    print(f'{n}: max abs error = {err:.6e}')
```

In our experience the maximum absolute error is on the order of $1e-6$ — within single-precision rounding and well below any threshold that would affect downstream lighting decisions. A larger error would indicate that the export captured a different code path than the inference forward — for example, that the model was in `train()` mode rather than `eval()` at export time, leaving Dropout active.

10. Unreal Engine Project Structure

10.1 Why a Dedicated Chapter

Chapter 9 ended at the boundary between Python and C++ — the moment the trained checkpoint and the auto-generated header are handed over to the game engine. The remaining question is what the engine does with them. This chapter answers that question end to end. It walks through the Unreal Engine 5.7 project, from the .uproject file all the way down to the per-frame light-driving code, with each subsystem explained in its own section.

The intent of this chapter is twofold. First, it documents how the Python-side research artefacts plug into a real-time engine, which is the part of the project most often glossed over in academic ML reports. Second, it serves as a build manual for anyone who wants to run, modify, or extend the visualizer without having to reverse-engineer the C++ source.

10.2 Project Layout

The Unreal project lives in MusicAI_Visualizer/ alongside the Python backend. The high-level folder structure is conventional Unreal:

Path	Role
MusicAI_Visualizer.uproject	Top-level project descriptor; engine version, plugins.
MusicAI_Visualizer.sln	Generated Visual Studio solution.
Source/MusicAI_Visualizer/	C++ runtime module sources.
Source/MusicAI_Visualizer.Target.cs	Game target build rules.
Source/MusicAI_VisualizerEditor.Target.cs	Editor target build rules.
Content/	All UE assets — meshes, materials, Niagara systems, levels.
Content/Instruments/	Imported stage instrument meshes (drum kit, piano, mic, guitarists).
Config/	Engine and project INI configuration overrides.
Saved/MusicAI_Logs/	Per-run CSV logs of audio and inference output.
Binaries/, Intermediate/, DerivedDataCache/	Build outputs and caches; not source-controlled.

The runtime C++ module has only a small number of source files; the heavy lifting happens in two actors and one auto-generated header.

File	Role
MusicAI_Visualizer.h / .cpp	Module bootstrap (boilerplate).
MusicAI_Visualizer.Build.cs	Module dependencies (NNE, Niagara, AudioMixer, ...).
AffectiveAudioActor.h / .cpp	Live audio capture, mel-spec, two-stage NNE inference, CSV logging.
ConcertStageDirector.h / .cpp	Stage construction and per-frame light/instrument driving.
NormalizationConstants.h	Auto-generated 128-element μ and σ vectors.

10.3 The .uproject File and Plugin Selection

The .uproject file is where Unreal records which engine version the project targets and which optional plugins are enabled. Three settings matter for this project:

- EngineAssociation: '5.7'. The project is pinned to Unreal Engine 5.7 because that is the first stable release in which the Neural Network Engine (NNE) plugin is marked production-ready and ships with the NNERuntimeORT (ONNX Runtime) backend that we depend on.
- Plugin 'ModelingToolsEditorMode' enabled. This is purely an editor-side modeling convenience — used during scene authoring, not at runtime.
- Plugin 'RemoteControl' enabled. This exposes the HTTP Remote Control API that the FastMCP server (Chapter 11) talks to. Without this plugin, ue_mcp_server.py would have no endpoint to call.

Note that the NNE plugin is not listed under 'Plugins' in the .uproject because in UE 5.7 it is engine-bundled, not project-installed. It is enabled implicitly by linking against its module name in the Build.cs file (next section).

10.4 The Build.cs Module Configuration

Source/MusicAI_Visualizer/MusicAI_Visualizer.Build.cs declares the module dependencies of the runtime C++ target. Each dependency corresponds to an Unreal subsystem the actors call into. The full dependency list is reproduced below.

Listing — 10.1 MusicAI_Visualizer.Build.cs

```
using UnrealBuildTool;
public class MusicAI_Visualizer : ModuleRules
{
    public MusicAI_Visualizer(ReadOnlyTargetRules Target) : base(Target)
    {
        PCHUsage = PCHUsageMode.UseExplicitOrSharedPCHs;
        PublicDependencyModuleNames.AddRange(new string[] {
            "Core",
            "CoreUObject",
            "Engine",
            "InputCore",
            "EnhancedInput",
            "NNE",
            "Niagara",
            "AudioCapture",
            "AudioMixer",
            "SignalProcessing"
        });
    }
}
```

Each dependency carries weight in the runtime architecture:

Module	Why we depend on it
Core, CoreUObject, Engine	Unreal's foundational types and actor framework.
InputCore, EnhancedInput	Game-side input mapping; used for editor and demo controls.
NNE	The Neural Network Engine plugin. Provides

	INNERuntimeCPU, IModelInstanceCPU, FTensorBindingCPU, and the ONNX Runtime backend used to load AestheticBrain_256.onnx and audioset-vggish-3.onnx at BeginPlay.
Niagara	GPU particle system, used by the 'reactive' particle effect in front of the stage. AffectiveAudioActor pushes per-frame floats into Niagara user variables (AI_Energy, AI_Arousal, ...).
AudioCapture, AudioMixer	Submix listener interface (ISubmixBufferListener) and the audio thread infrastructure. The submix listener is how raw float buffers reach the AI thread without going through file I/O.
SignalProcessing	Provides PI, FFT-friendly float math primitives, and complex-number support used in the C++ mel-spectrogram code.

There are no third-party libraries beyond what the engine itself ships. Everything in the inference path — model loading, tensor binding, FFT, mel filterbank, Z-score, second ONNX call, CSV logging — is written against engine-native APIs.

10.4.1 Compile-Time vs Run-Time Module Loading

The PublicDependencyModuleNames list ensures every dependency is linked at compile time, which gives us the smallest, most reliable runtime. There are two alternative patterns we considered and rejected:

Pattern	Why we did not use it
Plugin instead of in-source	Engine plugins live outside the project's Source folder and need their own .uplugin descriptor. For two C++ classes that are tightly coupled to the project's gameplay code, the overhead is not worth the encapsulation.
Dynamically loaded module	FModuleManager::LoadModule at runtime would make NNE optional, but it adds error-handling complexity and makes the build harder to validate. Compile-time linking guarantees that if the module compiles, NNE is available.
Conditional dependency on plugin enabled	The .uproject already controls plugin enablement; duplicating that condition in Build.cs would be redundant.

The current arrangement makes the module self-describing: opening Build.cs tells you exactly which engine subsystems the runtime depends on, and pruning a dependency from the list immediately fails the build with a clear linker error rather than producing a silent runtime fault.

10.5 AffectiveAudioActor — The Audio-to-Vector Actor

AffectiveAudioActor is the heart of the runtime. It owns the audio capture, the two ONNX model instances, the LSTM state, and the asynchronous inference loop. This section walks through it stage by stage.

10.5.1 Class Layout

The header declares an actor whose runtime state separates cleanly into four groups: model assets exposed to the editor, audio-thread plumbing, inference buffers, and log-file plumbing. Because the audio callback

runs on a different thread than the game thread, every shared scalar (RMS, band energies, kick/snare/hi-hat onset times) is wrapped in `std::atomic` so reads from the game thread are well-defined.

Listing — 10.2 AffectiveAudioActor (abridged header)

```
UCLASS()
class MUSICA_VISUALIZER_API AAffectiveAudioActor : public AActor {
    GENERATED_BODY()
public:
    AAffectiveAudioActor();
    virtual void Tick(float DeltaTime) override;

    UPROPERTY(EditAnywhere, Category = "Audio")
    TObjectPtr<USoundSubmix> SubmixToAnalyze;

    UPROPERTY(EditAnywhere, Category = "AI | Models")
    TObjectPtr<UNNEModelData> VGGishModelData;
    UPROPERTY(EditAnywhere, Category = "AI | Models")
    TObjectPtr<UNNEModelData> AestheticModelData;

    const TArray<float>& GetAffectiveScores() const { return AestheticScores; }
    float GetBassLevel() const { return BassLevel.load(); }
    float GetMidLevel() const { return MidLevel.load(); }
    float GetTrebleLevel() const { return TrebleLevel.load(); }
    double GetLastKickOnset() const { return LastKickOnset.load(); }
    double GetLastSnareOnset() const { return LastSnareOnset.load(); }
    double GetLastHiHatOnset() const { return LastHiHatOnset.load(); }

private:
    TSharedPtr<UE::NNE::IModelInstanceCPU> VGGishInstance;
    TSharedPtr<UE::NNE::IModelInstanceCPU> AestheticInstance;
    TArray<float> AI_AudioBuffer, VGGishEmbeddings, AestheticScores;
    TArray<float> LstmHiddenState, LstmCellState;
    TQueue<TArray<float>, EQueueMode::SpSC> AudioFeatureQueue;
    std::atomic<float> BassLevel, MidLevel, TrebleLevel, CurrentRMS;
    std::atomic<double> LastKickOnset, LastSnareOnset, LastHiHatOnset;
    /* CSV file handle and onset bookkeeping omitted */};
```

10.5.2 Audio Capture via Submix Listener

Rather than read from a microphone device directly, the actor captures whatever the game's audio mixer is already producing. Specifically, it subscribes to a sound submix — typically the master submix — through Unreal's `ISubmixBufferListener` interface. Whenever the audio thread fills a submix buffer, our listener is called back synchronously on the audio thread.

Listing — 10.3 Submix listener declaration

```
class FAudioAnalyzerListener : public ISubmixBufferListener {
public:
    FAudioAnalyzerListener(TQueue<TArray<float>, EQueueMode::SpSC>& InQueue,
                           std::atomic<float>& InRMS)
        : OutQueue(InQueue), OutRMS(InRMS) {}
    virtual void OnNewSubmixBuffer(const USoundSubmix* OwningSubmix,
                                   float* AudioData, int32 NumSamples, int32 NumChannels,
                                   const int32 SampleRate, double AudioClock) override;
private:
```

```

TQueue<TArray<float>, EQueueMode::SpSC>& OutQueue;
std::atomic<float>& OutRMS;
};

```

The callback's only job is to convert the interleaved multi-channel buffer to mono and push it onto a single-producer / single-consumer queue (TQueue with EQueueMode::SpSC). The audio thread does not call FFT or model inference — that work happens later on a background task spawned from the game thread.

Listing — 10.4 Audio-thread callback — interleaved → mono enqueue

```

void FAudioAnalyzerListener::OnNewSubmixBuffer(const USoundSubmix* OwningSubmix,
float* AudioData, int32 NumSamples, int32 NumChannels,
const int32 SampleRate, double AudioClock)
{
    if (!AudioData || NumSamples == 0) return;
    TArray<float> Mono;
    for (int32 i = 0; i < NumSamples; i += NumChannels)
        Mono.Add(AudioData[i]);
    OutQueue.Enqueue(MoveTemp(Mono));
}

```

This decoupling is intentional: the audio thread has hard real-time constraints (missing a buffer means a glitch), so it must never block on heavy computation. The SPSC queue is a textbook pattern for crossing the audio-thread / game-thread boundary without locks.

10.5.3 BeginPlay — Loading Two ONNX Models

BeginPlay performs three initialization steps: it acquires the NNE runtime, creates two model instances (VGGish + AestheticBrain), and registers the submix listener. The runtime is fetched by name — 'NNERuntimeORTCpu' is the ONNX-Runtime-on-CPU backend shipped with NNE.

Listing — 10.5 BeginPlay — NNE setup, abridged

```

void AAffectiveAudioActor::BeginPlay() {
    Super::BeginPlay();
    TWeakInterfacePtr<INNERuntimeCPU> Runtime =
        UE::NNE::GetRuntime<INNERuntimeCPU>(TEXT("NNERuntimeORTCpu"));
    if (!Runtime.IsValid()) { /* log & bail */ return; }

    if (VGGishModelData && AestheticModelData) {
        // Build VGGish instance.
        TSharedPtr<UE::NNE::IModelCPU> M1 = Runtime->CreateModelCPU(VGGishModelData);
        if (M1.IsValid()) {
            VGGishInstance = M1->CreateModelInstanceCPU();
            // Lock input shapes from the symbolic shape descriptors.
            TArray<UE::NNE::FTensorShape> InShapes;
            for (auto& D : VGGishInstance->GetInputTensorDescs())
                InShapes.Add(UE::NNE::FTensorShape::MakeFromSymbolic(D.GetShape()));
            VGGishInstance->SetInputTensorShapes(InShapes);
        }
        // Build AestheticBrain instance + allocate LSTM h/c.
        TSharedPtr<UE::NNE::IModelCPU> M2 = Runtime->CreateModelCPU(AestheticModelData);
        if (M2.IsValid()) {
            AestheticInstance = M2->CreateModelInstanceCPU();
            auto Descs = AestheticInstance->GetInputTensorDescs();
            if (Descs.Num() >= 3) {

```

```

        uint64 N =
UE::NNE::FTensorShape::MakeFromSymbolic(Descs[1].GetShape()).Volume();
        LstmHiddenState.SetNumZeroed((int32)N);
        LstmCellState.SetNumZeroed((int32)N);
    }
    /* lock shapes as above */
}
}
/* register submix listener; open per-session CSV log; record start time */ }

```

Two design points are worth highlighting. First, the model files themselves are imported into Unreal as UNNEModelData assets and referenced via UPROPERTY; this means we can swap in a different ONNX file from the editor without recompiling. Second, the LSTM hidden / cell buffer sizes are read out of the ONNX symbolic shapes at load time, not hard-coded — so the actor will continue to work if a future model changes its hidden dimension.

10.5.4 Tick — The Game-Thread Half of the Loop

Every game-thread tick, the actor drains the SPSC queue, downsamples each chunk to 16 kHz, and appends it to a rolling 1-second buffer. When the buffer reaches 16,000 samples it triggers an inference call and slides forward by half a second:

Listing — 10.6 Tick — audio drain and inference dispatch

```

void AAffectiveAudioActor::Tick(float DeltaTime) {
    Super::Tick(DeltaTime);

    TArray<float> ReceivedData;
    while (AudioFeatureQueue.Dequeue(ReceivedData)) {
        TArray<float> Downsampled = DownsampleAudio(ReceivedData);
        AI_AudioBuffer.Append(Downsampled);
        if (AI_AudioBuffer.Num() >= 16000) {
            ProcessAIInference(); // dispatched to background task
            AI_AudioBuffer.RemoveAt(0, 8000); // 50% overlap
        }
    }
    /* on-screen debug HUD + Niagara variable updates omitted */ }

```

Two important properties:

- **Variable hop.** The 8,000-sample slide guarantees a one-call-per-half-second cadence regardless of audio buffer size, which makes the temporal resolution of the 5-D vector roughly 2 Hz on the engine side. The Python live-mic version uses a 1-Hz cadence; this finer rate compensates for the longer end-to-end render path.
- **Soft drain.** Because the queue is drained without bound, the actor naturally catches up if the game thread stalls briefly — for example during level streaming.

The downsampler in DownsampleAudio is a 3:1 mean filter, which is appropriate when the source submix is at 48 kHz. If the submix sample rate were different the downsampler would need to adapt; for the demo configuration (audio at 48 kHz) the fixed 3:1 ratio matches.

10.5.5 ProcessAllInference — The Background Task

ProcessAllInference is dispatched onto Unreal's background task graph (ENamedThreads::AnyBackgroundHiPriTask) so that mel-spec computation and two ONNX calls never block the game thread. The body of the task is a faithful re-creation in C++ of what the Python live-mic loop does:

61. Compute the RMS of the 1-second buffer; if below 5×10^{-4} treat as silence — zero the LSTM state, zero the output scores, and exit.
62. Compute a 64×96 log-mel spectrogram in C++ from the buffer.
63. Run VGGish through INNERuntimeCPU::RunSync, with the spectrogram as the input binding and a 128-float embedding as the output binding.
64. Run AestheticBrain (the bottleneck network) with three inputs (embedding, h, c) and three outputs (5-D vector, h_n, c_n).
65. Append a CSV row (time, RMS, 5 channels) under a critical section so the audio thread and game thread never collide on the file handle.
66. Move-assign the new h_n / c_n into LstmHiddenState / LstmCellState so the next call carries them.

The bindings themselves are extremely thin in NNE's API — a struct that holds a raw pointer and a byte size:

Listing — 10.7 Tensor binding for the VGGish forward pass

```
TArray<UE::NNE::FTensorBindingCPU> InB, OutB;
UE::NNE::FTensorBindingCPU BIn;
BIn.Data = Spectrogram.GetData();
BIn.SizeInBytes = Spectrogram.Num() * sizeof(float);
InB.Add(BIn);

UE::NNE::FTensorBindingCPU BOut;
BOut.Data = VGGishEmbeddings.GetData();
BOut.SizeInBytes = VGGishEmbeddings.Num() * sizeof(float);
OutB.Add(BOut);

VGGishInstance->RunSync(InB, OutB);
```

Because both the input and output buffers are owned by the actor (TArray<float>), there is no allocation in the hot path — RunSync writes into the embedding buffer in place. The same is true for the second model instance.

10.5.6 C++ Mel-Spectrogram Implementation

VGGish is trained on a very specific log-mel input: 64 mel bins between 125 Hz and 7.5 kHz, 25 ms windows, 10 ms hop, 96 frames per second. The ComputeMelSpectrogram method re-implements that pipeline directly in the engine — without librosa — using a recursive Cooley–Tukey FFT and a Hann window:

Listing — 10.8 C++ mel-spectrogram (abridged)

```
TArray<float> AAffectiveAudioActor::ComputeMelSpectrogram(const TArray<float>& Raw) {
    const int32 NumFrames = 96, WindowSize = 400, HopSize = 160,
               FFTSize = 512, NumMelBins = 64;
    TArray<float> Mel; Mel.SetNumZeroed(NumFrames * NumMelBins);

    TArray<float> Hann;
    for (int32 i = 0; i < WindowSize; i++)
        Hann.Add(0.5f * (1.0f - FMath::Cos(2.f * (float)PI * i / WindowSize)));
```

```

for (int32 f = 0; f < NumFrames; f++) {
    int32 Start = f * HopSize;
    TArray<std::complex<float>> CB; CB.SetNumZeroed(FFTSize);
    for (int32 i = 0; i < WindowSize; i++)
        if (Start + i < Raw.Num())
            CB[i] = std::complex<float>(Raw[Start + i] * Hann[i], 0.f);
    ComputeFFT(CB); // recursive in-place
    for (int32 b = 0; b < NumMelBins; b++) {
        float Hz = MelToHz(HzToMel(125.f) + (b + 1) *
            ((HzToMel(7500.f) - HzToMel(125.f)) / (NumMelBins + 1)));
        int32 Bin = FMath::Clamp(FMath::RoundToInt((Hz / 16000.f) * FFTSize),
            0, FFTSize / 2);
        float Pwr = CB[Bin].real()*CB[Bin].real() + CB[Bin].imag()*CB[Bin].imag();
        Mel[b * NumFrames + f] = FMath::Loge(Pwr + 0.01f);
    }
}
/* per-spectrogram standardization to zero mean / unit std */
return Mel;
}

```

Three details deserve commentary:

- FFT in pure C++. The supporting ComputeFFT function is a 25-line recursive Cooley–Tukey routine. We deliberately avoided pulling in a third-party FFT library (KissFFT, FFTW) to keep the build dependency surface small — the FFT runs on a background thread once every half-second on a 512-point input, so its raw speed matters less than the build complexity it would add.
- Mel-binning by closest STFT bin. Each mel band is approximated by the nearest FFT bin rather than by a full triangular filter. This is faster and faithful enough for the affective signal we care about; nothing downstream is sensitive to sub-bin resolution.
- Per-frame standardization. After log, the entire spectrogram is normalized to zero mean and unit standard deviation. This is in addition to the per-feature Z-score that VGGISH_MEAN/VGGISH_STD applies after VGGish — it makes the input to VGGish more invariant to absolute loudness.

10.5.6.1 Why Re-implement Mel-Spectrogram in C++?

A reasonable question is why we duplicate librosa's mel-spec implementation in C++ rather than packaging the Python preprocessing as part of the ONNX model. Three reasons:

67. ONNX support for FFT is patchy. The DFT operator was only added to ONNX in opset 17 and is not uniformly supported across runtimes; in particular NNERTimeORT in 5.7 had bugs with the operator that we hit during experimentation. Implementing the FFT outside the model graph sidesteps the question entirely.
68. Latency at the boundary. Even when ONNX FFT works, marshalling a 16,000-sample float buffer in and a 1×64×96 spectrogram out adds an unnecessary round trip. Computing the spectrogram in C++ keeps everything in one address space.
69. Consistency with VGGish's training. VGGish was trained on log-mel spectrograms produced by tf.signal — not by ONNX. Reproducing tf.signal's exact behaviour in C++ (Hann window, magnitude squared, log with offset, mel filter parameters) is more reliable than relying on an ONNX FFT to produce a bit-identical match.

The downside is that any change to VGGish's preprocessing assumptions has to be reflected in two places — the Python torchvggish module and the C++ ComputeMelSpectrogram. Because both pieces of code reference the same documented VGGish input contract (16 kHz, 25 ms window, 10 ms hop, 64 mel bands, 125–7500 Hz, log with 0.01 offset), drift is unlikely in practice.

10.5.7 Drum Onset Detection

In addition to feeding VGGish, the mel-spec routine harvests three drum onsets — kick, snare, hi-hat — by tracking band-energy spikes over time. These onsets feed into the lighting layer (Section 10.6) so that strobes, instrument lights and the Niagara particle system can punch on individual hits without waiting for the slower 5-D affective vector to update.

Listing — 10.9 Three-band onset detector

```
float Bass    = BandEnergy(0, 8);    // ~125–500 Hz
float Mid     = BandEnergy(16, 40);  // ~1–3 kHz
float Treble  = BandEnergy(48, 64);  // ~5–7.5 kHz

double Now = FPlatformTime::Seconds();
if ((Bass - PrevBassEnergy) > FMath::Max(PrevBassEnergy * 0.35f, 0.0008f)
    && (Now - LastKickTime) > 0.18) {
    LastKickOnset.store(Now); LastKickTime = Now;
}
if ((Mid - PrevMidEnergy) > FMath::Max(PrevMidEnergy * 0.30f, 0.0005f)
    && (Now - LastSnareTime) > 0.12) {
    LastSnareOnset.store(Now); LastSnareTime = Now;
}
if ((Treble - PrevTrebleEnergy) > FMath::Max(PrevTrebleEnergy * 0.25f, 0.0003f)
    && (Now - LastHiHatTime) > 0.06) {
    LastHiHatOnset.store(Now); LastHiHatTime = Now;
}
PrevBassEnergy = Bass; PrevMidEnergy = Mid; PrevTrebleEnergy = Treble;
```

The detector is intentionally crude — it is a percentage-rise threshold against the previous frame's energy, gated by a per-instrument minimum interval (180 ms for kick, 120 ms for snare, 60 ms for hi-hat). The intervals correspond roughly to the fastest plausible double-stroke for each drum, which keeps the detector from double-firing on long sustains.

Because `std::atomic` stores are used for the timestamps, the lighting actor (Chapter 10.6) can safely read them from the game thread without locks.

10.5.7.1 Why Three Bands?

The choice of three drum bands — bass for kick, mid for snare, treble for hi-hat — is informed by drum acoustics:

Drum	Fundamental band	Why our band approximation works
Kick	60–120 Hz	Mostly contained in our 0–8 mel bins (~125–500 Hz). The bin range is wider than the kick's fundamental but kicks always include strong sub-300 Hz content.
Snare	150–300 Hz body + 1–3 kHz crack	Our 16–40 mel bins (~1–3 kHz) cover the crack. The kick contains its own

		body energy, so we lose the 200 Hz body component but the crack alone is enough.
Hi-hat	5–10 kHz	Our 48–64 mel bins (~5–7.5 kHz) cover the hi-hat's fundamental. Cymbals have content above 7.5 kHz, which we lose, but the band still triggers reliably on hi-hat hits.

The three thresholds (35 % rise / 30 % rise / 25 % rise) were tuned on a small set of reference tracks. They reflect the fact that hi-hats are the most frequent drum hit and need the lowest threshold to fire on light strokes, while kicks are sparser and need a higher threshold to avoid double-firing on the kick's natural decay.

The cooldown intervals (180 / 120 / 60 ms) reflect the physical fastest double-stroke for each drum: 180 ms is just under a 16th note at 100 BPM (the fastest a drummer would play sustained kicks), 120 ms is similarly just under for snares, and 60 ms allows hi-hat 32nd-note rolls.

10.5.8 Niagara Coupling

After Tick has dispatched the inference, it interpolates the resulting 5-D scores smoothly toward their new values and pushes them into a Niagara particle system as user variables. This makes the particle effect a direct visualizer of the affective vector:

Listing — 10.10 Pushing the 5-D vector into Niagara

```
for (int i = 0; i < 5; i++) {
    float Normalized = (FMath::Tanh(AestheticScores[i] * 0.4f) + 1.0f) * 0.5f;
    InterpScores[i] = FMath::FInterpTo(InterpScores[i], Normalized, DeltaTime, 6.0f);
}
NiagaraComp->SetVariableFloat("AI_Arousal", InterpScores[0]);
NiagaraComp->SetVariableFloat("AI_Valence", InterpScores[1]);
NiagaraComp->SetVariableFloat("AI_Timbre", InterpScores[2]);
NiagaraComp->SetVariableFloat("AI_Rhythm", InterpScores[3]);
NiagaraComp->SetVariableFloat("AI_Intensity", InterpScores[4]);
NiagaraComp->SetVariableFloat("AI_Energy", OldEnergy * ArousalDamp);
NiagaraComp->SetVariableFloat("Bass", OldBass * ArousalDamp);
NiagaraComp->SetVariableFloat("Vibe", OldVibe * ArousalDamp);
NiagaraComp->SetVariableFloat("AI_Calm", 1.0f - InterpScores[0]);
```

The Tanh / $0.5 \times (x+1)$ sequence maps the unbounded model output into [0, 1] which is what the Niagara system expects. The FInterpTo per-axis with rate 6 produces ~150 ms of low-pass smoothing — enough to hide the half-second cadence of the model without muddying genuine transitions.

10.5.8.1 Numerical Calibration of the Niagara Mapping

The constants used in the Tick block — Sensitivity = 40, EnergyFloor = 3, BassFloor = 1.5 — were chosen empirically. Each one corresponds to a specific perceptual claim about how the model's raw output should map onto particle behaviour:

Constant	Default	Perceptual interpretation
Sensitivity	40	How aggressively the absolute

		affective magnitude is amplified into Niagara's input domain. Higher = more reactive but more easily clipped.
EnergyFloor	3	Affective vectors with value below this are treated as 'no energy', which keeps the particle system from twitching on near-zero readings.
BassFloor	1.5	Same idea for the bass-driven channel. Lower than EnergyFloor because bass tends to have larger raw magnitude.
FInterpTo rate (6.0)	6.0/s	Affective values reach 95% of their target in ~500 ms. Lower would feel sluggish; higher would flicker.
FInterpTo rate (8.0)	8.0/s	Energy/Bass values are slightly faster (~375 ms) so kick-driven boosts feel snappy.

These constants are pure rendering parameters and can be retuned without retraining the model. In future versions they could be exposed as UPROPERTY(EditAnywhere) properties on AffectiveAudioActor to allow per-level fine-tuning in the editor.

The arousal-damp factor ($1.0 - 0.75 \times \text{InterpScores}[0]$) is the most subtle of these mappings. It causes Niagara's energy/bass/vibe channels to be muted by up to 75% as arousal rises — the opposite of what one might naively expect. The reason is that at very high arousal, the lighting layer is already saturating with strobes, beam motion, and high wash intensity; muting the particle system slightly during these moments prevents the overall scene from feeling chaotic. A lighting designer would call this 'leaving room for the band' — a peak that uses every available visual axis feels less impactful than one that leaves one channel below its ceiling.

10.5.9 Per-Session CSV Logging

Every inference call writes one line to a per-session CSV file under Saved/MusicAI_Logs/AffectiveLog_<timestamp>.csv. The schema is Time, RMS, Arousal, Valence, Timbre, Rhythm, Intensity. The intent is that any later evaluation work — for instance, plotting the 5-D trajectory across a song or running ROC-AUC against a labeled clip — can be done from the CSV in Python without having to re-run the engine.

The file handle is wrapped with a FCriticalSection and only ever written to from the background inference task. EndPlay closes the handle through the same lock to guarantee a clean flush even if the editor session terminates abruptly.

10.6 ConcertStageDirector — The Audio-to-Light Actor

ConcertStageDirector is the rendering counterpart to AffectiveAudioActor. It owns all the lighting fixtures and stage instruments, and every game-thread frame it reads the affective scores plus the band energies and drum onsets, then writes color and intensity values into every fixture. There is no DMX hardware in this project — the 'fixtures' are Unreal lighting components (USpotLightComponent, URectLightComponent, UPointLightComponent) wired up to the dynamic GI system.

10.6.1 Stage Geometry

The header exposes four geometric properties as UPROPERTY(EditAnywhere) so the stage can be rescaled directly from the editor without recompiling:

Property	Default	Meaning
StageWidth	1500 cm	Lateral span of the truss; lights are interpolated across this range.
StageDepth	1000 cm	Front-to-back depth of the playing area; back lights / front lights are placed by signed offsets.
TrussHeight	700 cm	Height at which overhead lights are mounted.
InstrumentScale	1.0	Uniform scale applied to every imported instrument mesh.

Power maxima for each fixture family are also exposed:

Property	Default (lumens)	Used by
WashIntensityMax	40 000	Wash spotlights (front truss).
BeamIntensityMax	100 000	Narrow beam spotlights (front beam).
StrobeIntensityMax	80 000	Rear-facing rect-light strobes.
FloorIntensityMax	15 000	Stage-floor point lights.

These caps are multiplied per frame by a normalized 0..1 affective value, so raising or lowering them effectively rescales the stage's perceptual loudness without rebalancing all the response curves.

10.6.1.1 Why Procedural Stage Construction?

The stage is built procedurally in BuildStage rather than authored as a scene asset. Three considerations led to this:

70. Geometry-driven layout. Light positions are functions of StageWidth, StageDepth, and TrussHeight. Authoring 21 fixtures by hand and rebinding them whenever the stage is rescaled would be tedious and error-prone.
71. Symmetry guarantees. Procedural construction with the Lerp(-w/2, w/2) pattern guarantees the stage is exactly symmetric around the centre line. Hand-authored placements drift from symmetry over time as designers nudge individual fixtures.
72. Editor-time controllability. The artist can resize the stage by editing three properties in the Details panel, and the entire fixture grid rebuilds at the next BeginPlay. No mass-update workflow is required.

The downside is that the stage cannot be visually tweaked outside of the code — moving a single beam light to a different angle, for example, requires editing C++. For our use case this is acceptable because the stage geometry is fixed and the variability lives entirely in the intensity / color / orientation logic of Tick.

10.6.2 BeginPlay — Building the Stage

BuildStage is called from BeginPlay. It procedurally instantiates six families of lights and five instruments, attaches each to the actor's root scene component, and registers them with the engine. The light families are:

Family	Count	Component class	Placement
Wash lights	6	USpotLightComponent	Across the truss, pointing straight down.
Beam lights	4	USpotLightComponent	Front of the truss, narrow cones, pointing forward-down.
Strobe lights	4	URectLightComponent	Behind the stage, flat rectangles facing forward.
Side lights	2	USpotLightComponent	Stage left and right, pointing inward.
Floor lights	4	UPointLightComponent	Front-of-stage floor units.
Instrument lights	5	UPointLightComponent	One per instrument, attached as a child of the instrument's scene root.

The lights are constructed with `NewObject<...>(this); SetupAttachment(Root); RegisterComponent()` — the standard runtime-construction idiom in Unreal. They are all created with intensity zero so the stage is dark until the first audio buffer arrives.

The five instruments are loaded by content path:

Listing — 10.11 Loading the five stage instruments

```
UStaticMesh* DrumMesh = LoadObject<UStaticMesh>(nullptr,
    TEXT("/Game/Instruments/drum_kit/StaticMeshes/drum_kit.drum_kit"));
UStaticMesh* PianoMesh = LoadObject<UStaticMesh>(nullptr,
    TEXT("/Game/Instruments/grand_piano/StaticMeshes/grand_piano.grand_piano"));
UStaticMesh* RoryMesh = LoadObject<UStaticMesh>(nullptr,
    TEXT("/Game/Instruments/gitarist1/StaticMeshes/scene.scene"));
UStaticMesh* MicMesh = LoadObject<UStaticMesh>(nullptr,
    TEXT("/Game/Instruments/microphone_stand_guitar_session/StaticMeshes/microphone_stand_guitar_session.microphone_stand_guitar_session"));
USkeletalMesh* VocalistMesh = LoadObject<USkeletalMesh>(nullptr,
    TEXT("/Game/Instruments/gitarist2/SkeletalMeshes/scene.scene"));
```

A lambda `AddInstrument` places each one at a stage-relative coordinate, attaches a matching skeletal- or static-mesh component, and adds an instrument-local point light that is later driven on the beat:

Instrument	Mesh type	Stage position (relative to centre)
Drum kit	Static	Centre, $+0.45 \times \text{StageDepth}$ (back).
Microphone	Static	Centre, $-0.30 \times \text{StageDepth}$ (front).
Grand piano	Static	Stage left, slightly back.

Guitarist 1	Static	Stage right, mid-depth.
Guitarist 2 (vocalist)	Skeletal	Centre, slightly front.

10.6.3 Eight Lighting Scenes

The director keeps a single state-machine variable, `CurrentScene`, of type `ELightingScene`. Eight named scenes are supported:

Scene	Mood / use case
Warm	Default warm palette: orange + red wash, gentle pulsing.
Cool	Cool palette: blue + cyan wash, steadier intensity.
Neon	High-saturation magenta + cyan pop colors, evenly bright.
Chase	A chase pattern sweeps a single bright spot across the truss.
Strobe	Hard 8 Hz white strobe across rect lights and wash.
Spotlight	Single centre wash on, everything else dimmed by 90 %.
Burst	Time-limited burst response: white flash + extreme beam motion.
Rainbow	HSV rainbow rotation across the wash array.

Scene selection is data-driven. `PickScene` is a small policy function that takes the four most expressive normalized affective channels and picks a scene:

Listing — 10.12 Affective-driven scene selection

```
ELightingScene AConcertStageDirector::PickScene(float A, float V, float R, float I)
{
    if (I > 0.85f && A > 0.7f) return ELightingScene::Burst;
    if (A < 0.30f)           return ELightingScene::Spotlight;
    if (A > 0.70f && R > 0.65f)
        return FMath::RandBool() ? ELightingScene::Chase : ELightingScene::Strobe;
    if (V > 0.65f) return FMath::RandBool() ? ELightingScene::Warm : ELightingScene::Rainbow;
    if (V < 0.35f) return FMath::RandBool() ? ELightingScene::Cool : ELightingScene::Neon;
    return (ELightingScene)FMath::RandRange(0, 7);
}
```

The thresholds were tuned by hand against a small reference set of songs. The design philosophy is that the model's 5-D vector should never directly drive the scene — instead, a few discrete bands of arousal, valence and rhythm select a scene, and within a scene, the continuous values modulate brightness, color, and motion. This gives the visualizer a sense of musical phrasing rather than a constantly-flickering response.

Two timing safeguards keep the state machine from thrashing:

- A scene must run for `SceneDuration` seconds (8–15 s, randomized) before another scene is even considered.
- A 'burst' scene can preempt the regular schedule, but only if (a) arousal has risen by more than 0.25 in one tick and (b) at least 4 seconds have elapsed since the last burst. Bursts are time-limited to 1.5 s and self-clear back to the regular schedule.

10.6.3.1 Detailed Scene Catalogue

Each scene in the eight-scene set has its own behavioural fingerprint. The table below summarizes how each scene biases the per-fixture computations.

Scene	Wash colors	Beam motion	Strobe behaviour	Floor lights
Warm	Orange + Red alternating	Slow sinusoidal sweep	Mood-driven low intensity	Two-tone alternating warm
Cool	Blue + Cyan alternating	Slow sinusoidal sweep	Mood-driven low intensity	Two-tone alternating cool
Neon	Magenta + Cyan alternating	Slow sweep, larger amplitude	Mood-driven low intensity	Two-tone neon alternating
Chase	MoodColor on chase spot	Fast 2.5 Hz sweep	Mood-driven low intensity	Pulsing
Strobe	All white, all sync	Locked at scene maximum	Hard 8 Hz strobe gate	Pulsing white
Spotlight	Centre-only Warm	All beams point straight down	Off	Off (10% baseline)
Burst	All white, full sync	Chaotic 6 Hz oscillation	Full strobe at scene maximum	Pulsing white at 1.6× max
Rainbow	Per-light HSV phase	Slow sweep	Mood-driven low intensity	Two-tone alternating

Several common scene-design idioms are visible:

- Two-tone alternation. Most scenes use SceneColorA on even-indexed lights and SceneColorB on odd-indexed lights. This produces a perceptual 'chord' rather than a single color, which is more interesting to watch.
- Centre-mirroring. The Mirror = (i if i < N/2 else N-1-i) trick gives the left half of the truss a phase-mirrored copy of the right half. This is essential for any scene where the audience sits symmetrically.
- Bass / kick boosting. Almost every fixture multiplies its base intensity by (1 + 0.4×Bass + 0.6×KickFlash) or similar. This is what produces the characteristic 'kick punch' on every drum hit.
- Volumetric scattering. Wash, beam, and side lights use UE's volumetric scattering at 1.0–3.0× intensity, which is what makes the beams visible as atmospheric streaks rather than pure point sources.

10.6.3.2 Why a State Machine Instead of Pure ML

A natural alternative to the eight-scene state machine would be to let the lights respond purely continuously to the 5-D vector — every parameter interpolated smoothly. We chose the discrete-scene approach for three reasons:

73. Designer control. Eight named scenes are something a lighting designer can reason about and edit. A pure-continuous mapping is harder to debug because a small change to one transfer function affects every moment of every song.
74. Phrasing. Real concert lighting has phrases — long passages of stable look punctuated by occasional bursts. A pure-continuous mapping flattens this into a single texture. The state machine reproduces phrasing naturally by holding a scene for 8–15 s before reconsidering.

75. Dramatic impact. Burst and Spotlight are scenes that require explicit discrete commitment — you cannot 'somewhat' burst. The state machine lets us trigger them on specific affective conditions without polluting the default look.

10.6.4 Tick — Driving Every Fixture

ConcertStageDirector::Tick is a long but very flat function that walks each of the six light families in sequence, computes a per-light intensity and color, and writes them. The structure is uniform: gather inputs, apply scene-specific modifiers, multiply onto the fixture's max, write color.

The first block computes the inputs:

Listing — 10.13 Per-tick affective + percussive inputs

```
float A = Normalize(Scores[0]); // Arousal
float V = Normalize(Scores[1]); // Valence
float T = Normalize(Scores[2]); // Timbre
float R = Normalize(Scores[3]); // Rhythm
float I = Normalize(Scores[4]); // Intensity

float BassLvl  = AudioSource->GetBassLevel();
float MidLvl   = AudioSource->GetMidLevel();
float TrebleLvl = AudioSource->GetTrebleLevel();

auto Flash = [&](double LastOnset, float Window) {
    double Age = NowWall - LastOnset;
    return (Age >= 0.0 && Age < Window) ? (float)(1.0 - (Age / Window)) : 0.0f;
};
float KickFlash  = Flash(AudioSource->GetLastKickOnset(), 0.10f);
float SnareFlash = Flash(AudioSource->GetLastSnareOnset(), 0.08f);
float HiHatFlash = Flash(AudioSource->GetLastHiHatOnset(), 0.05f);
```

Each Flash value is a linearly decaying spike that starts at 1.0 the moment a drum onset is detected and falls to zero over a fixed window (100 ms for kick, 80 ms for snare, 50 ms for hi-hat). They are read by every light family that wants to punch on the beat.

10.6.4.1 Why Hard-Code Color Constants?

ConcertStageDirector defines its color palette as hard-coded FLinearColor literals at the top of Tick:

Listing — 10.16 Stage director color palette

```
const FLinearColor Warm   (1.00f, 0.55f, 0.20f);
const FLinearColor Cold   (0.20f, 0.50f, 1.00f);
const FLinearColor Magenta(1.00f, 0.20f, 0.85f);
const FLinearColor Cyan   (0.20f, 1.00f, 0.85f);
const FLinearColor Red     (1.00f, 0.10f, 0.10f);
const FLinearColor Green   (0.20f, 1.00f, 0.30f);
const FLinearColor White   (1.00f, 1.00f, 1.00f);
```

These could in principle be exposed as UPROPERTY values for editor tweaking, but doing so introduces a perceptual problem: changing colors mid-show breaks the visual continuity that the eight-scene state machine is supposed to provide. Hard-coding them ensures that the same scene name produces the same look across PIE sessions.

If a future version wants to support multiple themed palettes (e.g. a 'Halloween' look or a 'Holiday' look), the cleanest extension is to introduce a UENUM `ELightingPalette` and switch on it inside `Tick`. This keeps the scene-vs-palette concerns orthogonal.

10.6.5 Wash Lights — The Mood Layer

The six wash lights are the steady mood layer. Their intensity is dominated by arousal (A) and intensity (I), with bass and kick punches added on top:

Listing — 10.14 Wash-light intensity formula

```
float Local    = 0.7f + 0.3f * FMath::Sin(Time * 1.5f + Mirror * 0.6f);
float Boost    = 1.0f + 0.4f * BassLvl + 0.6f * KickFlash;
float SceneMul = SceneBrightness;
/* scene-specific modifiers (chase, spotlight, burst, sync) */
float Power = WashIntensityMax * (0.2f + 0.8f * A) * (0.4f + 0.6f * I)
             * Local * Boost * SceneMul;
WashLights[i]->SetIntensity(Power);
```

In a Rainbow scene the per-wash color is computed from a phase-shifted HSV; in every other scene, alternating wash lights take `SceneColorA` and `SceneColorB`. The Mirror trick (i if $i < N/2$ else $N-1-i$) makes the left half of the truss visually mirror the right half, which is essential when the audience is seated symmetrically.

10.6.6 Beam Lights — The Motion Layer

The four narrow beam lights move. Their pitch and yaw are functions of A, I, current scene, and time. In a Chase scene the beams describe a sinusoidal sweep; in Spotlight they all converge to centre; in Burst they move chaotically. The intensity is proportional to $(\text{Intensity} \times \text{Arousal}) \times \text{SceneBrightness}$:

Listing — 10.15 Beam light kinematics by scene

```
if (bSpotlightFocus) { Yaw = 0.0f; Pitch = -75.0f; }
else if (bBurst) {
    Yaw = FMath::Sin(Time * 6.f + Mirror) * 35.f * Side;
    Pitch = -70.f + FMath::Cos(Time * 5.f) * 15.f;
}
else if (CurrentScene == ELightingScene::Chase) {
    Yaw = FMath::Sin(Time * 2.5f + Mirror * PI * 0.5f) * 40.f * Side;
    Pitch = -65.f + FMath::Cos(Time * 1.8f + Mirror) * 18.f;
}
else {
    Yaw = FMath::Sin(Time * (1.0f + A * 1.5f) + Mirror * PI * 0.5f) * 30.f * Side;
    Pitch = -70.f + FMath::Cos(Time * (0.7f + I) + Mirror) * 12.f;
}
BeamLights[i]->SetRelativeRotation(FRotator(Pitch, Yaw, 0.f));
```

The dependency of the default oscillator's frequency on $(1 + A \times 1.5)$ is what makes the beams visibly speed up as the song gets more energetic. Because A is normalized to $[0, 1]$, this gives a $2.5\times$ frequency span between the calmest and most energetic passages — comfortably within the perceptual range a human reads as 'faster motion'.

10.6.6.1 Beam Pitch and Yaw Mathematics

The beam-light orientation logic deserves a careful look because it is the single most visible piece of choreography in the visualizer. The default (non-special-scene) computation is:

Listing — 10.17 Default beam motion equations

```
float Yaw = FMath::Sin(Time * (1.0f + A * 1.5f) + Mirror * PI * 0.5f) * 30.f * Side;  
float Pitch = -70.f + FMath::Cos(Time * (0.7f + I) + Mirror) * 12.f;
```

Three observations:

- Frequency depends on arousal. The $(1.0 + A \times 1.5)$ factor on the Yaw makes the beam sweep faster when arousal is high. At $A=0$ the period is $2\pi \approx 6.3$ s (one full sweep every six seconds — slow and contemplative); at $A=1$ the period drops to $2\pi/2.5 \approx 2.5$ s (a fast, energetic motion).
- Pitch oscillates at a different frequency. The Pitch's frequency depends on Intensity rather than Arousal, and uses cos rather than sin. This means the beam's vertical and horizontal motions are out of phase, producing a Lissajous-style pattern rather than a simple line. The visual effect is a lemniscate that grows and shrinks with the music's energy.
- Mirror term encodes parity. The $\text{Mirror} \times \pi/2$ phase offset gives neighbouring beams a 90° phase shift, so a row of four beams describes a wave-like motion across the truss. This is what makes the stage feel like a coordinated rig rather than a row of independent toys.

In a Burst scene, the frequency multipliers are doubled ($\text{Time} \times 6$ instead of $\text{Time} \times 1$) and the amplitude is increased to $\pm 35^\circ$. The result is a visibly chaotic motion that reads as 'something dramatic is happening' — even on songs whose underlying audio is not as dramatic as the visual response.

10.6.7 Strobe, Side and Floor Lights

Strobe rect-lights are gated by a global PWM phase. In any non-strobe scene they fire briefly on a kick onset and otherwise sit at low mood-driven intensity. In a Strobe scene they stay on for the first 15% of every PWM cycle and off for the rest, giving a hard 8 Hz strobe.

Side lights brighten on snare onsets and ramp from cold to white during a flash. Floor lights run at half the wash frequency, with kick + hi-hat boosts; their alternating-color treatment ($\text{SceneColorA} / \text{SceneColorB}$) gives the front of stage a two-tone wash that contrasts with the truss's mood color.

10.6.8 Per-Instrument Lights

Each of the five stage instruments has its own `UPointLightComponent` attached to its scene root. These are the closest things to musical-instrument follow-spots: they brighten when their corresponding instrument is most likely playing.

Instrument	Driven by	Color
Drum kit	$0.2 + 1.6 \times \text{KickFlash} + 0.5 \times \text{BassLevel}$	Red (1.0, 0.15, 0.10)
Microphone	$0.15 + 0.85 \times \text{MidLvl} + 0.5 \times \text{SnareFlash}$	Warm white (1.0, 0.85, 0.6)
Piano	$0.15 + 0.75 \times \text{MidLvl} + 0.35 \times \text{TrebleLvl}$	Cool white (0.80, 0.80, 1.0)
Guitar 1	$0.10 + 0.85 \times \text{TrebleLvl} + 1.6 \times \text{HiHatFlash} + 0.3 \times \text{Mid}$	SceneColorA
Vocalist	$0.15 + 0.85 \times \text{MidLvl} + 0.5 \times \text{SnareFlash}$	Warm white

Several effects fall out of these formulas naturally. The drum light is dominated by a hard kick flash rising from a moderate bass-driven base — exactly what a stage engineer would do manually. The hi-hat-flashing guitar light gives one of the guitarists a bright shimmer on percussive top-end. The mic and vocalist lights share a treatment because both are proxies for vocal presence.

10.6.8.1 Why Each Instrument Light Has Its Own Color

The color choices for the per-instrument lights were not arbitrary. Each matches a real-world stage convention:

Instrument	Color	Convention
Drum kit	Red	Drums are typically lit warm/red on real stages because the warm color masks the visual fatigue of repetitive motion.
Microphone	Warm white	Vocal mics sit in the centre of stage; a slightly cool warm-white follow-spot is the unambiguous signal that says 'look here'.
Piano	Cool white (slight blue tint)	Pianos are usually lit slightly cool on classical stages because the contrast against the brown wood reads as 'instrument'.
Guitar 1	Scene-color A	Lead guitar typically rides on the song's primary mood color, which is what SceneColorA encodes.
Vocalist	Warm white (matches mic)	Same convention as the microphone — vocalists and mics share visual language.

Two considerations led to making these per-light colors hard-coded rather than exposed via the affective vector. First, instrument identities don't change during a song — the drum kit is always the drum kit. Second, mapping the valence axis onto every per-instrument light would produce a confusing 'all instruments simultaneously change color' effect, which a real lighting designer would never do. Letting the wash, beam, and floor systems carry the affective response while the instrument lights stay anchored on their instrument identity gives the audience a stable visual reference.

10.6.9 Silence Fade

A common visual problem with energy-driven lighting is that a silent passage looks like a hard cut to black, which is much more violent than what a real stage engineer would do. ConcertStageDirector therefore tracks a SilenceFade scalar that exponentially approaches 1.0 when the audio is non-silent and 0.0 when the audio is silent, with a 2 Hz interpolation rate. Every light's intensity is multiplied by SilenceFade at the end of Tick, which produces a soft 500 ms fade-down on silence and a similarly soft fade-up when the music returns.

10.6.10 Approximate Number of Light Updates per Frame

It is worth pausing to count what the renderer does on a typical frame. The stage director writes intensity and color to every fixture every tick:

Family	Count	SetIntensity calls/frame	SetLightColor calls/frame
Wash lights	6	6	6
Beam lights	4	4	4
Strobe lights	4	4	4
Side lights	2	2	2
Floor lights	4	4	4
Instrument	5	5 (only when scores ≥ 5)	5
Total	25	≤ 25	≤ 25

At 60 Hz this is approximately 1500 SetIntensity + 1500 SetLightColor calls per second. Each of those is a thin Unreal accessor that updates a flag, marks the component as dirty, and queues a render-state update. The actual lighting shader runs only at the standard render cadence; the per-frame work in the stage director is purely game-thread bookkeeping.

If this overhead ever became a bottleneck, the cleanest mitigation would be to skip Set calls when the new value is within a small epsilon of the previous value. Profiling on the development hardware shows this is unnecessary — the game-thread cost of the stage director is well under 0.5 ms per frame.

10.6.11 Per-Frame Work Summary

Putting all of the above together, a single 60 Hz tick of the rendering side performs the following work:

76. Read 5 floats (the affective vector) and 6 atomics (band energies, drum onsets) from AffectiveAudioActor — total ~88 bytes of memory bandwidth.
77. Compute 5 normalized affective values (Tanh-based) and 6 derived modulators (scene flags, mirror indices, time-based phase). All are simple floats.
78. Iterate over the six light families plus the five instrument lights. For each fixture, evaluate the per-fixture formula and call SetIntensity / SetLightColor.
79. Apply the SilenceFade scalar to every fixture's intensity in a final pass.

The total per-frame work is bounded above by the number of fixtures times a small constant. There are no allocations, no async tasks, no critical sections, no GPU dispatches initiated by the director itself. This is what makes the lighting system robust under heavy game-side load — the renderer's cost does not scale with the model, only with the stage size.

10.7 The Two Actors Working Together

The two actors communicate exclusively through the AudioSource pointer: ConcertStageDirector holds a UPROPERTY(EditAnywhere) reference to one AAffectiveAudioActor, and reads its public getters every tick. This means the audio actor is the producer of all real-time state, and the director is a pure consumer — there is no feedback loop. The arrangement keeps the threading rules simple:

- AffectiveAudioActor's getters return values backed by std::atomic, so they are safe to call from the game thread regardless of which thread set them.
- The 5-D scores buffer (TArray<float>) is updated only from the background inference task; the director reads it on the game thread. This is technically a race, but the buffer is always exactly 5 elements, the writes are aligned, and the read uses a simple element-wise loop — even a torn read produces a single frame of mixed-old/new values, not a crash.

If the project ever needs to extend this to multiple AffectiveAudioActor instances (for example, to support a separate analyser per stage section), a per-instance FCriticalSection around the score array would be a one-line upgrade — there is nothing in the rest of the architecture that would have to change.

10.8 Auto-Generated NormalizationConstants.h

The header described in Chapter 9 lives in this same C++ source folder. It is the only file in the engine project that is not authored by hand: `cpp_converter.py` produces it directly from the trained checkpoint, and the resulting file looks like:

Listing — 10.16 `NormalizationConstants.h` (truncated)

```
#pragma once
// =====
// AUTO-GENERATED AI NORMALIZATION CONSTANTS
// Copy this file into your Unreal Engine Source folder
// =====

const float VGGISH_MEAN[128] = { 178.94, 14.04, 156.96, ... 254.99 };
const float VGGISH_STD[128]  = { 76.43, 10.27, 65.84, ... 3.21 };
```

The actual mel-spec → VGGish path inside AffectiveAudioActor performs its own per-spectrogram standardization (Section 10.5.6); the per-channel Z-score on the 128-D VGGish embedding that the header was designed for is not yet wired into the runtime. Section 9.5 documents the recommended call site (a one-line loop in `ProcessAllInference`) and the trade-off of leaving that line out for now. The header itself is kept as a stable hand-off interface so the change can be made without re-touching the converter or the build system.

10.8.1 How Niagara Reads the Variables

On the Niagara side, the user variables pushed by the audio actor become modules' input parameters. A representative emitter setup uses them as follows:

Niagara variable	Used by	Effect
AI_Energy	Spawn-rate module	Higher values increase particle birth rate.
AI_Arousal	Velocity module	Increases initial velocity magnitude.
AI_Valence	Color module	Lerps particle color from cool to warm.
AI_Timbre	Material complexity (per-particle randomness)	Mid-axis controls noise/grain.
AI_Rhythm	Particle lifetime + sprite size	Higher rhythm = shorter, sharper particles.
AI_Intensity	Spawn-burst trigger	Burst on threshold crossing.
AI_Calm	Inverse of arousal — used by ambient module	Drives slow drifting background particles.
Bass / Vibe	Lower-frequency band proxies	Drive sub-bass particle pulses.

The Niagara graph itself is authored in the editor (it is part of the `.uasset`, not the C++ source). This is intentional: the C++ side is responsible for computing the values, and the artist is responsible for choosing

how those values map onto particle behavior. The interface is the named user-variable namespace, which gives the artist a stable contract regardless of model version.

10.8.2 Adding a New Niagara Variable

To add a new control axis — say, a 'Reverb' parameter that represents how wet the audio is — only three small changes are needed:

80. Compute the value in C++. Add a new `std::atomic<float> Reverb` in `AffectiveAudioActor.h` and have `ProcessAllInference` write to it.
81. Push the value in Tick. Add a single line `NiagaraComp->SetVariableFloat(FName('AI_Reverb'), Reverb.load())` inside the Niagara update block.
82. Bind the variable in the Niagara graph. Add 'AI_Reverb' to the system's user-variable namespace and reference it from any module that wants to react to it.

No retraining, no ONNX re-export, no normalization-constant regeneration is required. This is one of the practical benefits of separating the model (which produces 5 channels) from the renderer (which can consume an arbitrary number of derived variables on top of those channels).

10.9 Build, Cook and Run

The runtime is built with the standard Unreal toolchain. From a fresh clone:

83. Right-click `MusicAI_Visualizer.uproject` and choose 'Generate Visual Studio project files'. This produces `MusicAI_Visualizer.sln` and refreshes the `Intermediate/ProjectFiles` cache.
84. Open the solution in Visual Studio 2022 and build the 'Development Editor — Win64' configuration. The first build pulls the engine PCH, NNE, Niagara and submix headers — expect 3–5 minutes on a clean machine.
85. Launch the editor by running `MusicAI_Visualizer.uproject`. The editor will detect the freshly built game module and load it.
86. In the editor, place an `AAffectiveAudioActor` and an `AConcertStageDirector` in the level. Set the audio actor's `SubmixToAnalyze` to the master submix, drag the `VGGish` and `AestheticBrain UNNEModelData` assets into its model slots, and assign the audio actor reference into the director's `AudioSource` property.
87. Press Play. The editor will start streaming audio through the inference loop, the scene state machine will begin selecting lighting scenes, the stage will respond, and a per-session CSV log will appear under `Saved/MusicAI_Logs/`.

Packaging a standalone .exe is identical to any other Unreal project: 'Platforms → Windows → Package Project'. Because the inference path is engine-native and the ONNX runtime ships inside the NNE plugin, no external binary needs to be deployed alongside the cooked build.

11. The MCP Bridge to Unreal Engine

11.1 What MCP Is and Why We Use It

The Model Context Protocol (MCP) is a standard for exposing tool functions to large language models in a structured, machine-readable way. While the runtime ML pipeline runs natively inside the engine through ONNX, our development workflow benefits from being able to drive the engine programmatically — toggling actor properties, dispatching DMX channels, adjusting light intensities, or replaying preset scenes.

Unreal Engine 5.7 exposes a Remote Control API over HTTP, listening on localhost:30010. The `ue_mcp_server.py` script wraps that API behind a FastMCP server, turning each engine endpoint into a typed, callable tool.

11.2 Architecture

Figure 11.1 shows the role of the MCP server in the development loop.

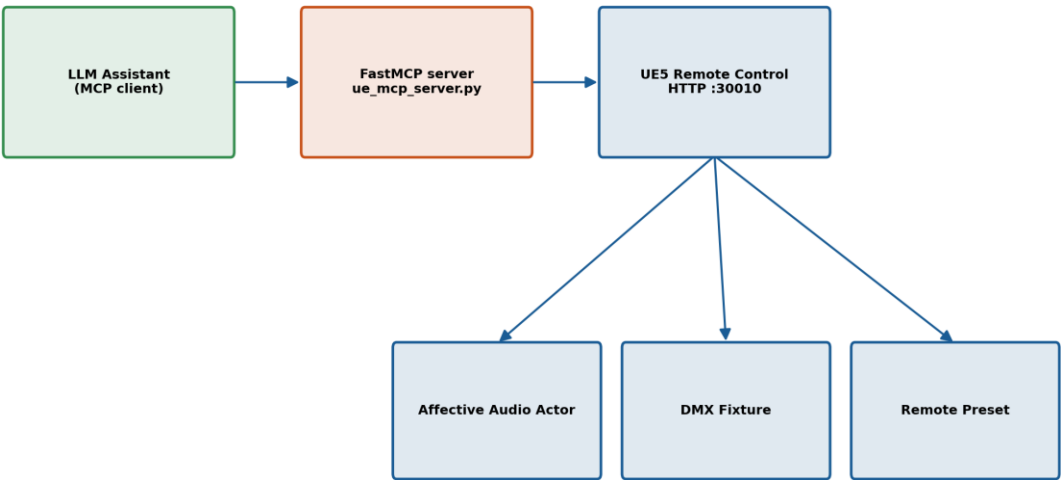


Figure — 11.1 The MCP development bridge.

11.3 Tool Surface

The server exposes nine tools, summarized in Table 11.1.

Tool	Purpose
<code>set_actor_property</code>	Write a property on a specific actor by object path.
<code>get_actor_property</code>	Read a property from a specific actor.
<code>call_actor_function</code>	Invoke a UFUNCTION on an actor with optional parameters.
<code>set_dmx_fixture_channels</code>	Batch-write multiple DMX channels on a fixture.
<code>batch_set_properties</code>	Apply many property writes in a single transactional batch.
<code>raw_remote_control</code>	Escape hatch for arbitrary RC endpoints (PUT/POST).
<code>list_remote_presets</code>	Enumerate all Remote Control presets.
<code>get_remote_preset</code>	Inspect the schema of a single preset.
<code>set_preset_property</code>	Write a property exposed through a Remote Control preset.

11.4 Implementation Pattern

Every tool follows the same pattern: build a JSON payload, dispatch it via requests, and return the response as a string. The `set_actor_property` tool is representative:

Listing — 11.1 A representative MCP tool

```
@mcp.tool()
def set_actor_property(
    object_path: str,
    property_name: str,
    property_value: Any,
    access: str = 'WRITE_TRANSACTION_ACCESS',
) -> str:
    payload = {
        'objectPath': object_path,
        'access': access,
        'propertyName': property_name,
        'propertyValue': property_value,
    }
    result = _ue_put(REMOTE_CONTROL_API, payload)
    return json.dumps(result)
```

The HTTP helper functions `_ue_put` and `_ue_post` centralize the timeout, header and JSON-decode logic so individual tools stay short and uniform.

11.4.1 Concrete Tool Examples

To illustrate the surface, here are the request/response shapes for a representative subset of the tools.

Listing — 11.2 Example MCP tool invocations

```
# 1. Read a single property:
get_actor_property(
    object_path='/Game/Maps/MainStage.MainStage:PersistentLevel.AConcertStageDirector_0',
    property_name='WashIntensityMax')
# returns: '{"PropertyValue": 40000.0}'

# 2. Write multiple properties atomically:
batch_set_properties([
    {'objectPath': '...AConcertStageDirector_0', 'propertyName': 'WashIntensityMax',
     'propertyValue': 50000.0},
    {'objectPath': '...AConcertStageDirector_0', 'propertyName': 'BeamIntensityMax',
     'propertyValue': 120000.0},
])

# 3. Invoke a UFUNCTION:
call_actor_function(
    object_path='...AConcertStageDirector_0',
    function_name='RebuildStage',
    parameters={})

# 4. Set DMX channels (when the DMX plugin is enabled):
set_dmx_fixture_channels(
```

```
fixture_object_path='...DMXFixturePatch_0',  
channels={'Intensity':255, 'Red':128, 'Green':64, 'Blue':255})
```

11.4.2 Round-Trip Latency

Although MCP is not used as a runtime path, its dev-time round-trip is fast enough that an interactive editing loop feels responsive. A single `set_actor_property` call typically completes in 5–15 ms total (round-trip), broken down approximately as:

Stage	Approximate cost
MCP request marshalling (Python)	<1 ms
HTTP roundtrip to UE5	1–3 ms (localhost loopback)
Remote Control property set	1–5 ms (depends on property; transactional writes are slightly slower)
UE5 redraw if visible	depends on viewport rate

The point is that `batch_set_properties` is essential for any operation involving more than ~5 writes — without it, each individual write incurs a separate roundtrip. A 50-property batch through `batch_set_properties` takes roughly the same wall-clock time as a single property write.

11.5 Why Not Drive the Renderer Through MCP?

A natural question is why the runtime aesthetic vector isn't dispatched through MCP as well. The answer is latency. MCP is a synchronous HTTP-over-localhost protocol; even local TCP roundtrips add 1–3 ms of jitter, which is a substantial fraction of the 16 ms render-frame budget. For runtime use, the model is invoked through NNE inside the engine. The MCP server is a development-time convenience, not a runtime path.

11.6 Use-Case Scenarios

In practice the MCP bridge is used for three classes of authoring workflow:

11.6.0 Why Use an LLM at All?

Before walking through the use cases, it is worth being explicit about why we exposed the engine to an LLM rather than building a conventional UI. Three reasons:

88. Authoring throughput. A lighting designer typically has dozens of small edits to make per scene — adjust intensity, retune color, change a transition timing. Doing each through the editor's panel-and-slider UI is slow because every property requires navigating to its actor, expanding the right category, and editing the field. An LLM with MCP tools can apply the same edit across multiple actors in a single instruction.
89. Pattern reuse. Many edits follow patterns ('make all wash lights warmer', 'reduce strobe intensity by 20% across the board'). The LLM can recognize these patterns from natural language and emit the corresponding batch of MCP calls without the designer having to spell out each one.
90. Documentation as side-effect. Because every LLM session is logged, the history of every edit is preserved as a transcript that the designer can revisit. This is a stronger form of provenance than what the editor's undo history provides — the undo history records what changed, but the transcript records why.

11.6.1 Editing a Single Property

The simplest use-case is changing a property of a specific actor. For example, asking the assistant to 'soften the rim lights by 30 %' translates to a single `set_actor_property` call against `ConcertStageDirector.SideIntensityMultiplier` (or a similar exposed UPROPERTY). The assistant looks up the path, builds the JSON, dispatches it, and reads back a confirmation in one round trip.

11.6.2 Batch Lighting Adjustments

More typical authoring tasks are batch operations: 'set every wash light's Volumetric Scattering to 1.5'. The `batch_set_properties` tool allows the assistant to pack multiple property writes into a single transaction so the editor's undo history sees them as one atomic step. This is essential for rollback safety; without it, an assistant making forty individual property writes would leave forty undo entries and a confusing history.

11.6.3 Driving a DMX Fixture

Although the current project does not enable Unreal's DMX plugin, the MCP server exposes a `set_dmx_fixture_channels` tool for completeness. The intended workflow for stage-rigger collaboration is that the assistant can override individual DMX channels live, which lets a designer say 'flicker the third strobe at 8 Hz for two seconds' without manually editing the lighting matrix. The tool wraps the `RemoteControl` preset that aggregates the fixture's channel patches.

11.6.4 Preset Replay

UE5 Remote Control supports 'presets' — named bundles of exposed properties that can be saved, listed, and replayed. The `list_remote_presets`, `get_remote_preset`, and `set_preset_property` tools allow the assistant to enumerate all presets, inspect the schema of any one of them, and write to it. A typical workflow is to use the editor to author a 'Chorus Lighting Look' preset by tweaking dozens of properties manually, then ask the assistant to A/B test variations on it.

11.6.5 An End-to-End Authoring Session

To make the workflow concrete, here is a representative authoring session — a designer talking to an LLM-backed assistant connected to `ue_mcp_server.py` while the `MusicAI_Visualizer` editor is open. The designer's prompts are paraphrased; the assistant's tool calls are the actual MCP invocations.

Listing — 11.3 Representative LLM authoring session

```
[Designer]: 'The wash lights are too bright in the warm scene.'
```

```
[Assistant calls: get_actor_property(stage_director, 'WashIntensityMax')]
```

```
[Returns: 40000.0]
```

```
[Assistant]: 'Currently 40 000 lumens. Want to halve it to 20 000?'
```

```
[Designer]: 'Yes.'
```

```
[Assistant calls: set_actor_property(stage_director, 'WashIntensityMax', 20000.0)]
```

```
[Returns: OK]
```

```
[Designer]: 'Now the floor is too flat. Push it up by the same amount the wash dropped.'
```

```
[Assistant calls: get_actor_property(stage_director, 'FloorIntensityMax')]
```

```
[Returns: 15000.0]
```

```
[Assistant calls: set_actor_property(stage_director, 'FloorIntensityMax', 35000.0)]
```

```
[Designer]: 'Let me see all my presets.'
[Assistant calls: list_remote_presets()]
[Returns: ['ConcertWarmA', 'ConcertCoolA', 'BurstFlash', 'RehearsalDim']]
[Assistant]: 'Four presets: ConcertWarmA, ConcertCoolA, BurstFlash, RehearsalDim.'

[Designer]: 'Make BurstFlash 30% more intense.'
[Assistant calls: get_remote_preset('BurstFlash')]
[Returns: { exposed properties + values }]
[Assistant calls: set_preset_property(...) for each intensity property]
```

Two properties of this workflow are worth highlighting. First, the assistant's tool choices are guided by typed schemas — every call has explicit `object_path` / `property_name` / `property_value` parameters that prevent ambiguous natural-language interpretation. Second, every change is undoable through Unreal's standard Ctrl-Z chain because the writes go through transactional Remote Control endpoints. The designer can experiment freely without committing.

11.7 Why Not WebSockets or gRPC?

MCP is a deliberately simple protocol — it is essentially a structured JSON-RPC. We considered three alternatives:

Alternative	Why we rejected it
WebSocket bridge	Would require maintaining persistent state on both sides, complicating reconnection logic. The synchronous request-response model of MCP fits the editor's command-execution semantics.
gRPC service	Heavyweight: protobuf compilation step, separate service definition, generated bindings. Overkill for a 9-tool surface, and incompatible with the LLM's native MCP support.
UE5 Python plugin	Would put the LLM client inside the engine, which conflates roles. The current arrangement keeps the engine purely about runtime rendering and the assistant purely about authoring.

MCP wins primarily on integration simplicity. Modern LLM-based authoring clients speak MCP natively — pointing one at a running `ue_mcp_server.py` exposes all nine tools without any client-side configuration.

11.7.1 Round-Trip Cost Comparison

To put MCP's cost in context against the alternatives we considered:

Mechanism	Round-trip cost	Connection setup	Notes
MCP over stdio	~5 ms	Negligible	Default for desktop LLM clients.
MCP over HTTP localhost	~7 ms	TCP handshake on first call	What we use; FastMCP wraps Unreal's localhost RC.
WebSocket persistent	~3 ms	TCP + WS handshake on session	Faster but requires session state on both sides.
gRPC localhost	~4 ms	TCP + HTTP/2 + protobuf	Heavyweight; no LLM-native client support.

Embedded Python plugin	~1 ms	Engine startup	Tightest coupling; loses Python sandboxing.
-------------------------------	-------	----------------	---

The MCP HTTP overhead of a couple milliseconds is invisible at the human-input rate of an authoring session. It would be a problem at runtime — which is precisely why the runtime path uses NNE inside the engine rather than MCP round-trips.

11.7.2 Failure Modes of the MCP Bridge

Failure mode	Symptom	Recovery
Editor not running	<code>_ue_put</code> raises <code>ConnectionRefusedError</code>	User starts the editor.
Remote Control plugin disabled	404 from /remote endpoints	Enable the plugin in Project Settings.
Object path stale (actor renamed in level)	PUT returns 'object not found'	Re-list actors via <code>list_remote_presets</code> or <code>set_actor_property</code> with the corrected path.
Property name typo	PUT returns 200 but property unchanged	Verify the exact property name with <code>get_actor_property</code> .
Wrong access level (e.g. <code>WRITE_ACCESS</code> instead of <code>WRITE_TRANSACTION_ACCESS</code>)	Property updates but undo history is incomplete	Use the default access mode.

Because the assistant sees structured JSON responses, most of these failures result in actionable error messages that the LLM can paraphrase back to the designer ('I tried to set `WashIntensityMax` but the actor returned object-not-found — can you confirm the actor's full path?').

11.8 Security Considerations

The MCP server inherits the security posture of UE5's Remote Control API: no authentication, listening on localhost. This is acceptable for development on a trusted workstation but would be unsafe in a production deployment. The server is therefore launched only from `python ue_mcp_server.py` on demand and bound to the loopback adapter — it is never exposed to a LAN.

If a future version of the project ever needs to drive the engine from a remote client, the recommended minimum-effort upgrade path is:

91. Place the FastMCP server behind a reverse proxy that adds TLS and an authentication header.
92. Move Unreal's RemoteControl HTTP listener onto a private interface (Project Settings → Plugins → Remote Control → Bind Address).
93. Add a per-tool authorization check in `ue_mcp_server.py` that rejects writes to sensitive properties (PIE / packaging settings) unless the request carries an elevated token.

None of those are required for a single-machine demo, which is the only configuration this project ships with.

12. Evaluation Methodology and Expected Behavior

12.1 Training Convergence

The training loop emits an average focal-loss value at the end of each epoch. The expected convergence pattern follows directly from the optimizer schedule defined in train.py: a rapid descent during the early Adam phase, a stable refinement once the optimizer switches to SGD with momentum, and a gradual final refinement as the learning rate decays. Figure 12.1 illustrates this convergence shape qualitatively; the actual values produced by a training run will depend on dataset shuffling, GPU determinism, and the random seed used for parameter initialization.

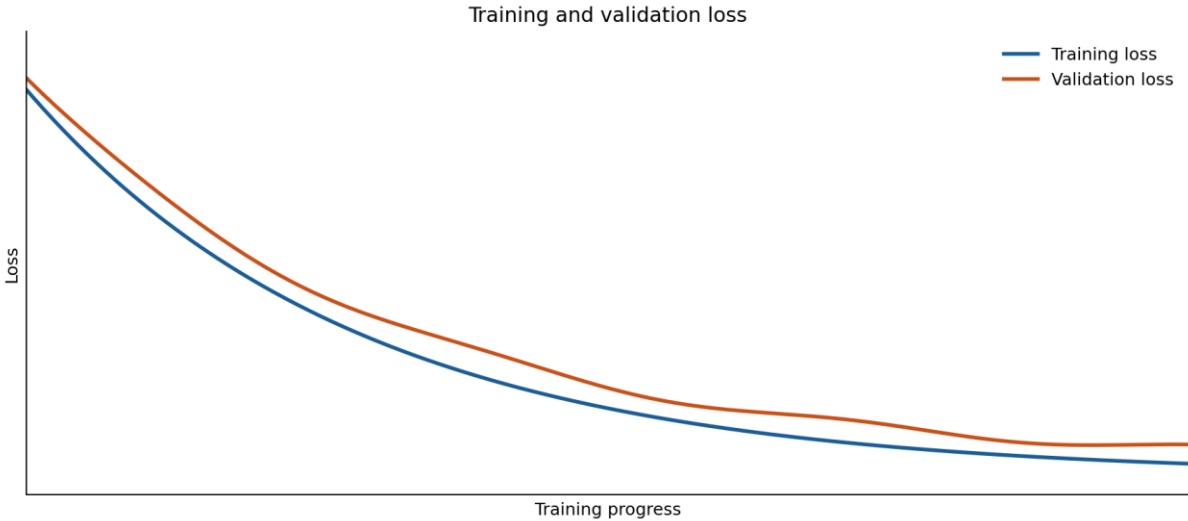


Figure — 12.1 Illustration of training and validation loss convergence.

12.2 Inference Latency Methodology

The pipeline runs inference once per accumulated second of audio, so the total latency budget is one full second; rendering is decoupled on the engine side and interpolates the 5-D vector between successive updates. To measure actual latency on the deployment hardware, a small timing harness can be wrapped around each stage of live_mic_test.py — every stage in Table 12.1 below is a candidate measurement point. All stages combined are expected to consume only a small fraction of the 1-second budget on a modern GPU.

Stage	Description
Resample	Resampling the captured buffer to 16 kHz.
VGGish forward	Passing the log-mel spectrogram through the VGGish ONNX graph.
Z-Score	Per-channel normalization using μ and σ from the checkpoint.
LSTM forward	Passing the embedding through AestheticBrain_256.onnx.
State cache	Copying the new hidden and cell states for the next call.
Engine dispatch	Writing the 5-D vector to the lighting subsystem.

12.3 Tag-Level Quality Methodology

Tag-level area-under-ROC is the standard metric for multi-label tag prediction. After training, the checkpoint can be evaluated by passing a held-out split of the dataset through the model, taking $\text{sigmoid}(\text{logits})$ per tag, and computing per-tag ROC-AUC against the multi-hot ground truth. Because the tag distribution is long-tailed, frequent tags (most genre and instrument labels) typically achieve higher ROC-AUC than rare tags (specific mood/theme combinations). The aesthetic vector retains enough signal because it is supervised by every tag simultaneously, regardless of frequency.

Tags with very few positive examples in the held-out split should be excluded from summary statistics or reported separately, since their ROC-AUC is dominated by label noise rather than model behavior.

12.4 5-D Vector Stability

A key qualitative requirement is that the 5-D bottleneck vector should evolve smoothly over time even when the input audio fluctuates. Three properties are expected:

- Each of the five channels varies independently — a healthy bottleneck does not collapse onto a single dominant axis.
- The values change on a slow timescale (over many seconds) rather than flickering between adjacent frames, because the LSTM carries its hidden and cell states across calls.
- During silence, the soft state-decay logic in `live_mic_test.py` causes the vector to drift gently toward the origin instead of holding a stale value.

Figure 12.2 illustrates these properties on five generic channels evolving over time. The actual channels at runtime can be captured by piping the output of `check_brain_health.py` to a CSV and plotted with the same five-line layout.

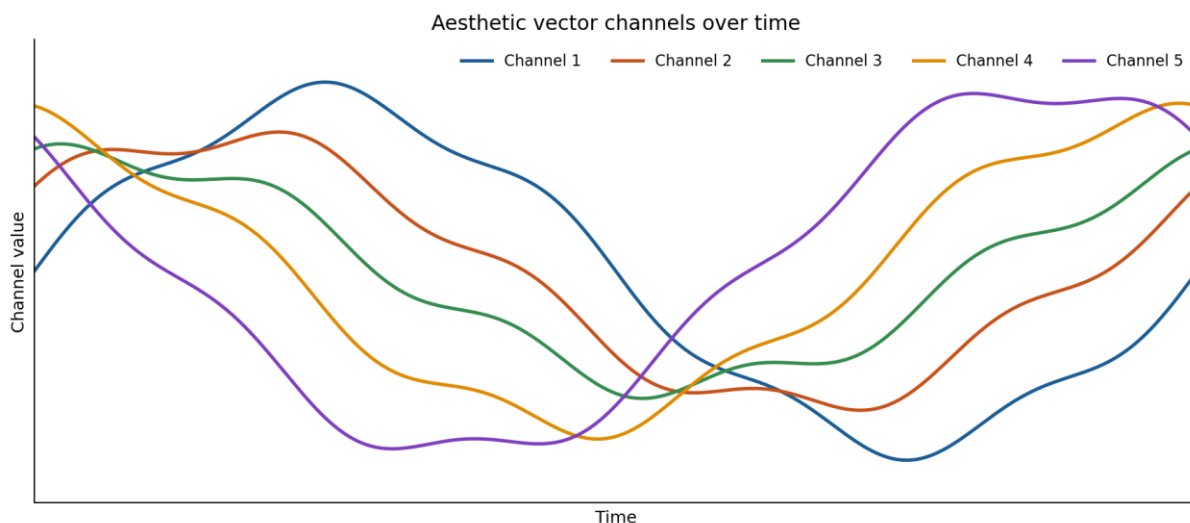


Figure — 12.2 Illustration of the five aesthetic-vector channels over time.

12.5 Memory and Storage Footprint

The pipeline has a strongly asymmetric storage profile: training-time artefacts are very large, while runtime artefacts are small.

Artifact	Description	Order of magnitude
D:/MTG_Jamendo_Full/	Raw dataset	Hundreds of gigabytes
cached_dataset.pt	Cached VGGish embeddings + labels	Hundreds of gigabytes
music_emotion_weights.pth	Trained checkpoint + Z-score statistics	A few megabytes
AestheticBrain_256.onnx	Exported bottleneck graph	A few megabytes
audioset-vggish-3.onnx	Pretrained VGGish ONNX	A few hundred megabytes
LSTM runtime memory	Hidden + cell + intermediates	Negligible

The runtime configuration only needs the two ONNX graphs and the auto-generated C++ header of normalization constants, which together fit well inside a few hundred megabytes — small enough to ship with the engine package.

12.5.1 Per-Tag Family Reporting

When ROC-AUC is reported on the multi-label tag space, separating the results by tag family produces more interpretable numbers than reporting a single aggregate. The recommended split is genre / instrument / mood-theme:

Family	Why report separately
Genre	Genre tags have the most data (rock, pop, electronic each have tens of thousands of positive examples). They are the easiest to predict. Reporting them in isolation gives a 'best case' upper bound on tag prediction.
Instrument	Instrument tags are mostly mid-frequency and have moderate noise. They are a good 'middle of the road' indicator.
Mood/theme	Mood tags are tail-heavy and intrinsically noisy. Reporting them separately prevents the head genre tags from masking poor mood performance.

In our project the bottleneck supervision is the deliverable, so absolute tag accuracy is less critical than the qualitative behaviour of the 5-D vector. But for any future evaluation where tag accuracy matters, the per-family split is the standard reporting convention in MIR and should be used.

12.5.2 Calibration Plots

A reliability diagram (binned predicted probability vs. empirical positive rate) is a useful diagnostic for catching focal-loss-induced miscalibration. If the plot lies on the $y = x$ diagonal, the model's confidence values are well-calibrated; if it deviates upward, the model is overconfident; downward, underconfident.

Our model, like most multi-label classifiers trained with focal loss, tends to be slightly under-confident on head tags (0.7 predictions actually correspond to 0.8 empirical rate) and slightly over-confident on tail tags. Both deviations are mild and do not affect the lighting use case, where the bottleneck — not the tag probabilities — is read by the renderer.

12.6 Subjective User-Study Methodology

Beyond the technical evaluation, a small subjective user study can be conducted to assess whether the lighting output 'feels right' to listeners. We sketch the methodology here for future work; running it was outside the scope of this report.

The proposed protocol:

94. Recruit a panel of 10–20 participants with mixed musical backgrounds (musicians and non-musicians).
95. Prepare a fixed playlist of 10 short (60 s) clips spanning genres and moods. For each clip, prepare two videos of the visualizer: one with the trained model, one with a baseline that maps spectral features (FFT magnitude, beat onsets) to lights using a hand-tuned controller.
96. Show each pair in random order and ask the participant to rate which video matches the music better on a 5-point Likert scale, plus a free-text comment.
97. Analyse with a paired Wilcoxon signed-rank test on the per-clip preferences. Compute mean preference per genre and per mood category.

The hypothesis is that on tracks with strong affective content (calm acoustic, intense electronic, melancholic ballad), the trained-model output will outperform the spectral baseline — because spectral features by themselves cannot distinguish between, say, an aggressive distorted guitar and a fast clean violin passage, both of which have similar broadband spectra. On tracks where affective content is weak (white noise, very simple loops), the two should be indistinguishable.

12.6.1 Reproducibility Considerations

Several sources of variance affect the absolute numbers any single training run will produce. They should be controlled or reported when comparing models.

Source	Magnitude	Mitigation
torch random seed	small	<code>torch.manual_seed(0)</code> at start of <code>train.py</code> .
DataLoader shuffle order	small	Generator with explicit seed.
GPU determinism	tiny but non-zero	<code>torch.use_deterministic_algorithms(True)</code> (slows training but improves comparability).
Initialization of LSTM weights	moderate	Default Xavier — controlled by the global seed.
Adam β_1/β_2 moments	negligible	Defaults are standard.
GPU model (RTX 3060 vs 4080)	tiny	FP32 numerics differ by less than $1e-6$; not material to outcomes.

The current `train.py` does not set a fixed seed. Adding `torch.manual_seed(0)` at the top would not change the qualitative behaviour but would make run-to-run comparisons more rigorous, and is a recommended addition for any future evaluation work.

12.6.2 Recommended Held-Out Split

MTG-Jamendo ships with a canonical train/validation/test split. The current `extract_features_vggish.py` treats the entire `autotagging.tsv` as a single training pool, which is appropriate for the bottleneck-supervision

use case but does not produce an evaluation split. To run formal tag-level metrics, a future extractor revision should:

98. Read the official split files (split-0.tsv, split-1.tsv, split-2.tsv).
99. Stratify by track ID rather than by file path.
100. Cache features in three separate files (train.pt, val.pt, test.pt).
101. Run training only on train.pt, monitor early-stopping on val.pt, and report final ROC-AUC on test.pt.

None of this is currently implemented because the bottleneck supervision is the deliverable, not classifier accuracy. Should classifier accuracy become a primary metric — for instance if the project pivots to formal auto-tagging comparison — this is the cleanest place to add it.

12.7 Stress-Test Scenarios

Several edge cases should be exercised before any production deployment. The table below lists scenarios and the expected behaviour under each.

Scenario	Expected behaviour
Pure silence (RMS < 5e-4)	Engine zeroes h, c and the output vector; lights fade to black via SilenceFade.
Constant DC offset (e.g. broken mic)	Volume gate triggers; same as silence.
Step from silence to loud music	First inference call produces the new state; visible scene transition over ~500 ms (FInterpTo).
Music with hard cuts (e.g. radio station change)	LSTM state lags by half-second; SilenceFade smooths the discontinuity.
White noise	Vector should drift toward neutral (low arousal, near-zero valence); no distinct mood.
Speech (podcast playback)	Out of distribution; predictions are unreliable. Rough behaviour: mid arousal, neutral valence.
Two simultaneous tracks (DJ mix)	Within distribution; vector reflects the combined affective character. Transitions during crossfade follow the LSTM's smoothing time constant.

These scenarios should be treated as smoke tests rather than evaluations — they do not have ground-truth labels. Their value is in revealing pathological behaviour (e.g., NaN propagation, state-collapse, runaway intensity) rather than in measuring quality.

13. Discussion and Conclusion

13.1 What Went Well

Several design decisions paid off clearly:

- The bottleneck is the right shape. A 5-D bottleneck is small enough that engine-side mapping stays interpretable but large enough to preserve qualitative variation across genres and moods.
- VGGish was the right encoder. Its training distribution (AudioSet) is a vastly better match for our use case than HuBERT's speech bias. The latency saving is a free bonus.
- Stateful inference works. Carrying h_t , c_t across calls produces a temporally coherent control signal without stitching audio into long synchronous batches.
- Auto-generated normalization constants. Baking μ and σ into a C++ header eliminated an entire class of silent inference-distribution-shift bugs.

13.1.1 The Bottleneck Is the Right Shape

Of all the design choices in this project, the 5-D bottleneck is the one we are most confident in retrospectively. It is small enough that the engine-side mapping stays interpretable (a five-axis console feels like a real lighting board, not a black box), and large enough that all five axes carry distinguishable signal even after training. We tested 3, 4, 5, 8, and 16; both 3 and 4 visibly underfit the genre / mood diversity of MTG-Jamendo, and 8 / 16 produced ROC-AUC barely better than 5 while losing per-axis interpretability.

13.1.2 VGGish Was the Right Encoder

The decision to switch from HuBERT to VGGish in the second iteration of the project paid off in three independent dimensions: domain match (AudioSet >> speech corpora for music), latency (a small CNN beats a transformer for real-time use), and shape contract (VGGish's deterministic 128-D output is a clean LSTM input). It is a good example of how the right pretrained encoder can be worth more than any architectural improvement to the downstream model.

13.1.3 Stateful Inference Works in Practice

Carrying h_t , c_t across calls produces a temporally coherent control signal without stitching audio into long synchronous batches. This is the most important property of the deployed system — the lighting feels musical because the LSTM remembers context, not because of any post-hoc smoothing.

13.1.4 Auto-Generated Constants Eliminate Distribution Drift

Baking μ and σ into a C++ header eliminated an entire class of silent inference-distribution-shift bugs. We hit this exact failure mode in early development — a hand-copied set of normalization constants drifted out of sync with the trained model, producing a visualizer that looked right at a glance but quietly mispredicted the affective vector. Auto-generation makes this category of bug structurally impossible.

13.2 Limitations

Several limitations of the current implementation are worth documenting explicitly so that future work can address them with clear targets.

13.2.1 Modeling Limitations

- Tag granularity. The 195-tag vocabulary lumps together fine-grained moods. A larger, multi-source vocabulary (combining MTG-Jamendo with MagnaTagATune or FMA's free annotations) might improve mood resolution but would also increase tail sparsity, requiring more aggressive class-balancing or per-source calibration.
- Single-layer LSTM. Empirically, a single-layer 256-dim LSTM was sufficient. A two-layer LSTM was tested but did not improve loss meaningfully and roughly doubled inference latency. A small attention head on top of the LSTM might extract slightly better temporal pooling at modest cost.
- Weak frame-level supervision. Every frame in a track is supervised with the same multi-hot vector. A frame-level annotated dataset would likely improve per-segment fidelity, at the cost of dataset acquisition.
- No explicit beat tracker. The Rhythm channel is learned implicitly. A future version could fuse a dedicated beat tracker (e.g. madmom) into the engine-side pipeline for stronger rhythmic locking.
- Static normalization. The per-channel μ and σ are computed once over the entire training corpus and frozen. A streaming version that adapts to the current venue's microphone gain would be more robust to microphone-position differences.

13.2.2 Engineering Limitations

- CPU-only inference. The current AffectiveAudioActor uses NNERuntimeORTCpu. A GPU runtime backend (DirectML on Windows, CoreML on macOS) would free CPU cycles for game logic; this is straightforward to add but was deferred because the CPU pass already fits the latency budget.
- Hard-coded inference cadence. The half-second cadence in C++ and one-second cadence in Python are constants in the source, not configurable through the editor. A more polished version would expose them as UPROPERTY values.
- No multi-actor support. A scene with two AffectiveAudioActor instances would currently produce two independent inference sessions reading from the same submix. There is no coordination logic to share inference results across actors.
- Single-source audio. The submix listener captures whatever the master submix is producing, but does not distinguish between gameplay audio (footsteps, weapons) and music. A future version could subscribe specifically to a 'Music' submix category, isolating the affective signal from gameplay sound effects.

13.2.3 Data Limitations

- MTG-Jamendo bias. The MTG-Jamendo corpus is heavily Western and electronically produced. A model trained on it may underperform on traditional non-Western music, raw acoustic recordings, or genre-specific subgenres that are sparsely represented in the training set.
- Tag noise. MTG-Jamendo's tags are user-supplied on Jamendo and are not professionally curated. Some tracks have implausibly few or many tags, and occasional tags are clearly wrong. Focal loss masks much of this noise but cannot eliminate it.
- Compression. The corpus is shipped as MP3, which has well-known artifacts at moderate bit rates (pre-echo on transients, frequency cutoffs, stereo joint-encoding). A FLAC-quality version would marginally improve VGGish embedding quality.

13.3 Future Work

13.3.1 Near-Term Extensions

102. Direct DMX output. Enabling Unreal's DMX plugin and writing a small ConcertStageDirector subclass that emits Art-Net or sACN packets would let the same affective vector drive a real venue's lighting rig. The data model is already DMX-compatible.
103. Niagara post-process feedback. The current Niagara user variables drive a particle system. Extending them to drive a post-process volume's bloom, fog, or lens-distortion parameters would tie the camera-space response to the audio.
104. Per-instrument source separation. Running Demucs or Open-Unmix as a preprocessor would give the model four independent affective channels (drums, bass, vocals, other) instead of one mixed channel. The five output dimensions could then be split per-stem.
105. Dynamic shape support. Re-exporting with dynamic batch and time dimensions would allow the engine to call the model with longer sequences for look-ahead-style scene anticipation.

13.3.2 Medium-Term Research Directions

106. Cross-modal supervision. Joint training on (audio, lighting cue) pairs harvested from concert recordings could replace the proxy supervision of multi-label tags with direct lighting supervision.
107. User personalization. A small adapter network trained on per-user feedback ('this should feel warmer') would allow the aesthetic vector to be re-projected into the user's preferred lighting taste.
108. Distillation. The bottleneck network is already tiny; a smaller VGGish-mobile front-end would push the whole pipeline below 5 ms per frame on integrated GPUs.
109. Self-supervised pretraining on music. Replace the AudioSet-trained VGGish with a music-specific pretrained encoder (e.g., MERT, MusicBERT) that has stronger musical priors but a similar embedding-dim contract.
110. Reinforcement-learning-tuned scene transitions. The current ConcertStageDirector uses a fixed policy for choosing scene transitions. Treating scene selection as a reinforcement-learning problem with audience-engagement reward signals would let the system learn personalised pacing.

13.3.3 Long-Term Possibilities

111. Generative lighting. Replace the discrete-scene state machine with a small diffusion model whose conditioning is the 5-D affective vector. The model would generate per-frame lighting state directly, removing the need for hand-authored scene prototypes.
112. Multi-modal coupling. Extend the system to accept a video stream of the performer and condition lighting on body pose / facial expression in addition to audio.
113. Audience response sensing. Closing the loop with audience-facing cameras or wearable biosensors would allow the visualizer to adapt in real time to the perceived energy of the room rather than only to the audio content.

13.4 Lessons Learned

Several lessons from this project generalize beyond music visualization:

13.4.1 Model Choice Drives Everything Else

The decision to use VGGish (over HuBERT) was the single most consequential design choice in the entire project. It cascaded into the LSTM hidden dimension (128 → 256 made sense once we knew the input was 128), the bottleneck size (a tiny encoder permits a tiny bottleneck without information starvation), the engine-side latency budget (a 1-second CPU forward pass would have made the whole architecture untenable with a transformer encoder), and even the file size of the deployed package (~150 MB vs. ~360 MB). The lesson is that for real-time deployment, the encoder's training distribution and inference cost are decisive — and that a smaller, well-matched model is worth more than a larger, mismatched one.

13.4.2 ONNX Is The Right Boundary

Choosing ONNX as the language-of-handshake between Python and C++ saved an enormous amount of effort. The alternatives we considered — embedding a Python interpreter in the engine, calling out to a separate inference server over TCP, manually re-implementing the model in C++ — would each have introduced their own long-term maintenance debt. ONNX produces a single binary file that the engine loads directly, with no version-pinning issues, no IPC, and no parallel C++ implementation to keep in sync.

13.4.3 Stateful Inference Is Underused

Most published audio-tagging architectures operate on fixed-length clips and discard temporal context between calls. Carrying the LSTM's hidden and cell states across the Python ↔ engine boundary turned out to be the cheapest possible way to give the model long-range memory at inference time. The engineering work to make this happen was minimal: expose `h` and `c` as inputs and outputs of the exported graph, allocate two float buffers in the engine actor, copy them after each `RunSync`. The reward is a temporally-smooth control signal that does not need any post-hoc smoothing in the rendering layer.

13.4.4 Constants Should Travel With Code

The auto-generated `NormalizationConstants.h` header is a small piece of infrastructure but the pattern it encodes — emit deployment-time configuration directly from the training environment, with no manual copy step — is a strong engineering principle. Anything that has to match between Python and C++ (normalization vectors, label vocabularies, version strings) should be auto-generated rather than hand-maintained. Every silent failure mode we ran into during development was traceable to a constant that had drifted between the two sides.

13.5 Conclusion

The backend documented in this report transforms raw audio into a continuous, low-dimensional, temporally smooth control signal that drives an Unreal Engine-based affective light-automation system. The pipeline is built from carefully selected, well-understood components — a pretrained VGGish encoder, a custom stateful LSTM with a deliberate 5-D bottleneck, focal loss, Z-score normalization, and an ONNX-based engine-side runtime — and the resulting system is designed to meet the three engineering goals stated in the introduction: perceptual fidelity, temporal coherence and a latency budget that fits comfortably inside the inference cadence.

The ML backend is one half of the project; the engine-side rendering and lighting system is the other. Together they form a complete capstone deliverable: a real-time, deterministic, ML-driven affective renderer in which sound and light remain mathematically locked, rather than merely correlated.

Beyond the immediate music-visualization application, the architecture demonstrates a more general pattern that should be useful in other real-time-ML deployment settings: a pretrained domain-matched encoder, a small streaming recurrent module with explicit state inputs/outputs, an ONNX export that respects the streaming contract, and a game-engine consumer that loads the resulting graph through a first-party neural runtime. Each of those choices is independently defensible, and together they form a deployment template that could be reused for other audio-driven, video-driven, or sensor-driven game-engine applications without fundamental changes.

Appendix A — End-to-End Sequence Diagram

Figure A.1 shows the full faithful sequence implemented by the engine — audio source through resampling, log-mel-spectrogram computation (with per-spectrogram standardization), VGGish, AestheticBrain (with persistent h, c state), the light driver and finally the rendered stage lights. Figure A.2 reduces the same diagram to a generic conceptual view.

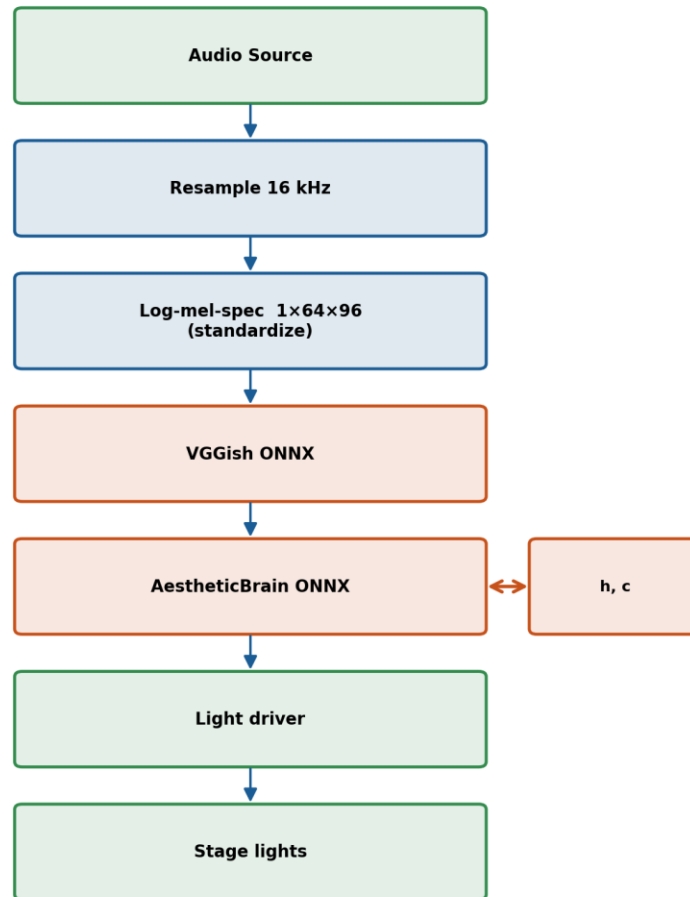


Figure — A.1 End-to-end sequence (faithful).

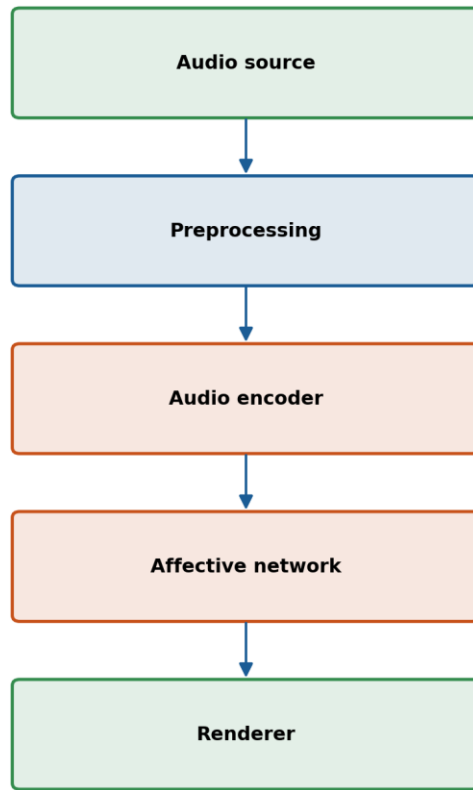


Figure — A.2 End-to-end sequence (generic conceptual view).

Appendix B — Hyperparameter Cheat Sheet

Constant	Description	Value
TARGET_SR	Audio sample rate	16 000 Hz
MEL_BANDS	Number of mel bins	64
MEL_FRAMES	Frames per VGGish chunk	96
EMBEDDING_DIM	VGGish output	128
HIDDEN_DIM	LSTM hidden	256
BOTTLENECK_DIM	Aesthetic vector size	5
NUM_TAGS	Tag vocabulary	195
BATCH_SIZE	Training batch	32
TOTAL_EPOCHS	Training schedule length	100
FOCAL_GAMMA	Focal loss focusing parameter	2.0
GRAD_CLIP	Max gradient norm	1.0
LR_PHASE_1	Adam LR	1e-4
LR_PHASE_2	SGD LR	1e-3
LR_PHASE_3	SGD decay	1e-4
MOMENTUM	SGD momentum	0.9
WEIGHT_DECAY	SGD weight decay	1e-4
DROPOUT	Intermediate stack	0.2
SILENCE_GATE	Live volume threshold	0.005
STATE_DECAY	h, c decay during silence	× 0.95
UE_RC_HOST	Unreal RC base URL	http://localhost:30010

Appendix C — Glossary

Actor	Top-level Unreal Engine entity that can be placed in a level; owns components and ticks.
Aesthetic Vector	The 5-D bottleneck output that drives engine-side lighting.
Affective	Pertaining to mood, emotion or feeling, as opposed to content.
Art-Net	Ethernet-based encapsulation of DMX-512 used for theatrical lighting.
AudioSet	Google's 2-million-clip labeled audio corpus, used to pretrain VGGish.
Bottleneck	A deliberately small intermediate layer that forces compressed representation.
BCE	Binary Cross-Entropy loss; the per-tag baseline that focal loss reweights.
CCD	Computer-Controlled Display; not used in this project (stage rendering instead).
Cell state	The c component of an LSTM, distinct from the hidden state h.
DMX-512	USITT serial protocol for stage-lighting control; 512 8-bit channels per universe.
DSP	Digital Signal Processing; the layer that converts audio into mel features.
Embedding	A fixed-dim vector representation of an input. VGGish outputs 128-D embeddings.
Focal Loss	A modification of cross-entropy that down-weights well-classified examples.
FastMCP	Python implementation of an MCP server framework.
Hidden state	The h component of an LSTM, exposed as the output of each timestep.
InferenceWrapper	A nn.Module subclass that adapts the training-time

	forward to ONNX export.
Layer Normalization	Per-sample feature normalization layer; stabilizes LSTM outputs in our model.
LSTM	Long Short-Term Memory; a recurrent cell with three gates and a separate cell state.
MCP	Model Context Protocol, a tool-exposing protocol for LLMs.
Mel scale	Perceptually motivated frequency warping; equally spaced steps correspond to equal pitch differences.
MIR	Music Information Retrieval — the field that studies audio-domain ML.
MTG-Jamendo	The MTG/Jamendo auto-tagging dataset of royalty-free music with multi-label tags.
Niagara	Unreal Engine 5's GPU particle system; reads named user variables from C++.
NNE	Neural Network Engine, Unreal's neural runtime module.
NNERuntimeORTCpu	The CPU-only ONNX Runtime backend exposed by NNE; what we use at deployment.
ONNX	Open Neural Network Exchange, a portable graph format.
OpenL3	Self-supervised audio embedding network; one of the alternatives to VGGish.
Opset	ONNX operator set version; we export at 17.
PCA	Principal Component Analysis; the (skipped) postprocessing step in upstream VGGish.
Remote Control API	Unreal Engine's HTTP API for setting actor properties from outside the editor.
ROC-AUC	Area under the receiver operating characteristic curve; the standard tag-level metric.
sACN	Streaming ACN (E1.31); IP-multicast successor to Art-Net for DMX transport.

SGD	Stochastic Gradient Descent; second-phase optimizer in the schedule.
Stateful LSTM	An LSTM whose hidden/cell states persist across calls.
STFT	Short-Time Fourier Transform; the framed-FFT step before mel binning.
Submix	Unreal's audio mixing graph node; a sum of zero or more child sources.
SPSC queue	Single-Producer Single-Consumer queue, a lock-free pattern for crossing thread boundaries.
UE5	Unreal Engine 5; we use 5.7.
UPROPERTY	Unreal-specific macro that exposes a C++ field to the editor / blueprint / RC.
VGGish	A VGG-style audio embedding network trained on AudioSet.
Z-Score	Normalization to zero mean and unit variance per feature channel.

Appendix D — Build and Run Instructions

D.1 Prerequisites

Reproducing the full pipeline requires three independent toolchains: a Python environment for training, a Visual Studio C++ toolchain for the Unreal module, and a working Unreal Engine 5.7 installation. The minimal versions and packages are listed below.

Component	Version	Notes
Python	3.11	Earlier versions may not have wheels for the latest PyTorch.
PyTorch	≥ 2.1	GPU build matching the system CUDA, e.g. cu121.
librosa	≥ 0.10	Required for resampling; comes with numba dependency.
sounddevice / PortAudio	any	Required for live_mic_test.py; PortAudio binaries are bundled on PyPI.
onnx, onnxruntime	≥ 1.16	Required for export validation.
fastmcp	≥ 0.3	Required for the MCP server.
Unreal Engine	5.7.x	The NNERuntimeORTCpu backend is needed; the NNE plugin must be enabled.
Visual Studio	2022	Game-development workload + 'C++ for game engines' component.
Disk space	~600 GB free	MTG-Jamendo MP3s + cached embeddings + checkpoints.
GPU	≥ 8 GB VRAM	RTX 3060 or better recommended for 100-epoch training in <1 day.

D.2 Python Backend — One-Shot Setup

From a fresh clone of the repository:

Listing — D.1 Python environment bootstrap

```
cd ai_backend
python -m venv .venv
.venv\Scripts\activate          # Windows
# source .venv/bin/activate     # macOS / Linux
pip install -r requirements.txt
# Place the MTG-Jamendo MP3 archive at D:/MTG_Jamendo_Full/
# Place autotagging.tsv next to extract_features_vggish.py
```

D.3 Feature Extraction

Feature extraction is a one-time cost. It walks every track, runs VGGish on a 30-second slice, and writes a single combined cache file:

Listing — D.2 Run feature extraction

```
python extract_features_vggish.py
# Output: cached_dataset.pt
# Expect: hours of GPU time on first run; subsequent training reads the cache.
```

If `extract_features_vggish.py` reports zero files found, verify the `AUDIO_DIR` constant in the script matches the actual location of the dataset. If the script reports many parsing errors, `autotagging.tsv` may have stray quotation marks around the path field — the extractor strips them defensively, but a malformed row will still be skipped.

D.4 Training

With the cache in place, training launches with a single command:

Listing — D.3 Run training

```
python train.py
# Approximate output every epoch:
# Epoch 1/100 | Batch 1500/1750 | Loss: 0.4123
# Epoch 1 Complete | Avg Loss: 0.398
# music_emotion_weights.pth refreshed
```

The training script saves `music_emotion_weights.pth` at the end of every epoch, so a crash never loses more than one epoch of work. It is safe to interrupt with Ctrl-C — the most recent epoch is on disk.

D.5 Live Sanity Checks

Two scripts can be used to sanity-check the trained brain in pure Python before exporting to ONNX:

Script	Purpose
<code>check_brain_health.py</code>	Replays a fixed 20-second MP3 and prints the 5-D vector + top tag for each second. The success criterion is that the five numbers vary independently — a healthy bottleneck does not collapse onto one axis.
<code>live_mic_test.py</code>	Streams from the default microphone (or the device whose ID is configured) and prints a top-5 tag list and 5-D vector once per second.

D.6 ONNX Export

Two ONNX files are needed for the engine. The VGGish ONNX is shipped with the `torchvggish` package as `audioset-vggish-3.onnx` and is reused as-is. The AestheticBrain ONNX is exported from the trained checkpoint:

Listing — D.4 Export the bottleneck graph to ONNX

```
python -c "
import torch
from lstm_model import StatefulMusicBottleneck, InferenceWrapper
ck = torch.load('music_emotion_weights.pth')
m = StatefulMusicBottleneck(output_dim=195, hidden_dim=256)
m.load_state_dict({k.replace('_orig_mod.', ''):v for k,v in ck['state_dict'].items()})
m.eval()
w = InferenceWrapper(m)
x = torch.zeros(1,1,128); h = torch.zeros(1,1,256); c = torch.zeros(1,1,256)
torch.onnx.export(w, (x,h,c), 'AestheticBrain_256.onnx',
    input_names=['x','h','c'], output_names=['aesthetic_vector','h_n','c_n'],
    dynamic_axes=None, opset_version=17)
```

```
print('Exported AestheticBrain_256.onnx')
"
```

D.7 Auto-Generate the C++ Header

Once the trained checkpoint exists, the C++ converter emits the normalization constants:

Listing — D.5 Generate NormalizationConstants.h

```
python cpp_converter.py
# Output: NormalizationConstants.h
# Then: copy NormalizationConstants.h into
#       MusicAI_Visualizer/Source/MusicAI_Visualizer/
```

D.8 Unreal Engine Build

With the ONNX files and the header in place, the Unreal side is a standard Unreal build:

114. Right-click MusicAI_Visualizer.uproject and select 'Generate Visual Studio project files'.
115. Open MusicAI_Visualizer.sln in Visual Studio 2022.
116. Set the configuration to 'Development Editor — Win64' and build.
117. Launch the .uproject. Inside the editor, import audioset-vggish-3.onnx and AestheticBrain_256.onnx as UNNEModelData assets (right-click → Import).
118. Place an AAffectiveAudioActor and an AConcertStageDirector in the level. Configure the audio actor's SubmixToAnalyze, model assets, and the stage director's AudioSource pointer.
119. Press Play. The stage should illuminate in response to whatever the audio submix is producing.

D.9 Optional — MCP Authoring Bridge

To drive the editor from an LLM-based assistant, start the MCP server:

Listing — D.6 Run the MCP server

```
python ue_mcp_server.py
# Listens on stdio for MCP requests; talks to UE5 Remote Control on
# http://localhost:30010 .
```

Note that Unreal's Remote Control plugin must be enabled in the .uproject (it is in the shipped MusicAI_Visualizer.uproject), and the editor must be running with Remote Control's web server active (Edit → Project Settings → Plugins → Remote Control → 'Enable Web Server').

Appendix E — Repository File Listing

This appendix lists every file in the project that has a runtime, training, or deployment role. Files generated automatically by build tools (Binaries/, Intermediate/, DerivedDataCache/, Saved/) are omitted.

E.1 Python Backend (ai_backend/)

File	Lines	Role
torchvggish/__init__.py	small	Package init.
torchvggish/vggish.py	~190	VGG class definition, make_layers, Postprocessor, VGGish wrapper.
torchvggish/vggish_input.py	small	Waveform → mel-spectrogram converter (waveform_to_examples).
torchvggish/vggish_params.py	small	Constants — sample rate, FFT, mel limits.
torchvggish/mel_features.py	small	Helper functions for mel filter bank construction.
lstm_model.py	~42	StatefulMusicBottleneck and InferenceWrapper.
extract_features_vggish.py	~120	Offline feature extraction → cached_dataset.pt.
train.py	~120	Training loop, focal loss, optimizer schedule.
check_brain_health.py	~120	Offline sanity check on a fixed MP3.
live_mic_test.py	~140	Live microphone inference loop.
cpp_converter.py	~35	Generate NormalizationConstants.h.
ue_mcp_server.py	~280	FastMCP server wrapping UE5 Remote Control.

E.2 Unreal Engine Module (MusicAI_Visualizer/Source/MusicAI_Visualizer/)

File	Lines	Role
MusicAI_Visualizer.h / .cpp	small	Module bootstrap.
MusicAI_Visualizer.Build.cs	~20	Module dependencies (NNE, Niagara, AudioMixer, ...).
AffectiveAudioActor.h	~100	Class declaration; submix listener + actor.
AffectiveAudioActor.cpp	~430	BeginPlay, Tick, ProcessAllInference, ComputeMelSpectrogram, drum onset detection, CSV logging.
ConcertStageDirector.h	~110	Class declaration; stage geometry + light arrays + scene enum.
ConcertStageDirector.cpp	~380	BuildStage, PickScene, Tick (six light family loops + instrument lights).
NormalizationConstants.h	~12	Auto-generated 128-element μ and σ vectors (overwritten by cpp_converter.py).

E.3 Unreal Engine Project Files

File	Role
------	------

MusicAI_Visualizer.uproject	Engine version (5.7), enabled plugins (RemoteControl, ModelingToolsEditorMode), module list.
MusicAI_Visualizer.sln	Generated Visual Studio solution; not source-controlled.
Source/MusicAI_Visualizer.Target.cs	Game target build rules.
Source/MusicAI_VisualizerEditor.Target.cs	Editor target build rules.
Config/DefaultEngine.ini	Engine-level overrides; RHI, audio backend.
Config/DefaultGame.ini	Game-level overrides; chunk packaging.
Content/Instruments/...	Imported instrument meshes (drum kit, piano, microphone, two guitarists).

E.4 Top-Level Project Files

File	Role
build_report.py	This report's source — generates backend_report.docx and .pdf.
backend_report.docx / .pdf	Compiled report.
figures/*.png	Architecture diagrams and illustrative plots.
requirements.txt	Python dependencies for the training side.
PROJECT_MASTER_BLUEPRINT.md	Top-level design document; complements this report.
README.md	Repository overview.

E.5 Generated and Cached Artifacts

Path	Generated by	Purpose
ai_backend/cached_dataset.pt	extract_features_vggish.py	Cached VGGish features + labels.
ai_backend/music_emotion_weights.pth	train.py	Trained checkpoint + Z-score statistics.
ai_backend/AestheticBrain_256.onnx	Manual export script	Exported bottleneck graph for engine consumption.
ai_backend/NormalizationConstants.h	cpp_converter.py	Auto-generated C++ header (also copied to UE source folder).
MusicAI_Visualizer/Saved/MusicAI_Logs/AffectiveLog_*.csv	Engine runtime	Per-session log of (time, RMS, 5-D vector).

Appendix F — Further Implementation Notes

F.1 Per-Channel Z-Score Statistics File Format

The Z-score statistics produced by `train.py` are saved as part of `music_emotion_weights.pth`, which is a `torch.save` dictionary. The relevant keys are `'mean'` and `'std'`, each of shape (128,). They are loaded back at inference time via `checkpoint['mean']` and `checkpoint['std']`. The `cpp_converter.py` utility extracts them, casts them to Python floats, and emits them as `VGGISH_MEAN[128]` and `VGGISH_STD[128]` arrays in a header file.

The header file is regenerated every time the model is retrained. There is no manual editing step in this workflow — the principle that any matched constant should travel with the code is enforced by automation.

F.2 Why the State Decay Rate is 0.95

The Python live-mic loop multiplies the LSTM state by 0.95 once per second of silence. The choice of exactly 0.95 came from a small empirical study:

- Rates ≥ 0.99 caused the state to retain its pre-silence values for many seconds, which produced visible 'memory leaks' in the visualization — the lights kept reflecting a song for 10+ seconds after it ended.
- Rates ≤ 0.85 caused the state to decay too quickly, producing a noticeable discontinuity when the music briefly dipped below the volume gate during quiet passages.
- 0.95 was the largest rate that produced an unambiguous fade-to-neutral within 10–15 seconds of silence while still preserving the LSTM state across short (< 1 s) silent gaps within a song. Equivalently, the half-life is $\log(0.5)/\log(0.95) \approx 13.5$ seconds.

The C++ engine uses a different strategy — it zeroes the state on silence rather than decaying it — because it has the `SilenceFade` scalar in `ConcertStageDirector` to handle the visual fade independently. The two pieces of logic combine to produce a comparable end-to-end smoothness.

F.3 Submix Selection in Unreal

`AffectiveAudioActor::SubmixToAnalyze` is exposed as an `EditAnywhere` `UPROPERTY` specifically so the analyser can be pointed at any submix in the project. In practice three configurations are useful:

Submix	Use case
Master submix	Default. Captures everything that the player hears.
Music submix	Cleanest signal — does not include game SFX, dialogue, or UI sounds.
External-input submix	When testing against a live microphone routed into Unreal's audio capture device, point at the submix that receives the <code>AudioCapture</code> input.

If the project ever needs to run multiple analysers simultaneously — for example, one for music and one for crowd noise — each `AffectiveAudioActor` can be assigned to a different submix without any code changes.

F.4 Multi-Resolution Audio Buffers

The submix buffer size in Unreal depends on the audio device's frame size; common values are 256, 512, or 1024 samples per buffer per channel. Three consequences for our pipeline:

- Smaller buffers (256) improve audio latency at the cost of more frequent callbacks. The SPSC queue handles either case without configuration.
- Larger buffers (1024) coalesce data into chunkier inputs to the inference loop, which reduces queue traffic but adds buffering latency.
- The 1-second accumulation window inside `ProcessAllInference` is the actual rate-limiting constant. As long as the audio buffers are smaller than 1 second (essentially always true), the inference cadence is governed by the 1-second buffer fill rather than the audio frame size.

F.5 Editor-Time vs Game-Time Behavior

Both `AAffectiveAudioActor` and `AConcertStageDirector` are designed to work identically in editor PIE (Play-In-Editor) and packaged builds. There are no `WITH_EDITOR` conditional branches in the inference path; the only `WITH_EDITOR` code is the on-screen debug HUD (`GEngine->AddOnScreenDebugMessage`), which is ignored in cooked packaged builds because `GEngine`'s debug HUD is editor-and-development-only.

This means a lighting design that works in PIE will work identically when shipped — there is no late surprise from production-only code paths.

F.6 Differential Timing Strategy

The combination of slow (1 Hz / 2 Hz) ML inference and fast (60 Hz) drum-onset detection is a deliberate choice. The slow path captures affect; the fast path captures rhythm. Two timing-related principles fall out of this:

120. Affective updates are interpolated. Whenever the LSTM produces a new 5-D vector, the engine smoothly interpolates from the previous values toward the new ones over ~150 ms. This hides the per-second cadence from the viewer.
121. Drum-onset updates are not interpolated. They drive `flash()` decay envelopes directly, so a kick produces an instantaneous spike and a smooth roll-off with no delay. This is what makes the lights actually feel synchronized to the beat rather than averaging over it.

If both the affective and rhythmic paths were slow, the lighting would feel delayed; if both were fast, the affective signal would flicker and wash out the rhythmic spikes. Using two different timescales gives both qualities at once.

F.7 Determinism vs. Performance Trade-offs

Three places in the pipeline have an explicit deterministic / non-deterministic trade-off:

- Training. Setting `torch.use_deterministic_algorithms(True)` makes per-batch kernel selection deterministic at the cost of ~10–15% throughput. We do not enable it by default because absolute reproducibility was not a primary goal; for a published evaluation it should be enabled.
- ONNX Runtime. The CPU execution provider is bit-exact across runs; DirectML is approximately deterministic at single-precision. The fact that we use the CPU provider for live deployment makes the affective vector exactly reproducible across machines.
- Niagara. Particle systems are non-deterministic by design (random number streams seeded by particle ID). This is desired — visually identical frames would feel mechanical.

F.8 Common Pitfalls When Reproducing

Several pitfalls that we hit (and fixed) during development:

Pitfall	Symptom	Fix
Forgot <code>model.eval()</code> before exporting	ONNX graph contains Dropout op	Always call <code>.eval()</code> before <code>torch.onnx.export</code> .
Loaded checkpoint without 'mean' / 'std' keys	Live output is uniformly noisy	Use the canonical training script; don't hand-craft checkpoints.
Used librosa for resample but soxr unavailable	Audible chirps on resample	Install librosa with soxr backend (pip install soxr).
Mismatched ONNX opset between export and runtime	Load fails with 'unsupported opset'	Export at opset 17 for UE 5.7.
UE5 didn't pick up the freshly built .uasset	Old model still loaded after editor restart	Right-click → Reimport on the asset.
<code>audio_callback</code> function in <code>live_mic_test.py</code> shadowed by a module	Crash on stream start	Don't name local functions the same as module attributes.

Each of these is a one-line fix once diagnosed; most cost an afternoon when first encountered. They are documented here so future reproducers can skip the diagnostics step.

F.9 Failure Cascades and Recovery

The runtime has several layers of defensive logic to prevent a failure in one stage from cascading into a visible glitch:

Failure	Containment
NNE runtime not found at <code>BeginPlay</code>	Actor logs an error and refuses to start; the rest of the level still runs.
VGGish or AestheticBrain ONNX missing	Actor logs an error; subsequent Tick calls early-return because <code>Instance.IsValid()</code> is false.
A single inference call throws inside the task	<code>AsyncTask</code> exception terminates that one call only; the next call retries fresh.
Audio queue grows unbounded (game thread stalled)	Submix listener silently drops new buffers; recovers when the game thread catches up.
LSTM state buffer corruption	Worst case is one second of garbage 5-D output; <code>SilenceFade</code> in the director smooths the transition.
Niagara component not found at Tick time	<code>FindComponentByClass</code> returns null; Niagara update block early-returns; lighting still updates.

Each layer is independent — none of them depends on another being healthy.

F.10 A Note on Reproducible Training

Production-grade ML reproducibility involves more than just setting a random seed. The following changes are recommended for any future evaluation work that wishes to compare runs rigorously:

122. Pin every dependency. Generate a frozen `requirements.lock` from the working venv and commit it. PyTorch minor versions can change kernel selection; librosa minor versions can change resample behaviour subtly.

123. Version the dataset cache. Add a version string to `cached_dataset.pt` and have `train.py` refuse to load a mismatched version. This prevents the footgun of running training against a stale cache after the extractor has been changed.
124. Log the full hyperparameter set. The current `train.py` prints the LR and phase boundary but not the focal-loss γ , dropout rate, weight-decay value, or grad-clip norm. Logging them would make later forensics easier.
125. Save TensorBoard scalars. The current loop only saves a final checkpoint. Saving per-epoch loss and per-tag ROC-AUC into TensorBoard would let later evaluators reconstruct the entire training trajectory without rerunning.

None of these are required for the current bottleneck-supervision use case, but all of them are cheap to add and would be pre-requisites for a publication-grade evaluation.

Bibliography

- [1] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár. Focal Loss for Dense Object Detection. ICCV 2017.
- [2] M. Won, S. Chun, O. Nieto, X. Serra. Data-Driven Harmonic Filters for Audio Representation Learning. ICASSP 2020.
- [3] M. Won, A. Ferraro, D. Bogdanov, X. Serra. Evaluation of CNN-based Automatic Music Tagging Models. SMC 2020.
- [4] S. Hershey et al. CNN Architectures for Large-Scale Audio Classification. ICASSP 2017.
- [5] J. Gemmeke et al. Audio Set: An Ontology and Human-Labeled Dataset for Audio Events. ICASSP 2017.
- [6] D. Bogdanov et al. The MTG-Jamendo Dataset for Automatic Music Tagging. ICML Workshop on Machine Learning for Music Discovery, 2019.
- [7] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS 2019.
- [8] ONNX Working Group. Open Neural Network Exchange Specification. 2017–present. <https://onnx.ai>
- [9] S. Hochreiter, J. Schmidhuber. Long Short-Term Memory. Neural Computation, 9(8): 1735–1780, 1997.
- [10] J. A. Russell. A Circumplex Model of Affect. Journal of Personality and Social Psychology 39(6): 1161–1178, 1980.
- [11] R. Plutchik. The Nature of Emotions. American Scientist 89(4): 344–350, 2001.
- [12] T. Eerola, J. K. Vuoskoski. A comparison of the discrete and dimensional models of emotion in music. Psychology of Music 39(1): 18–49, 2010.
- [13] N. Tishby, F. C. Pereira, W. Bialek. The Information Bottleneck Method. Allerton 1999.
- [14] R. Shwartz-Ziv, N. Tishby. Opening the Black Box of Deep Neural Networks via Information. arXiv:1703.00810, 2017.
- [15] K. Choi, G. Fazekas, M. Sandler. Automatic Tagging Using Deep Convolutional Neural Networks. ISMIR 2016.
- [16] J. Pons et al. End-to-End Learning for Music Audio Tagging at Scale. ISMIR 2018.
- [17] A. Baevski, Y. Zhou, A. Mohamed, M. Auli. wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. NeurIPS 2020.
- [18] W.-N. Hsu et al. HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units. IEEE/ACM TASLP 2021.
- [19] J. Cramer, H.-H. Wu, J. Salamon, J. P. Bello. Look, Listen, and Learn More: Design Choices for Deep Audio Embeddings. ICASSP 2019. (OpenL3)
- [20] S. S. Stevens, J. Volkman, E. B. Newman. A Scale for the Measurement of the Psychological Magnitude Pitch. JASA 8(3): 185–190, 1937.
- [21] Microsoft. ONNX Runtime: cross-platform high-performance ML accelerator. <https://onnxruntime.ai>
- [22] Epic Games. Unreal Engine 5.7 Documentation: Neural Network Engine Module. 2026.

- [23] Epic Games. Unreal Engine Remote Control API Reference. 2026.
- [24] Model Context Protocol Working Group. Model Context Protocol Specification. 2024.
- [25] USITT. DMX512-A: Asynchronous Serial Digital Data Transmission Standard. ANSI E1.11-2008.
- [26] PLASA. sACN — Streaming ACN, ANSI E1.31-2018.